SOS2901 Maskinlæring for samfunnsvitere

Oppgaver til hver undervisningsuke

Torbjørn Skardhamar

2026-02-23

Table of contents

Introduksjon **5**
Undervisningsvideoer . . . 5

Hva vi forutsetter av forkunnskaper **6**

R og Rstudio **7**
Installasjon . . . 7
Pakker . . . 7
Oversikt over pakker brukt i heftet . . . 8

1 Introduksjon til maskinlæring **10**
1.1 Noen innledende metodiske begrep . . . 10
1.2 Forklaringer og prediksjoner . . . 10
1.3 Overfitting: training og testing dataset . . . 10
1.4 Klassifikasjonsusikkerhet - grunnleggende begreper . . . 13
1.4.1 Confusion matrix: riktig og feil klassifisering . . . 14
1.4.2 Asymetriske kostnader . . . 14
1.5 Rettferdighet og rimelighet . . . 15
1.5.1 Fundamentale skjevheter i data . . . 15

2 Lineær regresjon **16**
2.1 Lese inn data . . . 16
2.1.1 Training og testing data . . . 17
2.1.2 Enkel lineær regresjon i R . . . 18
2.1.3 Multippel regresjon . . . 22
2.1.4 Sammenligning av modeller med ulik kompleksitet . . . 24
2.1.5 Eksempel på overfitting . . . 25
2.1.6 Predikere for nye data . . . 26
2.2 Splines . . . 29
2.2.1 Piecewise linear spline . . . 31
2.2.2 B-spline (kubisk) . . . 32
2.2.3 GAM med smoothing spline . . . 33
2.2.4 GAM med kommunedata . . . 35
2.2.5 Oppsummerende kommentar . . . 38

3 Logistisk regresjon **39**
3.1 Empirisk eksempel: Kategorisk utfall . . . 39
3.2 Estimere en sannsynlighet . . . 43
3.3 Logistisk regresjon i R . . . 44
3.4 Prediksjon . . . 47
3.4.1 ROC og AUC . . . 47
3.5 Multippel logistisk regresjon . . . 48
3.5.1 ROC og AUC . . . 50
3.6 Testing-data . . . 51
3.7 Klassifikasjon . . . 53
3.7.1 Confusion matrix . . . 54
3.8 Hvor feil kan man ta? . . . 55

4 Klassifikasjonstrær **56**
4.1 Lese inn data . . . 56
4.2 Logistisk regresjon som baseline . . . 57
4.3 Klassifikasjonstre . . . 60
4.4 Tuning/pruning . . . 63
4.4.1 Bruk av `cp = ...` . . . 63
4.4.2 Bruk av `maxdepth = ...` . . . 64
4.4.3 Bruk av `minbucket = ...` . . . 64
4.4.4 Sette delene sammen . . . 65
4.4.5 Pruning . . . 68
4.5 Asymetriske kostnader med loss matrix (**Ekstramateriale**) . . . 69

5 Bagging **72**

6 Introduksjon til bagging **73**
6.1 Empirisk eksempel: Syntetiske data . . . 73
6.1.1 Klassifikasjonstre som utgangspunkt . . . 74
6.1.2 Bagging . . . 75
6.2 Eksempel med Attrition-data . . . 77
6.3 Out-of-bag data (OOB) . . . 80
6.4 Mer tuning . . . 80

7 Random forest **82**
7.1 Eksempel med COMPAS-data . . . 82
7.1.1 Variable importance . . . 86
7.1.2 Partial dependence . . . 87
7.2 Tuning av random forest med sampsize . . . 88
7.2.1 Eksempel med COMPAS-data . . . 88
7.2.2 Eksempel med syntetiske data . . . 90

8 Boosting **95**
8.1 Stochastic gradient boosting . . . 96
8.2 Tolkbarhet . . . 99
8.3 Prediksjon og confusion matrix . . . 103
8.4 Asymetriske kostnader . . . 104
8.5 Hva med bias og fairness? . . . 106
8.6 Justere rettferdighet med vekting . . . 107

9 En lett introduksjon til fairness **112**
9.1 Hva slags rettferdighet . . . 112
9.2 Mer confusion matrix . . . 113
9.3 Mål på fairness . . . 116
9.3.1 Predictive rate parity . . . 116
9.3.2 Equalized odds . . . 120
9.3.3 False positive rate parity . . . 121
9.4 Litt oppsummerende . . . 122

10 Fairness og tuning **123**
10.1 Introduksjon til fairness . . . 123
10.2 Fairness: tuning med stratifisering . . . 128
10.2.1 Hva andre subgrupper? . . . 131

11 Oppgaver **136**
11.1 Logistisk regresjon . . . 136
11.2 Klassifikasjonstrær . . . 137
11.3 Bagging . . . 138
11.4 Random forest . . . 138
11.5 Intro til fairness . . . 139
11.5.1 Refleksjonsoppgaver . . . 139
11.6 Fairness og tuning . . . 140
11.7 Boosting . . . 140

References **142**

Introduksjon

Dette dokumentet gir en oversikt over hva vi skal dekke i løpet av semesteret. Det lages delvis underveis, så det vil bli oppdateringer innimellom. Det vil kunne skje endringer i dokumentet underveis gjennom hele semesteret.

Hvert kapittel starter med en introduksjon til temaet og et empirisk eksempel. Datasettene vil dere finne i rommet i Canvas. Last de ned til egen maskin for å jobbe med dem.

Undervisningsvideoer

Det finnes noen undervisningsvideoer fra noen år tilbake. Disse dekker omtrent de vi går gjennom i undervisningen, men er ikke nødvendigvis helt oppdaterte. Dere har tilgang til disse videoene på denne lenken

Hva vi forutsetter av forkunnskaper

Dette kurset forutsetter at man har grunnleggende ferdigheter i kvantitative metoder og har brukt *R* før. Hvis du vet du trenger det: frisk opp litt fra tidligere kurs.

Når det er sagt, så er det begrenset hvor mye man *må* kunne fra før hvis du er motivert til å jobbe med stoffet skal du nok få til dette. Det blir mye nytt uansett.

Hvis du trenger oppfriskning av hvordan R fungerer, så er Wickham & Grolemunds bok "R for data science" et utmerket oppslagsverk. Men merk at vi i begrenset grad vil bruke "Tidyverse" til annet enn datahåndtering og noe grafikk.

R og Rstudio

Installasjon

Du må installere både R og Rstudio på din datamaskin. Hvis du har Windows-maskin trenger du også installere Rtools. Hvis du har Mac OS X kan det hende du må installere XQuartz.

Se ellers video fra SICSS https://youtu.be/ulIv0NiVTs4

Pakker

R er basert på bruk av "pakker" som må installeres for å få tilgang til funksjoner vi skal bruke. Disse installeres med bruk av kommandoen `install.packages()`. F.eks. kan man installere pakken Tidyverse med følgende: `install.packages("tidyverse")`.

For å installere flere pakker kan man kjøre `install.packages()` flere ganger, men det er enklere å liste opp alle pakkene i et objekt og så kjøre `install.packages()` på dette objektet. Noe slikt:

Vi skal i hvert fall bruke de pakkene som installeres med følgende kode:

```
pkgs <- c("tidyverse", "randomForest", "rpart", "rpart.plot",
           "rsample", "fairness", "xgboost", "mgcv", "splines", "Ecdat",
           "ipred", "pROC", "caret", "forcats", "gtsummary", "janitor",
           "gbm", "dendextend", "directlabels", "GGally")
install.packages(pkgs)
```

Det kan hende vi kommer til å bruke flere etterhvert.

Men for at du skal kunne bruke pakkene må du aktivere dem i R ved `library()`. Dette må du gjøre hver gang du starter opp R. Hvert kapittel starter med en oversikt over hvilke pakker som brukes i det kapittelet.

Oversikt over pakker brukt i heftet

Tabellen under gir en oversikt over alle pakkene vi bruker, med en kort forklaring og de viktigste funksjonene.

Pakke	Beskrivelse	Viktige funksjoner	Kapittel
`tidyverse`	Samling av pakker for datahåndtering og grafikk	`mutate()`, `select()`, `filter()`, `ggplot()`, `glimpse()`	Alle
`rsample`	Splitting av data i training/testing	`initial_split()`, `training()`, `testing()`	Alle supervised-kapitler
`mgcv`	Generaliserte additive modeller (GAM)	`gam()`, `s()`	Lineær regresjon
`splines`	Spline-funksjoner for fleksibel regresjon	`bs()`	Lineær regresjon
`Ecdat`	Datasett for økonometri (Clothing-data)	`data(Clothing)`	Lineær regresjon
`pROC`	ROC-kurver og AUC-beregning	`roc()`, `auc()`	Logistisk regresjon
`caret`	Confusion matrix og modellevaluering	`confusionMatrix()`	Logistisk regresjon, CART, Bagging, Random forest, Boosting
`rpart`	Klassifikasjonstrær (CART)	`rpart()`, `prune()`	CART, Bagging
`rpart.plot`	Plotting av klassifikasjonstrær	`rpart.plot()`, `rpart.rules()`	CART, Bagging
`ipred`	Bagging av klassifikasjonstrær	`bagging()`	Bagging
`forcats`	Rekoding og håndtering av factor-variable	`fct_lump()`, `fct_recode()`	Bagging
`randomForest`	Random forest-algoritmen	`randomForest()`, `varImpPlot()`, `partialPlot()`	Random forest
`fairness`	Mål på rettferdighet/bias	`acc_parity()`, `fnr_parity()`, `fpr_parity()`	Fairness, Boosting
`gbm`	Gradient boosting	`gbm()`, `gbm.perf()`	Boosting

Pakke	Beskrivelse	Viktige funksjoner	Kapittel
`gtsummary`	Pene sammendragstabeller	`tbl_cross()`, `tbl_summary()`	Boosting, Fairness
`janitor`	Datarensing	`clean_names()`	Diverse
`dendextend`	Dendrogram-visualisering		Unsupervised
`directlabels`	Direkte labels i plot		Unsupervised
`GGally`	Utvidelser til ggplot2, scatterplotmatriser	`ggpairs()`	Unsupervised

1 Introduksjon til maskinlæring

Temaet er maskinlæring generelt og datadrevne beslutninger. Vi kommer inn på flere av temaene som vil behandles grundigere gjennom kurset.

1.1 Noen innledende metodiske begrep

I standard samfunnsvitenskapelige metodekurs lærer man først og fremst teknikker for å beskrive data og statistisk inferens for å beskrive usikkerheten rundt estimatene. Avhengig av studiets øvrige design kan resultatene tolkes kausalt og/eller generalisere til en nærmere veldefinert populasjon (R. Berk 2016).

Usikkerhet beskrives typisk ved hjelp standardfeil, p-verdier og konfidensintervall tilhørende spesifikke statistiske tester. Dette innebærer at man bruker statistiske *modeller* for hvordan resultatene ville sett ut under spesifikke forutsetninger. Samplingfordelinger som normalfordelingen og en del andre tilsvarende fordelinger er derfor sentralt. De fleste teknikkene vi skal bruke i dette kurset er *ikke* statistiske modeller i samme forstand og det er ingen antakelser om samplingfordelinger. Standardfeil og konfidensintervall kan derfor ikke regnes ut. Usikkerhet og hvem resultatene gjelder for er også relevante for maskinlæring, men ikke helt på samme måte.

1.2 Forklaringer og prediksjoner

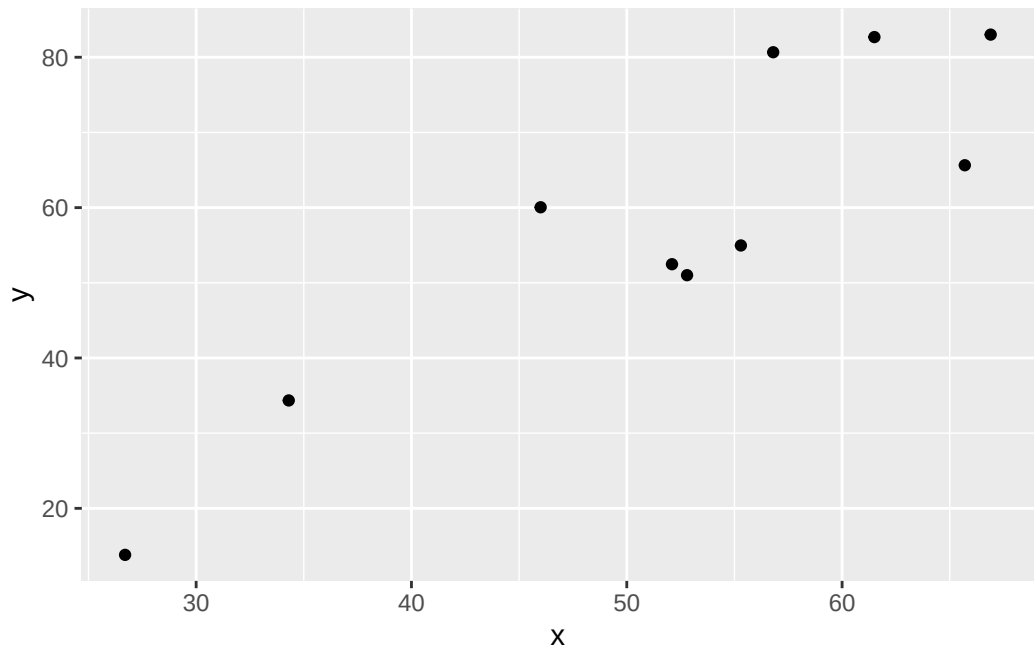
Vi er i liten grad interessert i regresjonskoeffisienter, β, og tolkning av denne. Derimot er vi interessert i det predikerte utfallet $\hat{y}_i$.

1.3 Overfitting: training og testing dataset

Når man tilpasser en statistisk modell eller algoritme til data så er det lett å tenke at modellen bør gjenspeile dataene så godt som mulig. Samtidig sies det ofte at modellene skal være så enkle som tilrådelig. En mer komplisert modell vil jo være i stand til å tilpasses dataene i større grad, så hvordan avveie dette?

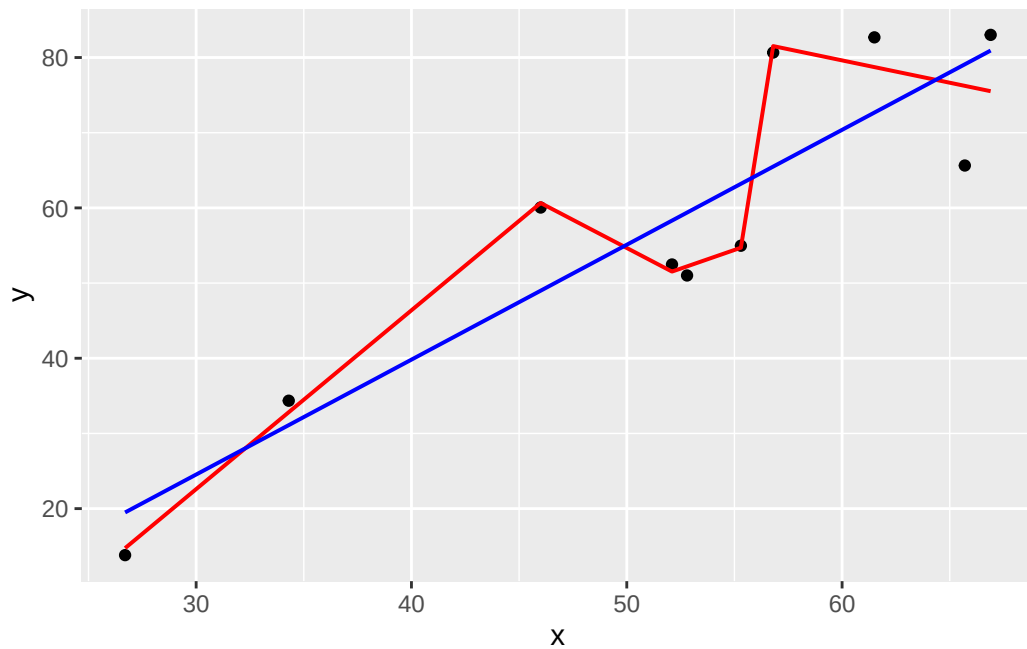
Spørsmålet nå er ikke hvor godt modellen passer til disse dataene, men hvordan den passer til *nye* data! Altså *fremtidige* data eller fremtidig situasjon. La oss si at vi har et datasett som kan plottes om følger:

```
library(tidyverse)
n <- 10
beta <- 1
set.seed(42)
x <- round(20 + runif(n)*50, digits = 1)
y <- 1 + beta*x + rnorm(n)*10

df <- data.frame(x = x, y = y) %>%
  mutate(d = case_when(x < 50 ~ 0,
                       x < 56 ~ 1,
                       TRUE ~2) %>% as_factor())


g1 <- ggplot(df, aes(x = x, y = y)) +
  geom_point()
g1
```



Vi kunne her tilpasse en enkel lineær regresjonsmodell eller en mer komplisert modell. Resultatet vises i grafen nedenfor.

```r
est1 <- lm(y ~ x + x*d, data = df)
est2 <- lm(y ~ x, data = df)

df_p1 <- df %>%
  mutate(pred = predict(est1))  %>%
  mutate(res_pred = y - pred,
         res_y = y - mean(y))


ggplot(df, aes(x = x, y = y)) +
  geom_point(col = "black") +
  geom_line(data = df_p1, aes(y = pred), col = "red", linewidth = .7) +
  stat_smooth(method='lm', formula = y ~ x, se = F, col = "blue", linewidth = .7)
```



Den kompliserte modellen gir $r^2 = 0.9567499$ mens den enkle lineære gir $r^2 = 0.8066771$.

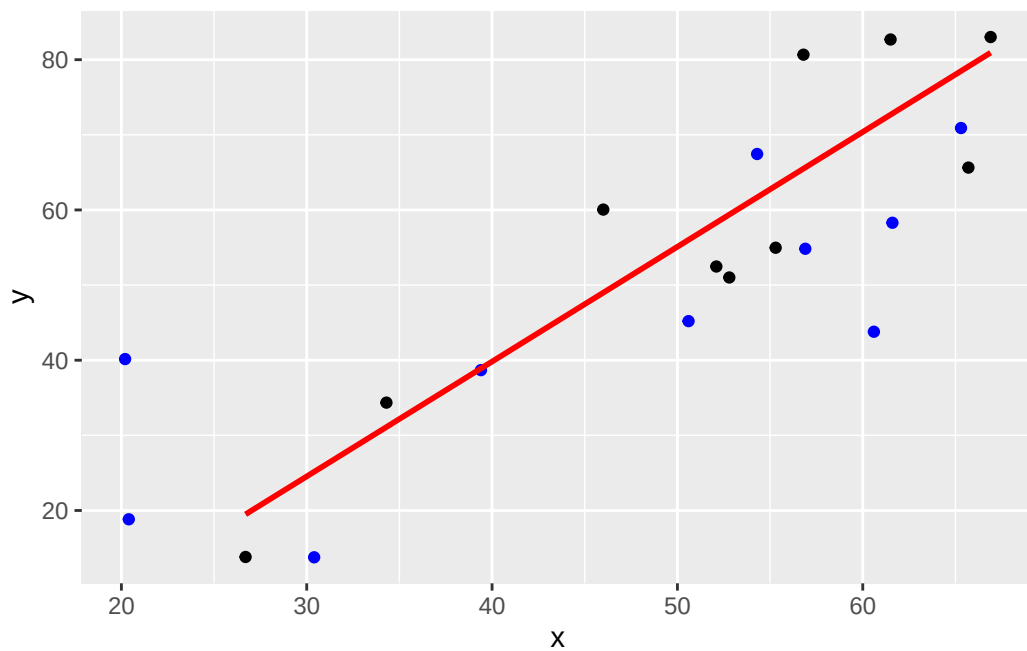
```r
summary(est1)$r.squared
```

```
[1] 0.9567499
```

```r
summary(est2)$r.squared
```

```
[1] 0.8066771
```

```r
x <- round(20 + runif(n)*50, digits = 1)
y <- 1 + beta*x + rnorm(n)*10
df2 <- data.frame(x = x, y = y)

g1 +
  geom_point(data = df2, col = "blue") +
  geom_smooth(method = lm, se = F, col = "red")
```



1.4 Klassifikasjonsusikkerhet - grunnleggende begreper

I praksis kommer vi til å fokusere mest på *klassifikasjon*, som altså er når det vi predikerer er en kategorisk variabel. For nye personer vil vi da gjerne predikere hvilken kategori vedkommende tilhører. En variant av dette er å predikere en fremtidig *handling* (slutte i jobb, ikke betale tilbake lån, begå ny kriminalitet etc).

1.4.1 Confusion matrix: riktig og feil klassifisering

Når vi predikerer et kategorisk utfall er det gjerne ett av utfallene vi primært er interessert i. Disse kalles *positive* og de andre er *negative*. Dette har ingenting å gjøre med om utfallet er bra eller dårlig å gjøre. Å predikere en sykdom vil være *positivt* og å være frisk vil være *negativt*. Å ha tilbakefall til kriminalitet vil være *positivt* og lovlydig vil være *negativt*.

En *positiv* prediksjon kan da være korrekt eller feil, og disse kalles da henholdsvis *sanne* eller *falske* positive. Tilsvarende kan en negaitv prediksjon være sann eller falsk.

		Predikert	
		Negativ	**Positiv**
Observert	**Negativ**	Sanne negative (TN)	Falske positive (FN)
	Positiv	Falske negative (FN)	Sanne positive (TP)

1.4.2 Asymetriske kostnader

Å predikere feil kan betraktes som en *kostnad*. Hvis vi skal bruke prediksjonenen til noe i praksis, så skal det jo få konsekvenser på en eller annen måte. Det kan innebære at man setter i verk tiltak som er unødvendige - eller ikke setter i verk tiltak der man burde gjort det.

Et sentralt spørsmål er derfor om begge typer feil er like viktig eller alvorlig. Noen ganger er det det, men det bør man ta stilling til helt konkret i det enkelte tilfellet. Det er ikke åpenbart hvem som har kompetanse til å vurdere dette. Det kan kreves inngående fagkunnskap for å gjøre en riktig vurdering, eller det kan være politiske prioriteringer, økonomiske forhold, rettferdighetsvurderinger osv. Det er i hvert fall ikke bare opp til forskeren eller IT-personalet å vurdere.

Dette kan koke ned til helt konkret vurdering av hvor mange falske positive er du villig til å godta per falske negative. Et konkret eksempel er studien til Berk et al (2016) av menn som er arrestert for vold i nære relasjoner. Problemstillingen er hvem skal i arrest frem til saken kommer opp og hvem skal løslates mot kausjon. Prediksjonen er da hvem som vil begå ny voldshandling mot partner. Forholdet mellom TN og FN oversettes konkret til hvor mange mistenkte skal sitte unødig i fengsel (dvs. falske positive) mot hvor mange partnere skal unødig utsettes for ny voldshendelse (dvs. falske negative)? I nevnte studie har noen "stakeholders" landet på at falske negative er vesentlig mer alvorlig enn falske positive. Prediksjonsmodellen utformes så for å reflektere akkurat det.

1.5 Rettferdighet og rimelighet

I diskusjoner av anvendelser av maskinlæring står rettferdighet helt sentralt. Men det er ikke alltid like klart hva dette egentlig betyr utover at det er forskjellsbehandling. Tross alt er hele formålet med prediksjon å nettopp forskjellsbehandle, eller *målrette* som det også kan kalles. Rettferdighet kommer inn på flere nivåer, fra det helt prinsippielle ved å la data og datamaskiner ha betydning for avgjørelser til det helt konkrete til hvordan feilratene bør se ut. Vurdering av asymetriske kostnader er åpenbart også et rettferdighetsspørsmål med etiske implikasjoner. En variant er at disse feilratene kan se forskjellige ut på tvers av undergrupper.

Hvilke konsekvenser som er akseptable er viktig. Men det er også viktig *før* man bygger modellen. Software kommer med default innstillinger, men det betyr jo ikke at de er "nøytrale". Resultatene kan til en viss grad styres, så det må man rett og slett gjøre.

1.5.1 Fundamentale skjevheter i data

Siden maskinlæring baserer seg på å lære av tilgjengelige data for å benytte det på nye tilfeller spiller det vesentlig rolle hvordan de opprinnelige dataene ble generert i utgangspunktet.

Et velkjent eksempel er hvordan Amazon besluttet å slutte å bruke en algoritme for rekruttering fordi den systematisk valgte bort kvinner. Grunnen til at algoritmen gjorde dette var så enkelt som at dataene den var trent opp på var mannsdominert. Algoritmen hadde altså primært tilgang til informasjon om hvilke egenskaper som kjennetegnet talentfulle *mannlige* kandidater, som altså kan være forskjellige fra talentfulle *kvinnelige* kandidater.

Når man skal ta en algoritme i bruk er det derfor helt avgjørende at man kan forsvare bruken av de dataene algoritmen er trent på. Kjente skjevheter kan i prinsippet motarbeides ved *tuning* (dette kommer vi tilbake til), men det er vanskelig å garantere at det er skjevheter man *ikke* har tenkt på.

2 Lineær regresjon

I dette kapittelt skal vi bruke følgende pakker:

```
invisible(Sys.setlocale(locale='no_NB.utf8'))

library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(splines)     # for spline-regresjon
library(Ecdat)       # inneholder datasettet Clothing
library(mgcv)        # for generaliserte additive modeller (GAM)
```

Lineær regresjon slik vi skal bruke det her er helt vanlig regresjon slik du har lært om i grunnleggende kurs i kvantitative metoder. Altså lineær funksjon estimert med minste kvadraters metode. Alt du har lært før gjelder også her. Forskjellen er at vi nå ikke er så interessert i å tolke β men i den predikerte $\hat{y}$.

2.1 Lese inn data

Vi illustrerer lineær regresjon med et empirisk eksempel. Her skal vi bruke data for norske kommuner i 2016. La oss si at vi er interessert i hvordan antall voldshendelser per 1000 innbyggere vil endre seg i en kommune. Dette kunne være relevant for langtidsplanlegging av forebygging, politibemanning, helsetjenester osv. Det kan være et område som er i stor endring slik at befolkningssammensetningen forventes å endre seg og/eller det er endrede lokale økonomiske utsikter.

Først leser vi inn dataene og tar en titt på variabellisten.

```
kommune <- readRDS( "data/kommunedata.rds")
glimpse(kommune)
```

```
Rows: 1,529
Columns: 28
$ kommune_nr             <chr> "0101", "0101", "0101", "0101", "0104", "0104",~
```

```
$ kommune                <chr> "Halden (-2019)", "Halden (-2019)", "Halden (-
2~
$ year                   <dbl> 2015, 2016, 2017, 2018, 2015, 2016, 2017, 2018,~
$ bef_18min              <int> 3556, 3503, 3505, 3544, 3594, 3652, 3704, 3655,~
$ bef_18_25              <int> 3575, 3585, 3432, 3438, 3405, 3404, 3355, 3370,~
$ bef_26_35              <int> 3728, 3804, 3985, 4035, 4057, 4071, 4124, 4110,~
$ bef_totalt             <int> 30328, 30544, 30790, 31037, 31802, 32182, 32407~
$ menn_18_25             <int> 1847, 1865, 1813, 1819, 1789, 1802, 1789, 1810,~
$ menn_26_35             <int> 1880, 1919, 2005, 2062, 2063, 2083, 2113, 2134,~
$ menn_36_67             <int> 7067, 7051, 7085, 7057, 7418, 7453, 7408, 7407,~
$ menn_67plus            <int> 2496, 2624, 2697, 2806, 2671, 2777, 2856, 2895,~
$ menn_18min             <int> 1880, 1847, 1873, 1876, 1842, 1885, 1919, 1878,~
$ kvinner_18_25          <int> 1728, 1720, 1619, 1619, 1616, 1602, 1566, 1560,~
$ kvinner_26_35          <int> 1848, 1885, 1980, 1973, 1994, 1988, 2011, 1976,~
$ kvinner_36_67          <int> 6880, 6832, 6844, 6848, 7479, 7519, 7537, 7596,~
$ kvinner_67plus         <int> 3026, 3145, 3242, 3309, 3178, 3306, 3423, 3555,~
$ kvinner_18min          <int> 1676, 1656, 1632, 1668, 1752, 1767, 1785, 1777,~
$ inntekt_totalt_median  <int> 555000, 562000, 580000, 591000, 561000, 568000,~
$ inntekt_eskatt_median  <int> 451000, 453000, 470000, 480000, 449000, 456000,~
$ ant_husholdninger      <int> 13890, 14124, 14281, 14454, 15046, 15132, 15313~
$ shj_klienter           <int> 1183, 1137, 1099, 1128, 1155, 1129, 1152, 1137,~
$ shj_unge               <int> 262, 247, 248, 242, 267, 263, 238, 222, 307, 28~
$ vinningskriminalitet   <dbl> 19.7, 18.7, 16.5, 14.5, 24.5, 21.5, 18.0, 18.0,~
$ voldskriminalitet      <dbl> 11.2, 12.6, 12.3, 11.2, 7.8, 8.3, 8.7, 9.7, 6.8~
$ nark_alko_kriminalitet <dbl> 21.0, 21.9, 21.0, 20.3, 12.0, 10.2, 10.9, 10.1,~
$ ordenslovbrudd         <dbl> 18.5, 16.5, 14.9, 13.7, 8.9, 9.0, 9.1, 9.2, 8.2~
$ trafikklovbrudd        <dbl> 15.5, 16.3, 16.7, 19.2, 7.4, 6.3, 6.9, 8.0, 9.6~
$ andre_lovbrudd         <dbl> 25.5, 26.5, 26.1, 25.2, 12.1, 12.2, 11.9, 12.4,~
```

2.1.1 Training og testing data

Vi starter med å dele datasettet i *training* og *testing*. Her kan vi bruke pakken 'rsample' og funksjonen `initial_split()` etterfulgt av funksjonene `training()` og `testing()`. Forhåndsvalget for splitten er 0.75, så dermed brukes 75% til training og resten til testing. Du kan også legge til argumentet `prop =` og sette en annen andel, f.eks. 0.7 for 70%.

Merk bruken av `set.seed()`. For å splitte genererer R tilfeldige tall, og *seed* styrer hvor den algoritmen starter. Når *seed* er satt vil du få nøyaktig samme resultatet som gjort her. Dette vil du trenge på eksamen for at sensor skal kunne sjekke resultatene dine, så begynn å bruke det med en gang. Tallet inni parentesen betyr ingenting[1] og bare sørger for reproduserbarhet der hvor det er tilfeldige tall involvert.

[1] Bortsett fra for dem som mener det er svaret på "the Ultimate Question of Life, the Universe, and Everything"

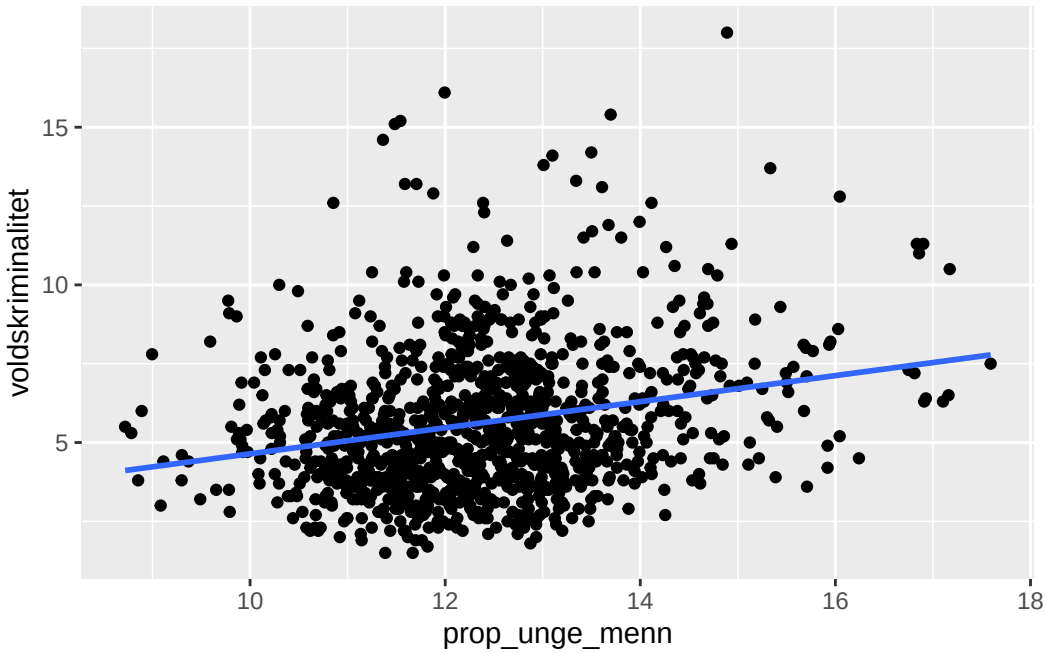
```r
set.seed(42)
kommune_split <- initial_split(kommune, prop = .7)

kommune_train <- training(kommune_split)
kommune_test <- testing(kommune_split)
```

2.1.2 Enkel lineær regresjon i R

En ganske åpenbar faktor som forklarer forekomsten av vold er andel unge menn i kommunen. Rett og slett fordi dette er den demografiske gruppen som begår mest vold - og kriminalitet generelt, faktisk. Hvis befolkningssammensetningen forventes å bli yngre vil det medføre flere unge menn, og da kan vi kanskje forvente at det blir flere voldshendelser bare av den grunn? Sammenhengen mellom unge menn og voldsrate kan estimeres med helt vanlig lineær regresjon.

En god start på de fleste empiriske analyser er å beskrive sammenhengen med et plot. Her legger vi på en lineær regresjonslinje med `geom_smooth()` der vi presiserer lineær modell med `method = "lm"` og lar være å ta med konfidensintervallet `se = FALSE`.

```r
kommune_train <- kommune_train %>%
  mutate(prop_unge_menn = (menn_18_25 + menn_26_35)/bef_totalt*100)

ggplot(kommune_train, aes(x = prop_unge_menn,
                      y = voldskriminalitet)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```



Lineær regresjon estimeres med `lm()` og du kan få en enkel output med bruk av `summary()`.

```
est <- lm(voldskriminalitet ~ prop_unge_menn, data = kommune_train)
summary(est)
```

```
Call:
lm(formula = voldskriminalitet ~ prop_unge_menn, data = kommune_train)

Residuals:
    Min      1Q  Median      3Q     Max 
-4.0302 -1.5764 -0.3951  1.1826 11.3367 

Coefficients:
               Estimate Std. Error t value Pr(>|t|)    
(Intercept)     0.50954    0.63482   0.803    0.422    
prop_unge_menn  0.41329    0.05097   8.109  1.4e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.248 on 1068 degrees of freedom
Multiple R-squared:  0.05799,	Adjusted R-squared:  0.05711 
F-statistic: 65.75 on 1 and 1068 DF,  p-value: 1.395e-15
```

Man kan også hente ut kun r^2 med følgende kode:

```
summary(est)$r.squared
```

```
[1] 0.05799176
```

For ordens skyld: I tidligere metodekurs har du kanskje lært å få ut en penere regresjonstabell med f.eks. gtsummary-pakken. (Det finnes andre pakker også som gjør tilsvarende). Hvordan output er formatert spiller ingen rolle. I denne sammenhengen har vi lite bruk for en pen regresjonstabell da β ikke er av primær interesse.

Med andre ord kan voldsraten beskrives som:

$$\text{voldskriminalitet}\hat{} = 0.51 + 0.41(\text{prop_unge_menn})$$

Men vi har også sett at r^2 er ganske lav, bare 0.058. Denne koeffisienten kalles også "coefficient of determination" og sier noe om i hvor stor grad modellen fanger opp variasjoenen i dataene. En lav r^2 betyr at modellen i liten grad gjør det. Vi må altså forvente at modellen vil bomme ganske kraftig i sine prediksjoner. Vi kan velge å ta modellen seriøst likevel, men ikke ha for store forventninger for prediksjonene!

Et annet mål på hvor godt modellen treffer er "Root mean square error", RMSE. Dette kan skrives som:

$$rmse = \sqrt{\frac{\sum (O_i - P_i)^2}{N}}$$

der O er de observerte verdiene og P er de predikerte verdiene for observasjon i. Merk at $(O_i - P_i)$ er residualene. I R kan vi hente ut residualene fra regresjons-objektet med dollartegnet `...$res` etter objektnavnet. Da kan du regne ut RMSE som følger:

```
rmse <- sqrt(mean(est$res^2))
rmse
```

```
[1] 2.245685
```

RMSE sier altså omtrentlig hvor mye modellen i gjennomsnitt bommer på de observerte verdiene. [2]. Hvorvidt det er presist *nok* eller ikke vil vel strengt tatt komme an på behovet for presisjon, altså: hva man skal bruke det til.

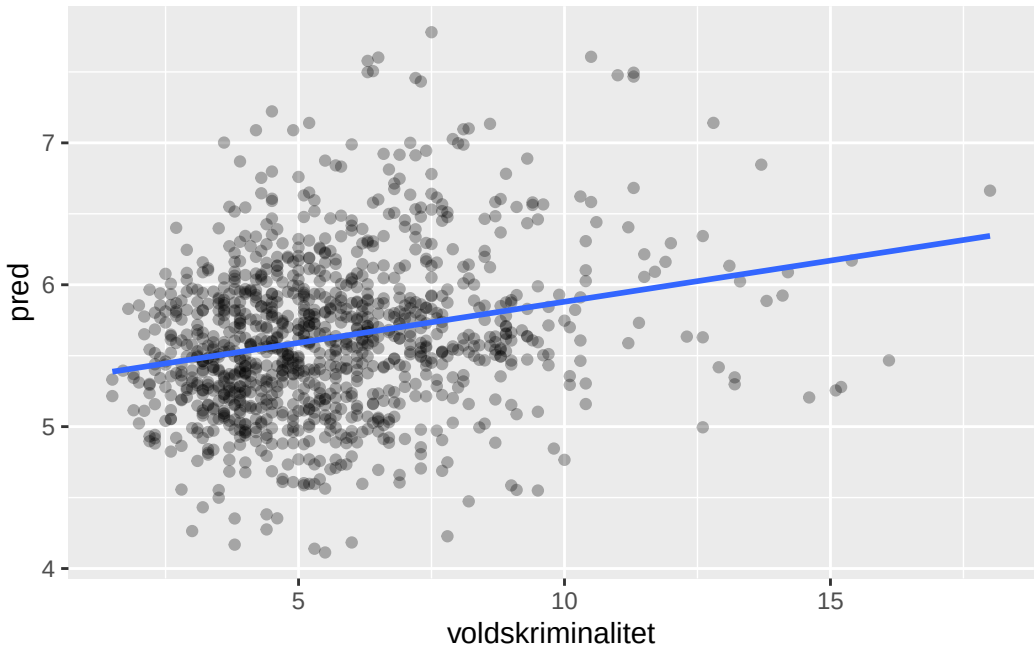
[2] Denne formuleringen er ganske omtrentlig. RMSE er egentlig kvadratroten av gjennomsnittet til de kvadrerte residualene, som er noe litt annet enn gjennomsnittet av de absolutte verdiene av residualene. Det gir bl.a. litt mer vekt til store residualer enn et vanlig gjennomsnitt.

For å få litt bedre tak på hva RMSE betyr kan vi se på et plot av de predikerte og observerte verdiene. Vi kan predikere vold for hver enkelt kommune basert på denne modellen, som altså er den forventede voldsraten *hvis modellen er sann*. Funksjonen `predict()` gir oss hva vi trenger.

```
kom <- kommune_train %>%
  mutate(pred = predict(est))
```

Merk at koden her lagde en kopi av datasettet der vi har alle de opprinnelige variablene pluss en variabel med de predikerte verdiene. Vi kan nå sammenlignet prediksjonene med de observerte utfallene.

```
ggplot(kom, aes(x = voldskriminalitet, y = pred)) +
  geom_point(alpha = .3) +
  geom_smooth(method = "lm", se = FALSE)
```



Hvis prediksjonen hadde vært perfekt ville disse punktene ligget på linja, noe den jo ikke gjør. Modellen bommer altså ganske mye.

Hva hvis vi vil vite forventet voldsrate for en kommune for en gitt andel unge menn? Løsningen er å lage et nytt datasett med de verdiene vi er interessert i og så predikere for dette datasettet med å spesifisere `newdata = dt`. Her er et eksempel der vi ønsker å vite voldsraten hvis andelen unge menn er 15%.

```r
dt <- data.frame(prop_unge_menn = 15)
p <- predict(est, newdata = dt)
p
```

```
       1
6.708919
```

I følge modellen vil altså en kommune der 15% av populasjonen er unge menn ha en 6.709 voldshendelser per 1000 innbyggere. Fra tradisjonell statistikk vet vi jo at det er usikkerhet knyttet til dette estimatet og vi kan også ta det med i beregningen her. Vanligvis vil man estimere med et *konfidensintervall*, som gjelder hvis man estimerer et gjennomsnitt i en gruppe. Her skal vi derimot predikere for en enkelt kommune, som da har større usikkerhet enn om man estimerer for en enkelt observasjon. Dette kalles prediksjonsintervall og må spesifiseres i koden. Hvis det ikke er gitt vil R gi konfidensintervallet.

```r
p_ki <- predict(est, newdata = dt, interval = "prediction")
p_ki
```

```
       fit      lwr      upr
1 6.708919 2.288514 11.12932
```

Tolkningen er ellers tilsvarende som for konfidensintervall: vi forventer med "95% sannsynlighet"[3] at voldsraten vil være mellom 2.3 og 11.1 per 1000 innbyggere.

2.1.3 Multippel regresjon

Enkel regresjon er nettopp enkel og prediksjonen blir ikke så god. Men vi kan komplisere vesentlig ved å inkludere flere variable og bruke alle triksene man evt. har lært om multippel regresjon tidligere, primært interaksjonsledd, polynomer og transformasjoner osv. (Det er ok om du ikke har lært alt dette tidligere).

I R vil vi da bare legge til flere variabelnavn i formelen. Ellers er det meste likt som for enkel lineær regresjon.

```r
est_m <- lm(voldskriminalitet ~ prop_unge_menn + inntekt_totalt_median + shj_unge +
                   ant_husholdninger ,
            data = kommune_train)
summary(est_m)
```

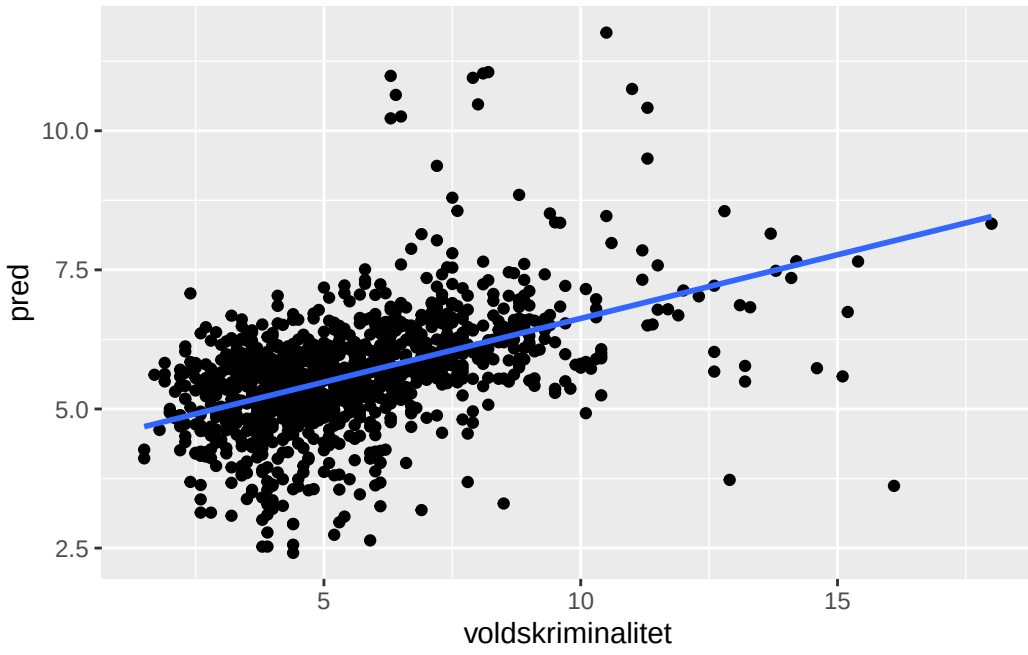
[3] Dette er en omtrentelig formulering. Alle sannsynligheter gjelder i det lange løp: altså hvis man gjør undersøkelsen veldig mange ganger.

```
Call:
lm(formula = voldskriminalitet ~ prop_unge_menn + inntekt_totalt_median +
    shj_unge + ant_husholdninger, data = kommune_train)

Residuals:
    Min      1Q  Median      3Q     Max
-4.6862 -1.3599 -0.2042  1.0689 12.4822

Coefficients:
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)            7.421e+00  7.333e-01  10.121  < 2e-16 ***
prop_unge_menn         4.177e-01  5.328e-02   7.840 1.09e-14 ***
inntekt_totalt_median -1.099e-05  8.539e-07 -12.871  < 2e-16 ***
shj_unge               4.461e-03  1.016e-03   4.392 1.23e-05 ***
ant_husholdninger     -2.163e-05  8.361e-06  -2.587  0.00983 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.037 on 1065 degrees of freedom
Multiple R-squared:  0.2288,    Adjusted R-squared:  0.2259
F-statistic:    79 on 4 and 1065 DF,  p-value: < 2.2e-16
```

Merk at r^2 nå har gått betraktelig opp, til ca 0.23. Gitt at vi tolker dette som i hvor stor grad vi kan *predikere* utfallet fra datasettet, så er det kanskje likevel ikke imponerende høyt: vi vil fremdeles forvente mye feil prediksjon.

Her er et scatterplot av observert mot forventet voldsrater:

```r
kom_pred <- kommune_train %>%
  mutate(pred = predict(est_m))

ggplot(kom_pred, aes(x = voldskriminalitet, y = pred)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```



2.1.4 Sammenligning av modeller med ulik kompleksitet

La oss nå systematisk sammenligne flere modeller med ulik kompleksitet. Vi lager først et datasett med utvalgte variable og bruker formelen `voldskriminalitet ~ .` for å ta med alle variable.

```
kom_s <- kommune_train %>%
  select(-c(kommune, kommune_nr, ordenslovbrudd,
            nark_alko_kriminalitet, trafikklovbrudd, andre_lovbrudd,
            prop_unge_menn))

full_mod <- lm(voldskriminalitet ~ . , data = kom_s)
summary(full_mod)
```

```
Call:
lm(formula = voldskriminalitet ~ ., data = kom_s)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.9650 -1.2333 -0.1813  0.8919 12.2114 
```

```
Coefficients: (4 not defined because of singularities)
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)           -1.198e+02  9.309e+01  -1.287  0.19851
year                   6.234e-02  4.646e-02   1.342  0.17998
bef_18min              3.156e-03  1.839e-03   1.716  0.08637 .
bef_18_25             -1.885e-05  1.513e-03  -0.012  0.99006
bef_26_35              1.180e-03  1.006e-03   1.173  0.24109
bef_totalt            -2.272e-03  7.130e-04  -3.187  0.00148 **
menn_18_25             3.698e-03  2.136e-03   1.731  0.08373 .
menn_26_35            -1.481e-03  2.013e-03  -0.736  0.46197
menn_36_67             2.710e-03  9.461e-04   2.865  0.00425 **
menn_67plus            1.654e-03  1.339e-03   1.236  0.21676
menn_18min             2.461e-03  2.875e-03   0.856  0.39223
kvinner_18_25                 NA         NA      NA       NA
kvinner_26_35                 NA         NA      NA       NA
kvinner_36_67         -1.269e-04  9.021e-04  -0.141  0.88816
kvinner_67plus                NA         NA      NA       NA
kvinner_18min                 NA         NA      NA       NA
inntekt_totalt_median -4.693e-05  7.600e-06  -6.176 9.40e-10 ***
inntekt_eskatt_median  5.580e-05  1.108e-05   5.037 5.55e-07 ***
ant_husholdninger      1.555e-03  3.169e-04   4.908 1.07e-06 ***
shj_klienter           2.170e-03  9.967e-04   2.177  0.02968 *
shj_unge               2.191e-03  3.054e-03   0.718  0.47322
vinningskriminalitet   1.066e-01  1.014e-02  10.513  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.891 on 1052 degrees of freedom
Multiple R-squared:  0.343, Adjusted R-squared:  0.3324
F-statistic:  32.3 on 17 and 1052 DF,  p-value: < 2.2e-16
```

r^2 gikk noe opp, til 0.343.

2.1.5 Eksempel på overfitting

Men vi kan gjøre modellen ekstra komplisert ved inkludere alle mulige interaksjonsledd. En åpenbar ulempe med dette er at hver enkelt koeffisent blir svært mye vanskeligere å tolke. Vi fokuserer derfor kun på r^2 og RMSE.

Her bygger vi fire modeller med økende kompleksitet og sammenligner RMSE for trainingdata:

```r
est1 <- est     # enkel modell med bare prop_unge_menn (estimert over)
est2 <- est_m   # multippel regresjon (estimert over)
est3 <- lm(voldskriminalitet ~ .^2, data = kom_s)    # alle 2-veis interaksjoner
est4 <- lm(voldskriminalitet ~ .^3, data = kom_s)    # alle 3-veis interaksjoner
```

RMSE for training-data:

```r
sqrt(mean(est1$res^2))
```

```
[1] 2.245685
```

```r
sqrt(mean(est2$res^2))
```

```
[1] 2.031873
```

```r
sqrt(mean(est3$res^2))
```

```
[1] 1.598957
```

```r
sqrt(mean(est4$res^2))
```

```
[1] 0.9963221
```

For training-data ser vi altså at RMSE synker for hver modell. Mer kompleksitet gir tilsynelatende bedre tilpassning. Men hva skjer med testing-data?

2.1.6 Predikere for nye data

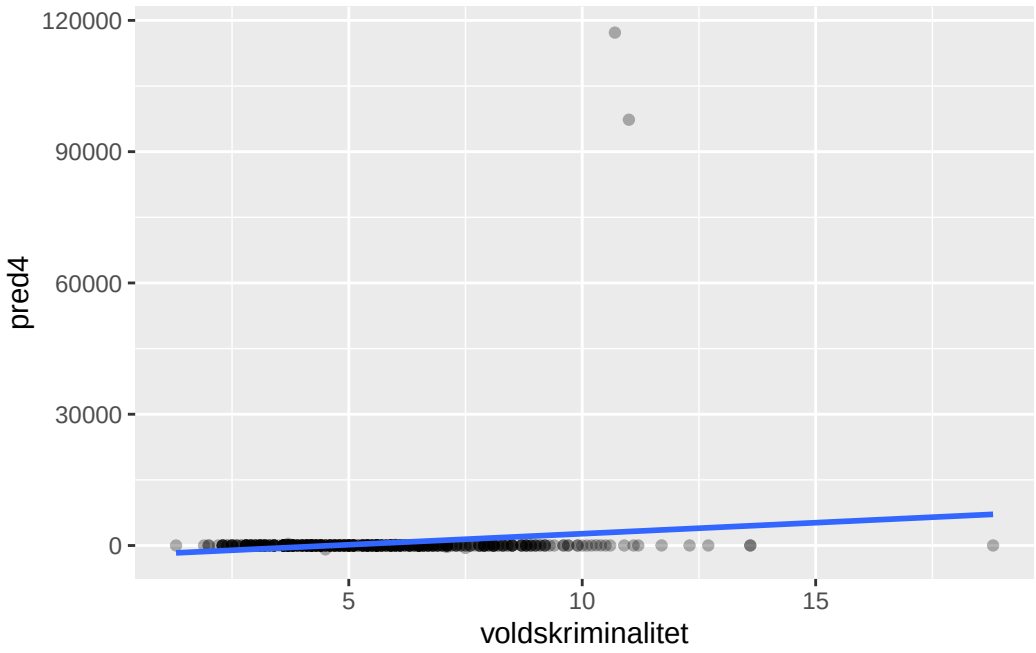
Nå har vi bare sett på hvordan prediksjonen fungerer på training-datasettet. Vi må sjekke med testing-datasettet. Det lagde vi i begynnelsen, så nå henter vi det frem. I `predict()` må vi nå angi `newdata =`. Vi predikerer fra alle fire modellene samtidig og regner ut RMSE for testing-data:

```r
kommune_test <- kommune_test %>%
  mutate(prop_unge_menn = (menn_18_25 + menn_26_35)/bef_totalt*100)

kommune_pred <- kommune_test %>%
  mutate(pred1 = predict(est1, newdata = .),
         pred2 = predict(est2, newdata = .),
         pred3 = predict(est3, newdata = .),
         pred4 = predict(est4, newdata = .))

rmse_test <- kommune_pred %>%
  mutate(res1 = pred1 - voldskriminalitet,
         res2 = pred2 - voldskriminalitet,
         res3 = pred3 - voldskriminalitet,
         res4 = pred4 - voldskriminalitet) %>%
  summarise(RMSE_enkel = sqrt(mean(res1^2)),
            RMSE_multippel = sqrt(mean(res2^2)),
            RMSE_2veis = sqrt(mean(res3^2)),
            RMSE_3veis = sqrt(mean(res4^2)))

rmse_test
```

```
  RMSE_enkel RMSE_multippel RMSE_2veis RMSE_3veis
1   2.147951       1.876014   23.41166   7109.869
```

Her ser vi at de mest kompliserte modellene gir *dårligere* prediksjon på nye data! La oss også plotte predikert mot observert for den mest kompliserte modellen:

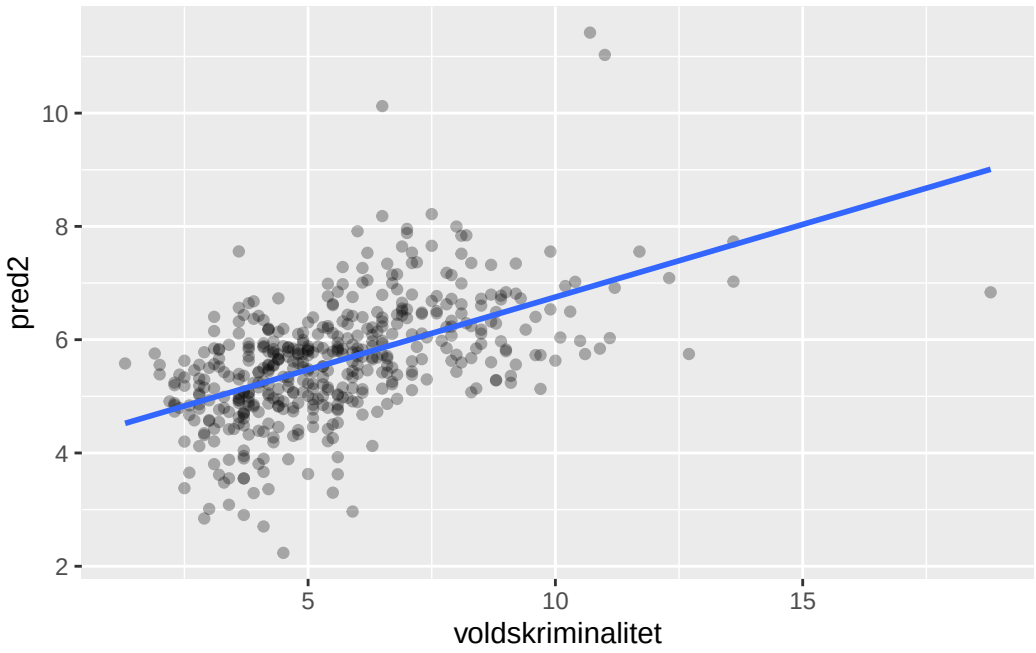
```r
ggplot(kommune_pred, aes(x = voldskriminalitet, y = pred4)) +
  geom_point(alpha = .3) +
  geom_smooth(method = "lm", se = FALSE)
```



Ser dette rart ut? Det er noen observasjoner som har predikert skyhøy voldsrate! Disse prediksjonene bommer voldsomt.

Sammenlignet er den enklere multippelregresjonsmodellen vesentlig bedre på nye data:

```
ggplot(kommune_pred, aes(x = voldskriminalitet, y = pred2)) +
  geom_point(alpha = .3) +
  geom_smooth(method = "lm", se = FALSE)
```



Dette er en ganske ekstrem variant av *overfitting*. Ved å formulere en veldig komplisert modell som passer veldig godt til training-dataene kan vi ende opp med en modell som passer veldig, veldig dårlig til nye data. Hvis man skulle utforme f.eks. politikktiltak på grunnlag av en slik prediksjon vil det kunne gå riktig så dårlig.

Det er altså slik at en enklere modell kan passe til *nye* data langt bedre enn en veldig komplisert modell. Grunnen er overfitting: modellen fanger opp vel så mye tilfeldig støy som det underliggende mønsteret. Støyen vil jo være annerledes i de nye dataene.

2.2 Splines

I modellene over har vi antatt en lineær sammenheng mellom prediktor og utfall. Men hva hvis sammenhengen *ikke* er lineær? En mulighet er å bruke *splines*, som er stykkevis definerte funksjoner som kan tilpasse mer fleksible former. Splines er nyttige for prediksjon fordi de kan fange opp ikke-lineære mønstre i dataene uten at vi trenger å spesifisere funksjonsformen på forhånd.

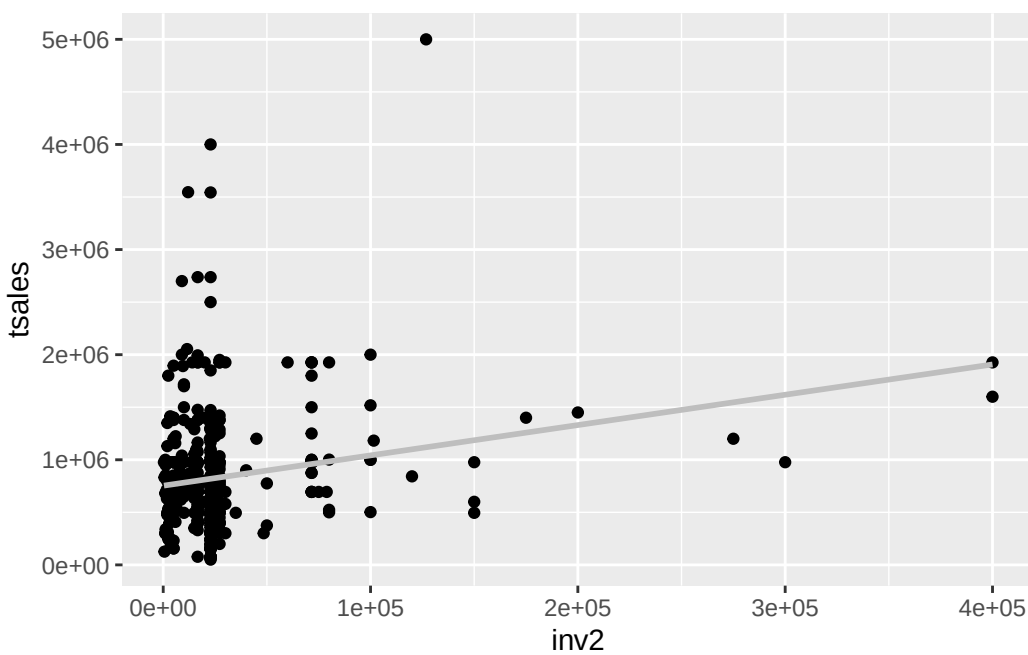
Vi bruker datasettet Clothing fra pakken `Ecdat` som eksempel. Variabelen `tsales` er totalt salg og `inv2` er investeringer.

```
data(Clothing)
glimpse(Clothing)
```

```
Rows: 400
Columns: 13
$ tsales  <int> 750000, 1926395, 1250000, 694227, 750000, 400000, 1300000, 495~
$ sales   <dbl> 4411.765, 4280.878, 4166.667, 2670.104, 15000.000, 4444.444, 3~
$ margin  <dbl> 41, 39, 40, 40, 44, 41, 39, 28, 41, 37, 32, 30, 39, 37, 34, 44~
$ nown    <dbl> 1.0000, 2.0000, 1.0000, 1.0000, 2.0000, 2.0000, 1.2228, 2.0000~
$ nfull   <dbl> 1.0000, 2.0000, 2.0000, 1.0000, 1.9556, 1.9556, 1.0000, 1.9556~
$ npart   <dbl> 1.0000, 3.0000, 2.2222, 1.2833, 1.2833, 1.2833, 3.0000, 1.2833~
$ naux    <dbl> 1.5357, 1.5357, 1.4091, 1.3673, 1.3673, 1.3673, 4.0000, 1.3673~
$ hoursw  <int> 76, 192, 114, 100, 104, 72, 161, 80, 158, 87, 156, 40, 96, 305~
$ hourspw <dbl> 16.755960, 22.493760, 17.191200, 21.502600, 15.742790, 10.8988~
$ inv1    <dbl> 17166.67, 17166.67, 292857.20, 22207.04, 22207.04, 22207.04, 2~
$ inv2    <dbl> 27177.04, 27177.04, 71570.55, 15000.00, 10000.00, 22859.85, 22~
$ ssize   <int> 170, 450, 300, 260, 50, 90, 400, 100, 450, 75, 120, 40, 90, 11~
$ start   <dbl> 41, 39, 40, 40, 44, 41, 39, 28, 41, 37, 32, 30, 39, 37, 34, 44~
```

La oss først se på sammenhengen med et scatterplot og en lineær regresjonslinje:

```r
p0 <- ggplot(Clothing, aes(inv2, tsales)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, col = "gray")

p0
```



Sammenhengen ser ikke helt lineær ut. Vi kan prøve en *piecewise linear spline* der vi angir knuter (knots) der funksjonen skifter retning. Funksjonen `bs()` fra pakken `splines` brukes til dette.

2.2.1 Piecewise linear spline

Med `degree = 1` får vi en stykkevis lineær funksjon, altså rette linjer mellom knutene:

```
model1 <- lm(tsales ~ bs(inv2, knots = c(12000, 60000, 150000), degree = 1), data = Clothing)
summary(model1)
```

```
Call:
lm(formula = tsales ~ bs(inv2, knots = c(12000, 60000, 150000),
    degree = 1), data = Clothing)

Residuals:
    Min      1Q  Median      3Q     Max
-983725 -333443  -97927  178664 3670029

Coefficients:
                                                        Estimate Std. Error
(Intercept)                                               717496      84585
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)1     78041     107382
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)2    183187     134824
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)3    761569     242818
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)4    804300     365549
                                                        t value Pr(>|t|)
(Intercept)                                               8.483 4.47e-16 ***
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)1    0.727  0.46780
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)2    1.359  0.17501
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)3    3.136  0.00184 **
bs(inv2, knots = c(12000, 60000, 150000), degree = 1)4    2.200  0.02837 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 572000 on 395 degrees of freedom
Multiple R-squared:  0.0486,    Adjusted R-squared:  0.03897
F-statistic: 5.044 on 4 and 395 DF,  p-value: 0.0005649
```

2.2.2 B-spline (kubisk)

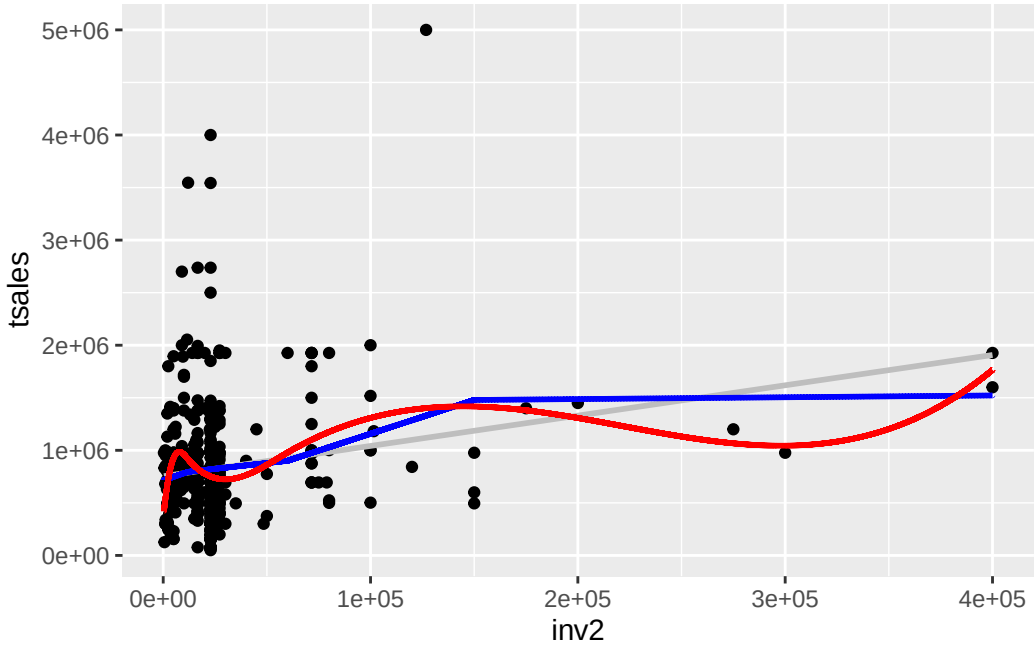
Med `degree = 3` får vi en glattere, kubisk funksjon:

```
model2 <- lm(tsales ~ bs(inv2, knots = c(12000, 60000, 150000), degree = 3), data = Clothing)
summary(model2)
```

```
Call:
lm(formula = tsales ~ bs(inv2, knots = c(12000, 60000, 150000),
    degree = 3), data = Clothing)

Residuals:
    Min      1Q  Median      3Q     Max
-921708 -329164 -125551  223018 3598276

Coefficients:
                                                        Estimate Std. Error
(Intercept)                                               419589     136370
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)1   712180     213810
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)2    63428     140939
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)3   847253     269728
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)4  1308842     707178
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)5   -14067     996832
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)6  1345263     419450
                                                        t value Pr(>|t|)
(Intercept)                                               3.077 0.002238 **
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)1   3.331 0.000948 ***
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)2   0.450 0.652929
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)3   3.141 0.001810 **
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)4   1.851 0.064949 .
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)5  -0.014 0.988748
bs(inv2, knots = c(12000, 60000, 150000), degree = 3)6   3.207 0.001450 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 562100 on 393 degrees of freedom
Multiple R-squared:  0.08582,   Adjusted R-squared:  0.07186
F-statistic: 6.149 on 6 and 393 DF,  p-value: 3.54e-06
```

For å visualisere forskjellen lager vi et datasett med predikerte verdier for begge modellene:

```r
inv2.grid <- data.frame(inv2 = seq(min(Clothing$inv2), max(Clothing$inv2)))

pred1 <- predict(model1, newdata = inv2.grid, se = TRUE, interval = "prediction")$fit %>%
  bind_cols(inv2.grid)

pred2 <- predict(model2, newdata = inv2.grid, se = TRUE, interval = "prediction")$fit %>%
  bind_cols(inv2.grid)

ggplot(Clothing, aes(inv2, tsales)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, col = "gray") +
  geom_line(data = pred1, aes(y = fit), linewidth = 1, col = "blue") +
  geom_line(data = pred2, aes(y = fit), linewidth = 1, col = "red")
```



Den blå linjen er den stykkevis lineære spline og den røde er den kubiske. Begge tilpasser dataene bedre enn den rette linjen.

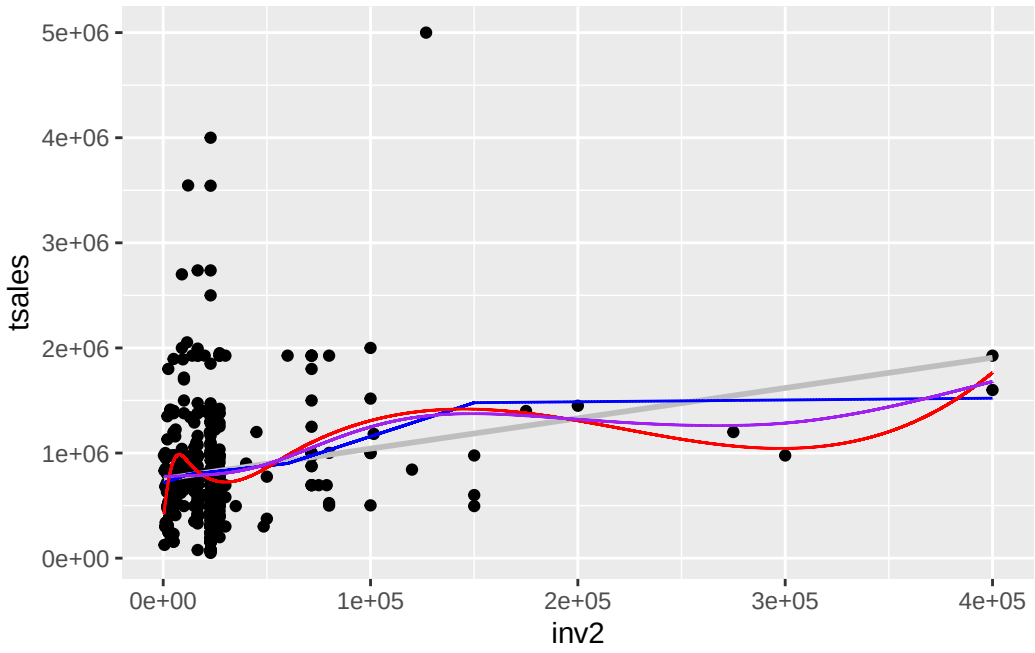
2.2.3 GAM med smoothing spline

En enda enklere tilnærming er å bruke `gam()` fra `mgcv`-pakken med `s()` som automatisk velger en god smoothing:

```r
model3 <- gam(tsales ~ s(inv2), data = Clothing)
summary(model3)
```

```
Family: gaussian
Link function: identity

Formula:
tsales ~ s(inv2)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   833584      28433   29.32   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
          edf Ref.df     F  p-value
s(inv2) 3.903  4.748 4.844 0.000409 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.0501   Deviance explained = 5.94%
GCV = 3.2739e+11  Scale est. = 3.2338e+11  n = 400
```

```r
pred3 <- inv2.grid %>%
  mutate(fit = predict(model3, newdata = inv2.grid))

ggplot(Clothing, aes(inv2, tsales)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, col = "gray") +
  geom_line(data = pred1, aes(y = fit), col = "blue") +
  geom_line(data = pred2, aes(y = fit), col = "red") +
  geom_line(data = pred3, aes(y = fit), col = "purple")
```



Den lilla linjen er GAM-modellen. Fordelen med GAM er at du ikke trenger å velge knuter selv – funksjonen finner en god tilpassning automatisk.

2.2.4 GAM med kommunedata

Generaliserte additive modeller er i utgangspunktet som lineær regresjon, men inkluderer en eller flere *smoothing*-parametre. Disse smoothing-parametrene "lærer" funksjonsformen fra dataene i stedet for å anta en lineær sammenheng. Funksjonen `gam()` fra pakken `mgcv` brukes for dette, og `s()` rundt en variabel angir at den skal ha en fleksibel form.

Her er et eksempel der vi bruker GAM med noen utvalgte variable fra kommunedataene:

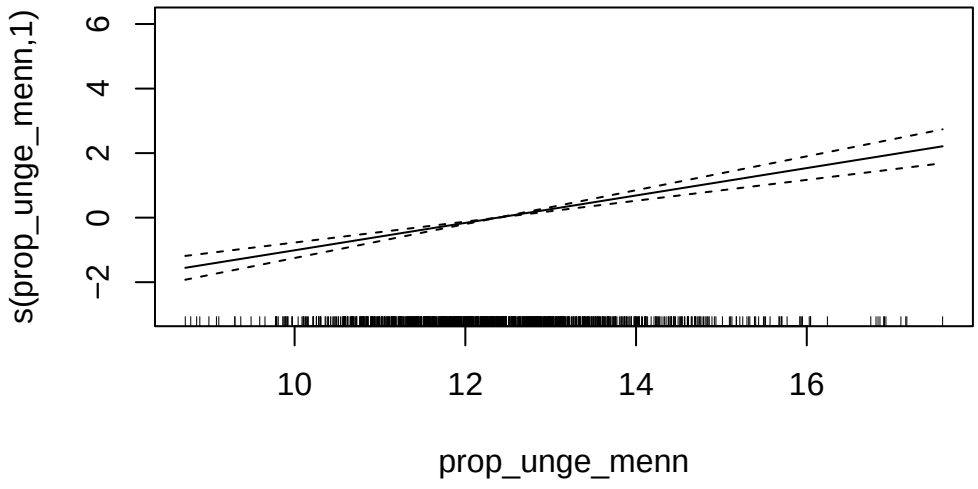
```
gam_mod <- gam(voldskriminalitet ~ s(prop_unge_menn) + s(inntekt_totalt_median) + s(shj_unge
               data = kommune_train)
summary(gam_mod)
```

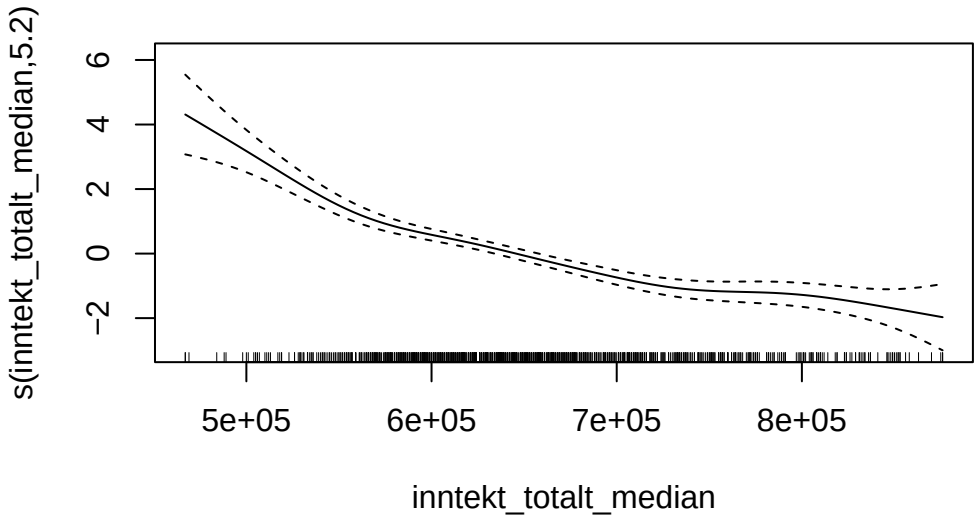
```
Family: gaussian
Link function: identity

Formula:
voldskriminalitet ~ s(prop_unge_menn) + s(inntekt_totalt_median) +
```

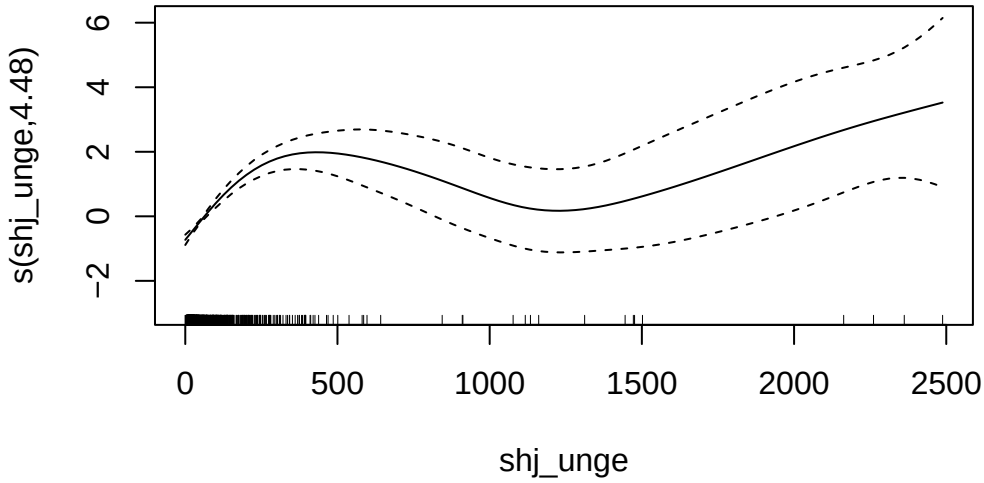
```
    s(shj_unge)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.62673    0.05937   94.77   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
                           edf Ref.df     F p-value
s(prop_unge_menn)        1.000  1.000 68.21  <2e-16 ***
s(inntekt_totalt_median) 5.197  6.339 37.48  <2e-16 ***
s(shj_unge)              4.478  5.462 22.78  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.296   Deviance explained = 30.3%
GCV = 3.8136  Scale est. = 3.772     n = 1070
```

Vi kan plotte de estimerte smoothing-funksjonene for å se hvordan sammenhengen ser ut:

```r
plot(gam_mod)
```







Merk at `s()` tilpasser en fleksibel kurve til dataene. Fordelen med GAM er at vi ikke trenger å anta en lineær sammenheng. Ulempen er at det er vanskeligere å tolke koeffisientene direkte, men for *prediksjon* er dette ofte ikke et problem.

2.2.5 Oppsummerende kommentar

Det gjelder generelt at mer kompliserte regresjonsmodeller vil gi tilsynelatende bedre tilpassning til data, enten man måler med r^2 eller RMSE. Man kan riktignok bruke mål på tilpassning som justerer for antall parametre etc (f.eks. justert r^2, AIC eller BIC), som kanskje kan bedre dette noe, men det vil ofte lede til overfitting likevel.

Det er derfor veldig viktig å teste modellen mot *nye* data – i praksis testing-dataene man lagde til å begynne med. Splines og GAM gir mer fleksible modeller enn vanlig lineær regresjon, men de er langt fra så risikable som f.eks. 3-veis interaksjoner. Man kan godt bruke interaksjonsledd, polynomer, splines og andre verktøy. Men med måte.

3 Logistisk regresjon

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(pROC)        # Beregne ROC-curve
library(caret)       # Funksjonen confusionMatrix()
```

3.1 Empirisk eksempel: Kategorisk utfall

Som eksempel bruker vi et datasettet Attrition. Dette er et datasett over arbeidstakere i en bedrift der utfallsvariabelen er om arbeidstakeren slutter i jobben eller ikke.

For arbeidsgivere kan det være kostbart med endringer i staben. Arbeidstakere som slutter tar med seg erfaring og kompetanse, og nye arbeidstakere må læres opp. Arbeidsgiver bør derfor generelt legge til rette for at arbeidstakere ønsker å bli værende, men det kan også være aktuelt med mer målrettede tiltak. Når en arbeidstaker har fått et nytt jobbtilbud kan det være for sent. Hvis man derimot kan komme i forkjøpet kan man kanskje gjøre noe *før* vedkommende går til det skrittet å søke ny jobb. Hvis man kunne predikere hvem som kommer til å slutte kunne man altså gjort tiltak i forkant.[1]

Først leser vi inn datasettet. Deretter kan vi se på innholdet med `glimpse()`:

```
attrition <- readRDS("data/attrition.rds")
glimpse(attrition)
```

```
Rows: 1,470
Columns: 32
$ Age                      <int> 41, 49, 37, 33, 27, 32, 59, 30, 38, 36, 35, 2~
$ Attrition                <fct> Yes, No, Yes, No, No, No, No, No, No, No, No,~
```

[1] Her kunne man jo også tenke seg at den gode lederen har en dialog med de ansatte og fanger opp deres frustrasjoner og behov slik at maskinell prediksjon ikke trengs. Det er jo også en form for prediksjon med kvalitative data! Så her er vi i en setting der dette ikke fungerer eller det er så store forhold at en kvalitativ tilnærming ikke er praktisk mulig eller noe sånt.

```
$ BusinessTravel           <fct> Travel_Rarely, Travel_Frequently, Travel_Rare~
$ DailyRate                <int> 1102, 279, 1373, 1392, 591, 1005, 1324, 1358,~
$ Department               <fct> Sales, Research & Development, Research & Dev~
$ DistanceFromHome         <int> 1, 8, 2, 3, 2, 2, 3, 24, 23, 27, 16, 15, 26, ~
$ Education                <int> 2, 1, 2, 4, 1, 2, 3, 1, 3, 3, 3, 2, 1, 2, 3, ~
$ EducationField           <fct> Life Sciences, Life Sciences, Other, Life Sci~
$ EmployeeNumber           <int> 1, 2, 4, 5, 7, 8, 10, 11, 12, 13, 14, 15, 16,~
$ EnvironmentSatisfaction  <int> 2, 3, 4, 4, 1, 4, 3, 4, 4, 3, 1, 4, 1, 2, 3, ~
$ Gender                   <fct> Female, Male, Male, Female, Male, Male, Femal~
$ HourlyRate               <int> 94, 61, 92, 56, 40, 79, 81, 67, 44, 94, 84, 4~
$ JobInvolvement           <int> 3, 2, 2, 3, 3, 3, 4, 3, 2, 3, 4, 2, 3, 3, 2, ~
$ JobLevel                 <int> 2, 2, 1, 1, 1, 1, 1, 1, 3, 2, 1, 2, 1, 1, 1, ~
$ JobRole                  <fct> Sales Executive, Research Scientist, Laborato~
$ JobSatisfaction          <int> 4, 2, 3, 3, 2, 4, 1, 3, 3, 3, 2, 3, 3, 4, 3, ~
$ MaritalStatus            <fct> Single, Married, Single, Married, Married, Si~
$ MonthlyIncome            <int> 5993, 5130, 2090, 2909, 3468, 3068, 2670, 269~
$ MonthlyRate              <int> 19479, 24907, 2396, 23159, 16632, 11864, 9964~
$ NumCompaniesWorked       <int> 8, 1, 6, 1, 9, 0, 4, 1, 0, 6, 0, 0, 1, 0, 5, ~
$ OverTime                 <fct> Yes, No, Yes, Yes, No, No, Yes, No, No, No, N~
$ PercentSalaryHike        <int> 11, 23, 15, 11, 12, 13, 20, 22, 21, 13, 13, 1~
$ PerformanceRating        <int> 3, 4, 3, 3, 3, 3, 4, 4, 4, 3, 3, 3, 3, 3, 3, ~
$ RelationshipSatisfaction <int> 1, 4, 2, 3, 4, 3, 1, 2, 2, 2, 3, 4, 4, 3, 2, ~
$ StockOptionLevel         <int> 0, 1, 0, 0, 1, 0, 3, 1, 0, 2, 1, 0, 1, 1, 0, ~
$ TotalWorkingYears        <int> 8, 10, 7, 8, 6, 8, 12, 1, 10, 17, 6, 10, 5, 3~
$ TrainingTimesLastYear    <int> 0, 3, 3, 3, 3, 2, 3, 2, 2, 3, 5, 3, 1, 2, 4, ~
$ WorkLifeBalance          <int> 1, 3, 3, 3, 3, 2, 2, 3, 3, 2, 3, 3, 2, 3, 3, ~
$ YearsAtCompany           <int> 6, 10, 0, 8, 2, 7, 1, 1, 9, 7, 5, 9, 5, 2, 4,~
$ YearsInCurrentRole       <int> 4, 7, 0, 7, 2, 7, 0, 0, 7, 7, 4, 5, 2, 2, 2, ~
$ YearsSinceLastPromotion  <int> 0, 1, 0, 3, 2, 3, 0, 0, 1, 7, 0, 0, 4, 1, 0, ~
$ YearsWithCurrManager     <int> 5, 7, 0, 0, 2, 6, 0, 0, 8, 7, 3, 8, 3, 2, 3, ~
```

Merk at det er en variabel vi helt sikkert ikke trenger, så vi sletter denne like gjerne med en gang: *EmployeeNumber* er et løpenummer for person. Siden det er et 1:1 forhold mellom dette og utfallsvariabelen, så bør den tas ut. Vi lager også en numerisk versjon av utfallsvariabelen som vi trenger for plotting.

```r
attrition <- attrition %>%
  select(- EmployeeNumber) %>%
  mutate(Attrition.num = as.numeric(Attrition == "Yes"))
glimpse(attrition)
```

```
Rows: 1,470
```

```
Columns: 32
$ Age                      <int> 41, 49, 37, 33, 27, 32, 59, 30, 38, 36, 35, 2~
$ Attrition                <fct> Yes, No, Yes, No, No, No, No, No, No, No, No,~
$ BusinessTravel           <fct> Travel_Rarely, Travel_Frequently, Travel_Rare~
$ DailyRate                <int> 1102, 279, 1373, 1392, 591, 1005, 1324, 1358,~
$ Department               <fct> Sales, Research & Development, Research & Dev~
$ DistanceFromHome         <int> 1, 8, 2, 3, 2, 2, 3, 24, 23, 27, 16, 15, 26, ~
$ Education                <int> 2, 1, 2, 4, 1, 2, 3, 1, 3, 3, 3, 2, 1, 2, 3, ~
$ EducationField           <fct> Life Sciences, Life Sciences, Other, Life Sci~
$ EnvironmentSatisfaction  <int> 2, 3, 4, 4, 1, 4, 3, 4, 4, 3, 1, 4, 1, 2, 3, ~
$ Gender                   <fct> Female, Male, Male, Female, Male, Male, Femal~
$ HourlyRate               <int> 94, 61, 92, 56, 40, 79, 81, 67, 44, 94, 84, 4~
$ JobInvolvement           <int> 3, 2, 2, 3, 3, 3, 4, 3, 2, 3, 4, 2, 3, 3, 2, ~
$ JobLevel                 <int> 2, 2, 1, 1, 1, 1, 1, 1, 3, 2, 1, 2, 1, 1, 1, ~
$ JobRole                  <fct> Sales Executive, Research Scientist, Laborato~
$ JobSatisfaction          <int> 4, 2, 3, 3, 2, 4, 1, 3, 3, 3, 2, 3, 3, 4, 3, ~
$ MaritalStatus            <fct> Single, Married, Single, Married, Married, Si~
$ MonthlyIncome            <int> 5993, 5130, 2090, 2909, 3468, 3068, 2670, 269~
$ MonthlyRate              <int> 19479, 24907, 2396, 23159, 16632, 11864, 9964~
$ NumCompaniesWorked       <int> 8, 1, 6, 1, 9, 0, 4, 1, 0, 6, 0, 0, 1, 0, 5, ~
$ OverTime                 <fct> Yes, No, Yes, Yes, No, No, Yes, No, No, No, N~
$ PercentSalaryHike        <int> 11, 23, 15, 11, 12, 13, 20, 22, 21, 13, 13, 1~
$ PerformanceRating        <int> 3, 4, 3, 3, 3, 3, 4, 4, 4, 3, 3, 3, 3, 3, 3, ~
$ RelationshipSatisfaction <int> 1, 4, 2, 3, 4, 3, 1, 2, 2, 2, 3, 4, 4, 3, 2, ~
$ StockOptionLevel         <int> 0, 1, 0, 0, 1, 0, 3, 1, 0, 2, 1, 0, 1, 1, 0, ~
$ TotalWorkingYears        <int> 8, 10, 7, 8, 6, 8, 12, 1, 10, 17, 6, 10, 5, 3~
$ TrainingTimesLastYear    <int> 0, 3, 3, 3, 3, 2, 3, 2, 2, 3, 5, 3, 1, 2, 4, ~
$ WorkLifeBalance          <int> 1, 3, 3, 3, 3, 2, 2, 3, 3, 2, 3, 3, 2, 3, 3, ~
$ YearsAtCompany           <int> 6, 10, 0, 8, 2, 7, 1, 1, 9, 7, 5, 9, 5, 2, 4,~
$ YearsInCurrentRole       <int> 4, 7, 0, 7, 2, 7, 0, 0, 7, 7, 4, 5, 2, 2, 2, ~
$ YearsSinceLastPromotion  <int> 0, 1, 0, 3, 2, 3, 0, 0, 1, 7, 0, 0, 4, 1, 0, ~
$ YearsWithCurrManager     <int> 5, 7, 0, 0, 2, 6, 0, 0, 8, 7, 3, 8, 3, 2, 3, ~
$ Attrition.num            <dbl> 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ~
```

Del datasettet i to deler. Vi trekker tilfeldig 70% og legger dette i datasettet training. Resten legges i testing.

```r
set.seed(426)
attrition_split <- initial_split(attrition)
training <- training(attrition_split)
testing  <- testing(attrition_split)
```

Sjekk at antallet i hvert datasett summeres til totalen

```r
nrow(attrition)
```

```
[1] 1470
```

```r
nrow(training)
```

```
[1] 1102
```

```r
nrow(testing)
```

```
[1] 368
```

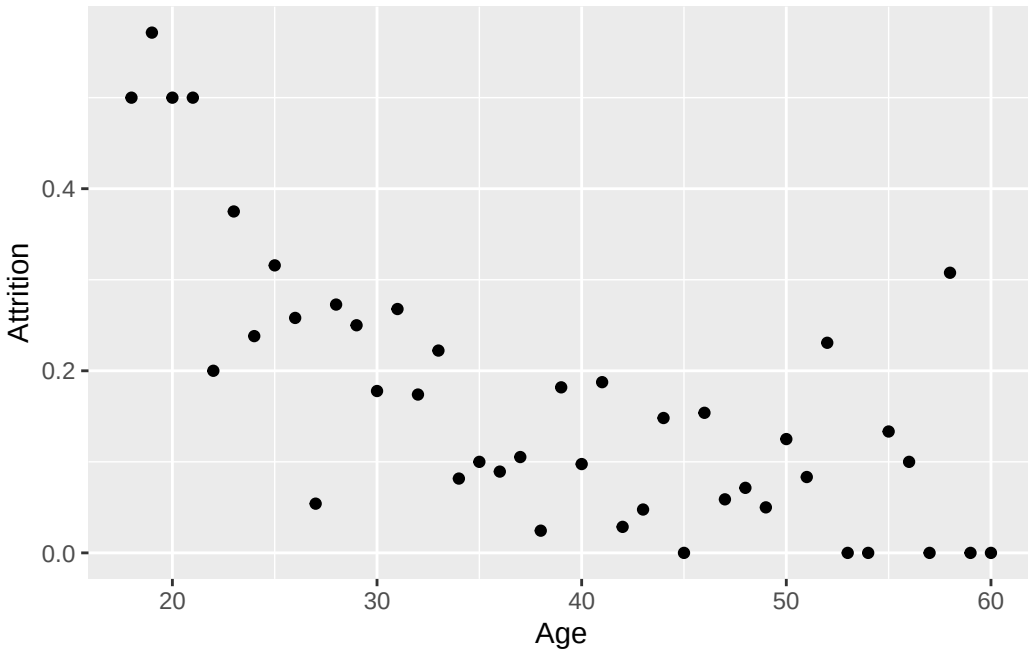
Andelen som slutter kan vi få med `mean()`:

```r
mean(training$Attrition.num)
```

```
[1] 0.1533575
```

Vi kan vise hvordan det å slutte i jobben varierer med f.eks. alder ved å beregne andel per verdi av alder.

```r
training_p <- training %>%
  group_by(Age) %>%
  summarise(Attrition = mean(Attrition.num))
ggplot(training_p, aes(x=Age, y=Attrition))+
  geom_point()
```



3.2 Estimere en sannsynlighet

Når utfallsvariabelen er binær (to verdier) kan man likevel bruke lineær regresjon. Det kalles da gjerne en lineær sannsynlighetsmodell.

```
lm_est <- lm(Attrition.num ~ Age , data = training)
summary(lm_est)
```

```
Call:
lm(formula = Attrition.num ~ Age, data = training)

Residuals:
     Min       1Q   Median       3Q      Max 
-0.28444 -0.18737 -0.14576 -0.06256  0.99292 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  0.409254   0.044335   9.231  < 2e-16 ***
Age         -0.006934   0.001166  -5.948 3.65e-09 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.355 on 1100 degrees of freedom
Multiple R-squared:  0.03116,   Adjusted R-squared:  0.03027
F-statistic: 35.37 on 1 and 1100 DF,  p-value: 3.652e-09
```

Den viktigste ulempen med lineære sannsynlighetsmodeller er at modellen da kan predikere sannsynligheter lavere enn 0 og høyere enn 1. Når man er mest interessert i β er det ikke sikkert det er så nøye. Men når vi er interessert i $\hat{y}$ kan det derimot være viktig.

3.3 Logistisk regresjon i R

Logistisk regresjon har det til felles med lineær regresjon at utfallet er en lineær spesifikasjon.

$$log(\frac{\pi}{1-\pi}) = \alpha + \beta X$$

Venstresiden av ligningen kalles en *logit*, der π er en sannsynlighet. Uttrykket $\frac{\pi}{(1-\pi)}$ er en *odds*, som er et forholdstall mellom sannsynligheten for at utfallet skjer mot sannsynligheten for det motsatte. Tolkningen av β er da en endring av *odds* på logaritisk skala. Hvis man eksponensierer β er den da tolkbar som en *oddsrate*.

Man kan regne om til sannsynligheter som er vesentlig enklere å forstå. Ligningen kan skrives om slik:

$$\hat{\pi} = \frac{e^{\alpha+\beta X}}{1+e^{\alpha+\beta X}}$$

En relativt enkel omregning av regresjonsresultatet gir altså en *sannynlighet*. Denne sannsynligheten kan vi da bruke til *klassifikasjon* hvis det er formålet med analysen. Hvis utfallsvariabelen har to kategorier, så er en nærliggende mulighet å klassifisere til den gruppen hver person mest sannsynlig tilhører. Altså: de som har $\hat{\pi} = P(y = 1) > 0.5$ tilhører den ene gruppen og resten i den andre gruppen.

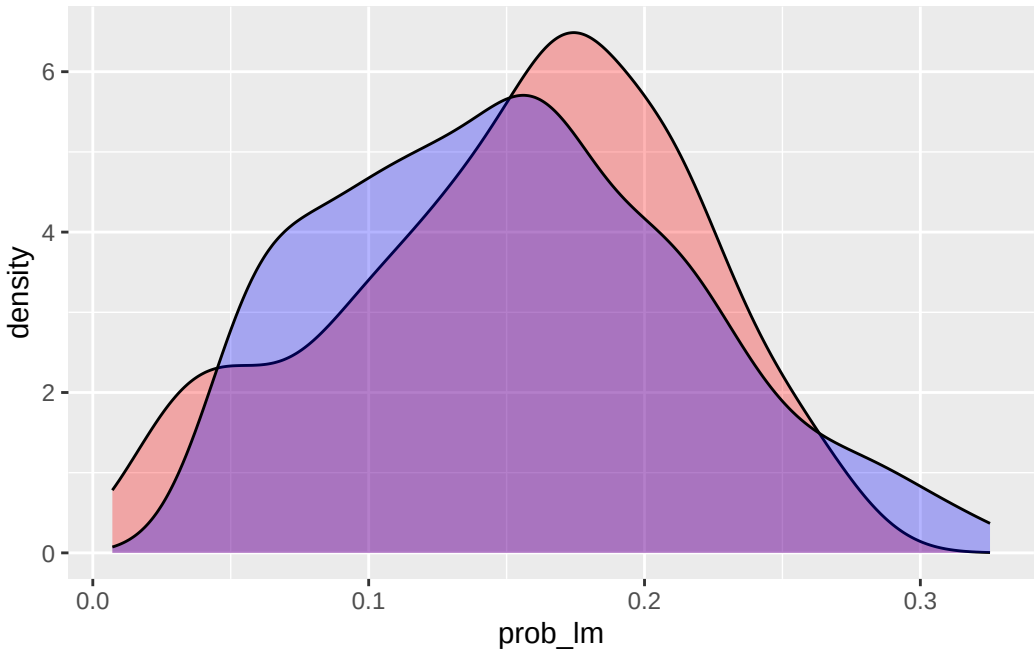
Her er et eksempel på hvordan estimere enkel logistisk regresjon i R:

```r
est_logit <- glm(Attrition ~ Age, data = training, family = "binomial")
summary(est_logit)
```

```
Call:
glm(formula = Attrition ~ Age, family = "binomial", data = training)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.41516    0.36402   1.140    0.254
Age         -0.06026    0.01050  -5.741 9.43e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 944.39  on 1101  degrees of freedom
Residual deviance: 907.48  on 1100  degrees of freedom
AIC: 911.48

Number of Fisher Scoring iterations: 5
```

Vi kan sammenligne lineær og logistisk modell for å se forskjellen i predikert sannsynlighet. Den røde kurven er den lineære modellen og den blå er den logistiske:

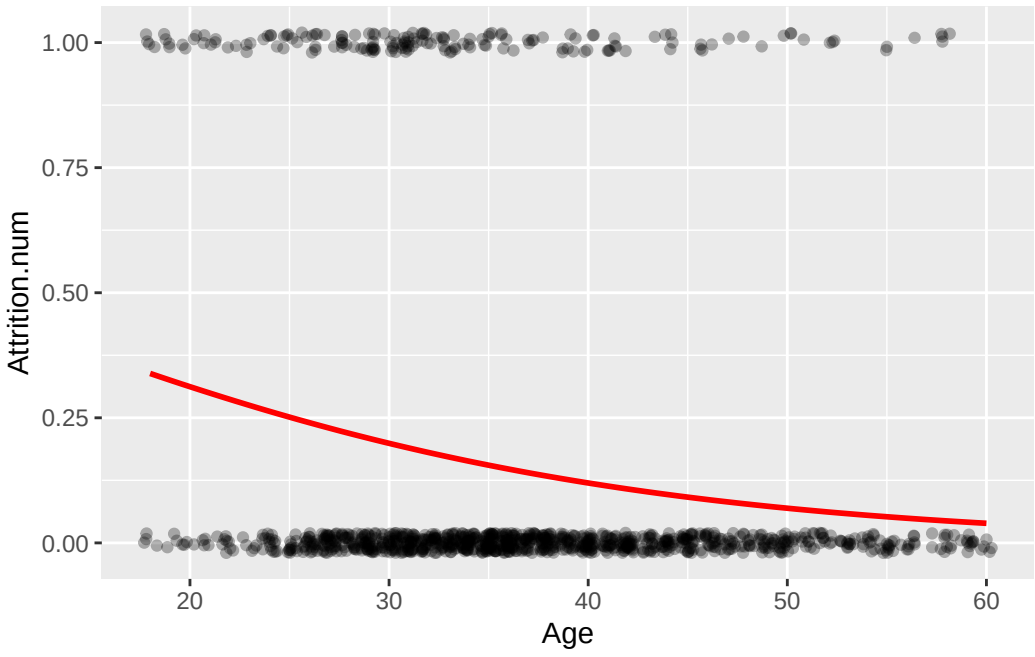
```r
testing <- testing %>%
  mutate(prob_lm = predict(lm_est, newdata = testing, type = "response"),
         prob_glm = predict(est_logit, newdata = testing, type = "response"))

ggplot(testing, aes(x = prob_lm)) +
  geom_density(fill = "red", alpha = .3) +
  geom_density(aes(x = prob_glm), fill = "blue", alpha = .3)
```



Forskjellen er som regel ikke så stor i midten av fordelingen, men den lineære modellen kan gi verdier utenfor 0-1-intervallet. For klassifikasjon spiller det sjelden noen stor rolle, men for å være korrekt bruker vi logistisk regresjon når utfallet er binært.

Hvordan plotte slike data? Bruk `geom_jitter` eller `geom_point`.

```
ggplot(training, aes(x=Age, y=Attrition.num))+
  geom_jitter(height = .02, alpha=.3)+
  stat_smooth(method="glm", method.args=list(family="binomial"), se=FALSE, col="red")
```



3.4 Prediksjon

Vi kan predikere med bruk av `predict()` som tidligere. Nå er det viktig å presisere `type = "response"` for å få sannsynligheter i stedet for log-odds.

```
attrition_pred <- training %>%
  mutate(prob = predict(est_logit, type = "response"))
```

3.4.1 ROC og AUC

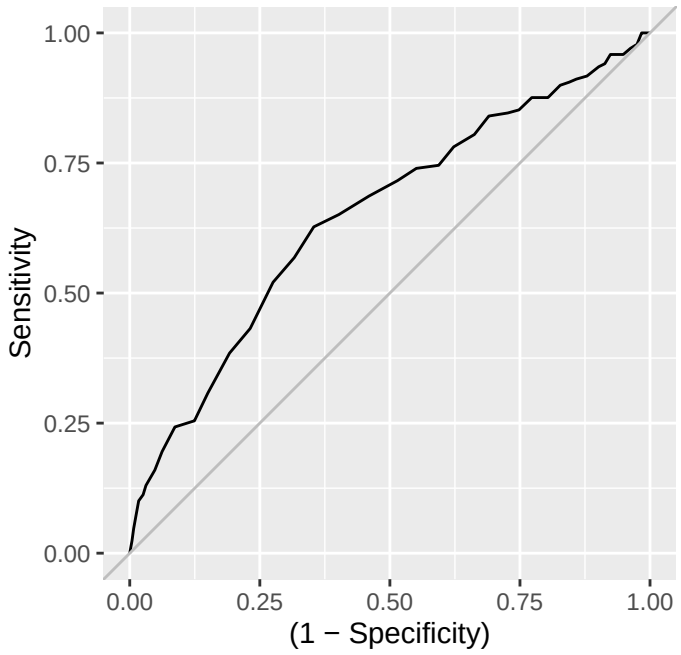
For å vurdere hvor god prediksjonen er *uavhengig* av valg av cut-off bruker vi ROC-kurven (Receiver Operating Characteristic). ROC-kurven viser sammenhengen mellom sensitivity (andelen sanne positive) og 1-specificity (andelen falske positive) for alle mulige verdier av cut-off. En modell som predikerer perfekt vil gi en kurve som går rett opp og deretter bort til høyre. En modell som gjetter tilfeldig vil ligge langs den diagonale grå linjen.

```
ROC <- roc( attrition_pred$Attrition, attrition_pred$prob )
```

```
Setting levels: control = No, case = Yes
```

```
Setting direction: controls < cases
```

```
df <- data.frame(Sensitivity = ROC$sensitivities,
                 Specificity = ROC$specificities)

ggplot(df, aes(y = Sensitivity, x= (1-Specificity))) +
  geom_line() +
  geom_abline(intercept = 0, slope = 1, col = "gray")+
  coord_equal()
```



Arealet under ROC-kurven (AUC) gir et enkelt tall som oppsummerer modellens prediksjonsevne. AUC varierer mellom 0.5 (tilfeldig gjetting) og 1.0 (perfekt prediksjon). For den enkle modellen er AUC 0.65.

3.5 Multippel logistisk regresjon

Vi kan estimere en multippel regresjon på tilsvarende måte som for lineær regresjon ved å legge til flere variable eller angi å bruke samtlige variable i datasettet med `Attrition ~ .`.

```
est_multlogit <- glm(Attrition ~ ., data = training[, !names(training) %in% "Attrition.num"]
summary(est_multlogit)
```

```
Call:
glm(formula = Attrition ~ ., family = "binomial", data = training[,
    !names(training) %in% "Attrition.num"])

Coefficients:
                                  Estimate Std. Error z value Pr(>|z|)
(Intercept)                     -1.035e+01  4.542e+02  -0.023 0.981824
Age                             -3.565e-02  1.628e-02  -2.190 0.028495 *
BusinessTravelTravel_Frequently  1.628e+00  4.675e-01   3.482 0.000498 ***
BusinessTravelTravel_Rarely      8.988e-01  4.267e-01   2.106 0.035192 *
DailyRate                       -3.117e-04  2.648e-04  -1.177 0.239034
DepartmentResearch & Development 1.203e+01  4.542e+02   0.026 0.978861
DepartmentSales                  1.239e+01  4.542e+02   0.027 0.978244
DistanceFromHome                 4.145e-02  1.300e-02   3.189 0.001428 **
Education                       -1.376e-02  1.039e-01  -0.132 0.894627
EducationFieldLife Sciences     -8.013e-01  1.027e+00  -0.781 0.435083
EducationFieldMarketing         -2.129e-01  1.076e+00  -0.198 0.843199
EducationFieldMedical           -8.617e-01  1.028e+00  -0.838 0.401983
EducationFieldOther             -7.977e-01  1.098e+00  -0.727 0.467345
EducationFieldTechnical Degree   1.370e-01  1.048e+00   0.131 0.896014
EnvironmentSatisfaction         -4.957e-01  9.942e-02  -4.986 6.18e-07 ***
GenderMale                       2.156e-01  2.149e-01   1.003 0.315665
HourlyRate                      -3.532e-03  5.385e-03  -0.656 0.511937
JobInvolvement                  -5.422e-01  1.452e-01  -3.734 0.000189 ***
JobLevel                         1.864e-02  3.817e-01   0.049 0.961047
JobRoleHuman Resources           1.308e+01  4.542e+02   0.029 0.977028
JobRoleLaboratory Technician     1.531e+00  5.543e-01   2.762 0.005746 **
JobRoleManager                  -1.451e+00  1.269e+00  -1.144 0.252636
JobRoleManufacturing Director   -3.131e-01  6.086e-01  -0.514 0.606923
JobRoleResearch Director        -2.440e+00  1.244e+00  -1.961 0.049843 *
JobRoleResearch Scientist        3.417e-01  5.694e-01   0.600 0.548406
JobRoleSales Executive           3.595e-01  1.603e+00   0.224 0.822589
JobRoleSales Representative      1.518e+00  1.650e+00   0.920 0.357555
JobSatisfaction                 -3.169e-01  9.701e-02  -3.267 0.001087 **
MaritalStatusMarried             3.453e-01  3.135e-01   1.101 0.270697
MaritalStatusSingle              8.690e-01  4.056e-01   2.143 0.032140 *
MonthlyIncome                    8.613e-05  9.875e-05   0.872 0.383105
MonthlyRate                      8.922e-06  1.482e-05   0.602 0.547277
NumCompaniesWorked               1.721e-01  4.623e-02   3.722 0.000198 ***
OverTimeYes                      1.877e+00  2.296e-01   8.174 2.98e-16 ***
PercentSalaryHike               -6.102e-02  4.687e-02  -1.302 0.192953
PerformanceRating                6.920e-01  4.794e-01   1.443 0.148883
```

```
RelationshipSatisfaction         -3.738e-01  9.827e-02  -3.804 0.000142 ***
StockOptionLevel                 -3.059e-01  1.810e-01  -1.690 0.091064 .
TotalWorkingYears                -6.605e-02  3.580e-02  -1.845 0.065052 .
TrainingTimesLastYear            -2.129e-01  8.533e-02  -2.495 0.012582 *
WorkLifeBalance                  -2.150e-01  1.456e-01  -1.476 0.139869
YearsAtCompany                    6.067e-02  4.877e-02   1.244 0.213537
YearsInCurrentRole               -9.948e-02  5.569e-02  -1.786 0.074038 .
YearsSinceLastPromotion           2.052e-01  5.182e-02   3.960 7.50e-05 ***
YearsWithCurrManager             -1.866e-01  6.072e-02  -3.073 0.002122 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 944.39  on 1101  degrees of freedom
Residual deviance: 619.99  on 1057  degrees of freedom
AIC: 709.99

Number of Fisher Scoring iterations: 14
```

3.5.1 ROC og AUC

For å beregne ROC og AUC gjør vi tilsvarende som over med predict og angi type respons.

```r
attrition_pred <- training %>%
  mutate(prob = predict(est_multlogit, type = "response"))
```

Funksjonen `roc()` gjør utregningene som trengs for ROC-kurven basert på observert utfall og predikerte sannsynligheter (Hsieh 2008).

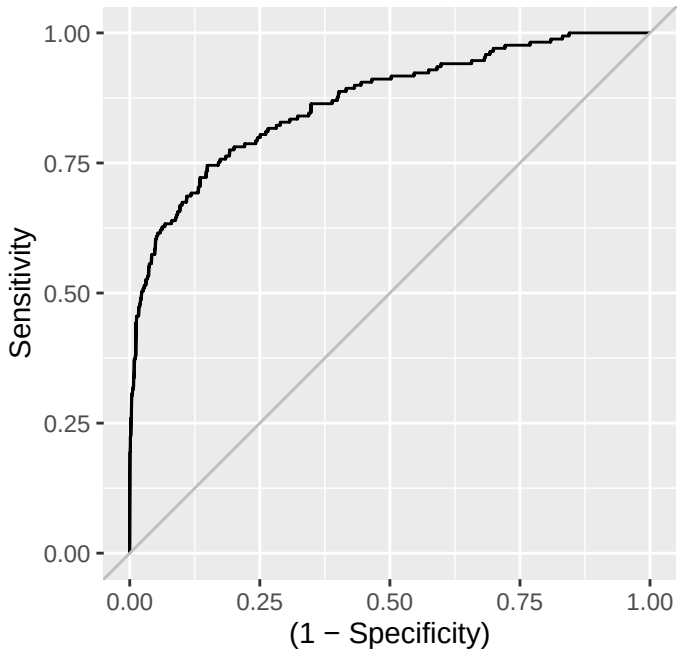
OBS! Man må man angi data som *første* argument i funksjonen `roc()`, deretter observerte utfall og til sist predikert sannsynlighet. Rekkefølgen er viktig!

```r
ROC <- roc(attrition_pred, Attrition, prob)
```

```
Setting levels: control = No, case = Yes
```

```
Setting direction: controls < cases
```

```r
df <- data.frame(Sensitivity = ROC$sensitivities,
                 Specificity = ROC$specificities)

ggplot(df, aes(y = Sensitivity, x= (1-Specificity))) +
  geom_line() +
  geom_abline(intercept = 0, slope = 1, col = "gray")+
  coord_equal()
```



Vi kan da få rapportert arealet under kurven med `auc()` slik:

```r
auc(ROC)
```

```
Area under the curve: 0.8675
```

Når arealet under kurven (AUC) er 0.867 er det vesentlig bedre prediksjon enn den enkle modellen.

3.6 Testing-data

Ovenfor er øvelsen gjort på training-data, men vi må sjekke på testing-dataene.

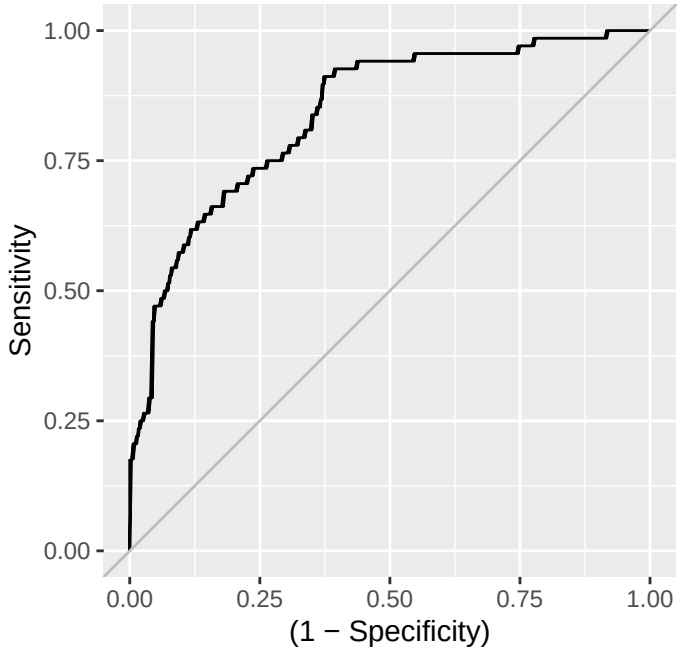
For å beregne ROC og AUC gjør vi tilsvarende som over med predict, men nå er det viktig å angi `newdata = ...` slik at prediksjonen gjøres på riktig datasett.

```
attrition_test <- testing %>%
  mutate(prob = predict(est_multlogit, newdata = testing, type = "response"))
```

```
ROC_test <- roc(attrition_test, Attrition, prob)
```

```
Setting levels: control = No, case = Yes

Setting direction: controls < cases
```

```
df <- data.frame(Sensitivity = ROC_test$sensitivities,
                 Specificity = ROC_test$specificities)

ggplot(df, aes(y = Sensitivity, x= (1-Specificity))) +
  geom_line() +
  geom_abline(intercept = 0, slope = 1, col = "gray")+
  coord_equal()
```



Vi kan da få rapportert arealet under kurven med `auc()` slik:

```r
auc(ROC_test)
```

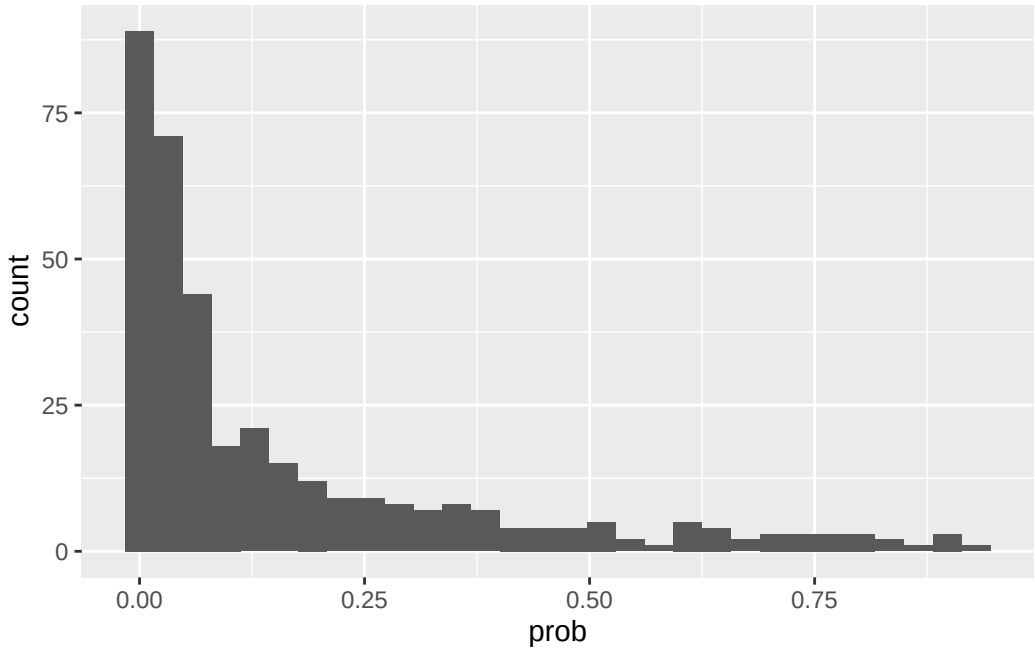
```
Area under the curve: 0.8397
```

Når arealet under kurven (AUC) er 0.84. Kanskje litt overraskende, men dette like godt som for på training dataene. AUC er altså *arealet under kurven*. Litt ulik form kan i prinsippet ha samme areal.

3.7 Klassifikasjon

Men for et handlingsvalg må vi gjøre faktisk klassifisering. Det vi har estimert så langt er bare en sannsynlighet. Selve klassifiseringen krever at man tar et aktivt valg om en cut-off for hvem man tror faktisk slutter. La oss først se på fordelingen av predikerte sannsynligheter:

```r
ggplot(attrition_test, aes(x = prob)) +
  geom_histogram()
```



De aller fleste har lav predikert sannsynlighet for å slutte. Det er som forventet siden de fleste faktisk *ikke* slutter. Spørsmålet er nå: ved hvilken sannsynlighet skal vi si at noen er i faresonen?

Vi kan bestemme oss for at et rimelig cut-off er 50/50, altså med sannsynlighet 0.5. Her gjøres en klassifisering for testing-datasettet, og lager en krysstabell med den klassifiserte etter prediksjon mot observert utfall. En slik krysstabell kalles altså en *confusion matrix*.

```
attrition_test <- attrition_test %>%
  mutate(attrition_class = as.factor(ifelse(prob < .5, "No", "Yes")))

tab <- attrition_test %>%
  select(attrition_class, Attrition) %>%
  table()

tab
```

```
               Attrition
attrition_class  No Yes
            No  287  45
            Yes  13  23
```

3.7.1 Confusion matrix

Fra pakken `caret` er det en funksjon for confusion matrix som regner ut masse greier for oss. Merk at `positive = "Yes"` angir hva som er utfallsverdi av interesse.

```
confusionMatrix(reference = attrition_test$Attrition, attrition_test$attrition_class, positiv
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  287  45
       Yes  13  23

               Accuracy : 0.8424
                 95% CI : (0.8011, 0.8781)
    No Information Rate : 0.8152
    P-Value [Acc > NIR] : 0.09925

                  Kappa : 0.3605

 Mcnemar's Test P-Value : 4.691e-05
```

```
            Sensitivity : 0.33824
            Specificity : 0.95667
         Pos Pred Value : 0.63889
         Neg Pred Value : 0.86446
             Prevalence : 0.18478
         Detection Rate : 0.06250
   Detection Prevalence : 0.09783
      Balanced Accuracy : 0.64745

       'Positive' Class : Yes
```

3.8 Hvor feil kan man ta?

I klassifiseringen over er det gjort et klart valg for hvem man tror faktisk vil slutte i jobben eller ikke. Det er selvsagt slik at noen er mer sannsynlige vil slutte enn andre, men ingen har 0 sannsynlighet. Ingen har 1 heller, for den saks skyld. Det er altså usikkerhet. Men hvis vi skal gjøre et *tiltak*, så må vi ta det valget!

Hvis vi gjør klassifiseringen på 0.5 som over, så betyr jo det at vi synes begge feil er like viktige: Falske positive eller falske negative. Hvis det er et langt større problem at folk slutter enn at noen f.eks. får tilbud om goder eller ekstra oppfølging etc, så kan det hende cut-off skal settes lavere? Da får man flere sanne positive, men også flere feil. Det kan det jo være verd, men kommer jo an på hva konsekvensene. Vi kommer tilbake til dette, men test gjerne ut selv med ulik cut-off og se hvordan resultatene endrer seg.

4 Klassifikasjonstrær

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(rpart)       # funksjoner for CART
library(rpart.plot)  # funksjon for å plotte CART
library(caret)       # inneholder funksjon for confusion matrix
```

Klassifikasjonstrær (CART: Classification and Regression Trees) er en helt annen tilnærming enn regresjon. I stedet for å estimere koeffisienter i en ligning, deler algoritmen dataene i stadig mindre grupper basert på verdier av prediktorene. Resultatet er et "tre" som kan leses som en serie if-else-regler. Fordelen er at trær er enkle å tolke og kan fange opp ikke-lineære sammenhenger og interaksjoner automatisk. Ulempen er at enkelttrær kan være ustabile og har en tendens til overfitting.

Vi skal her bruke Attrition-datasettet fra kapittelet om logistisk regresjon. Utfallsvariabelen er `Attrition` som angir om en arbeidstaker slutter i jobben ("Yes") eller ikke ("No"). Vi starter med å lage en logistisk regresjonsmodell som sammenligningsgrunnlag, og deretter bygger vi klassifikasjonstrær.

4.1 Lese inn data

```
attrition <- readRDS("data/attrition.rds") %>%
  select(-EmployeeNumber)
glimpse(attrition)
```

```
Rows: 1,470
Columns: 31
$ Age                     <int> 41, 49, 37, 33, 27, 32, 59, 30, 38, 36, 35, 2~
$ Attrition               <fct> Yes, No, Yes, No, No, No, No, No, No, No, No,~
$ BusinessTravel          <fct> Travel_Rarely, Travel_Frequently, Travel_Rare~
$ DailyRate               <int> 1102, 279, 1373, 1392, 591, 1005, 1324, 1358,~
```

```
$ Department               <fct> Sales, Research & Development, Research & Dev~
$ DistanceFromHome         <int> 1, 8, 2, 3, 2, 2, 3, 24, 23, 27, 16, 15, 26, ~
$ Education                <int> 2, 1, 2, 4, 1, 2, 3, 1, 3, 3, 3, 2, 1, 2, 3, ~
$ EducationField           <fct> Life Sciences, Life Sciences, Other, Life Sci~
$ EnvironmentSatisfaction  <int> 2, 3, 4, 4, 1, 4, 3, 4, 4, 3, 1, 4, 1, 2, 3, ~
$ Gender                   <fct> Female, Male, Male, Female, Male, Male, Femal~
$ HourlyRate               <int> 94, 61, 92, 56, 40, 79, 81, 67, 44, 94, 84, 4~
$ JobInvolvement           <int> 3, 2, 2, 3, 3, 3, 4, 3, 2, 3, 4, 2, 3, 3, 2, ~
$ JobLevel                 <int> 2, 2, 1, 1, 1, 1, 1, 1, 3, 2, 1, 2, 1, 1, 1, ~
$ JobRole                  <fct> Sales Executive, Research Scientist, Laborato~
$ JobSatisfaction          <int> 4, 2, 3, 3, 2, 4, 1, 3, 3, 3, 2, 3, 3, 4, 3, ~
$ MaritalStatus            <fct> Single, Married, Single, Married, Married, Si~
$ MonthlyIncome            <int> 5993, 5130, 2090, 2909, 3468, 3068, 2670, 269~
$ MonthlyRate              <int> 19479, 24907, 2396, 23159, 16632, 11864, 9964~
$ NumCompaniesWorked       <int> 8, 1, 6, 1, 9, 0, 4, 1, 0, 6, 0, 0, 1, 0, 5, ~
$ OverTime                 <fct> Yes, No, Yes, Yes, No, No, Yes, No, No, No, N~
$ PercentSalaryHike        <int> 11, 23, 15, 11, 12, 13, 20, 22, 21, 13, 13, 1~
$ PerformanceRating        <int> 3, 4, 3, 3, 3, 3, 4, 4, 4, 3, 3, 3, 3, 3, 3, ~
$ RelationshipSatisfaction <int> 1, 4, 2, 3, 4, 3, 1, 2, 2, 2, 3, 4, 4, 3, 2, ~
$ StockOptionLevel         <int> 0, 1, 0, 0, 1, 0, 3, 1, 0, 2, 1, 0, 1, 1, 0, ~
$ TotalWorkingYears        <int> 8, 10, 7, 8, 6, 8, 12, 1, 10, 17, 6, 10, 5, 3~
$ TrainingTimesLastYear    <int> 0, 3, 3, 3, 3, 2, 3, 2, 2, 3, 5, 3, 1, 2, 4, ~
$ WorkLifeBalance          <int> 1, 3, 3, 3, 3, 2, 2, 3, 3, 2, 3, 3, 2, 3, 3, ~
$ YearsAtCompany           <int> 6, 10, 0, 8, 2, 7, 1, 1, 9, 7, 5, 9, 5, 2, 4,~
$ YearsInCurrentRole       <int> 4, 7, 0, 7, 2, 7, 0, 0, 7, 7, 4, 5, 2, 2, 2, ~
$ YearsSinceLastPromotion  <int> 0, 1, 0, 3, 2, 3, 0, 0, 1, 7, 0, 0, 4, 1, 0, ~
$ YearsWithCurrManager     <int> 5, 7, 0, 0, 2, 6, 0, 0, 8, 7, 3, 8, 3, 2, 3, ~
```

Vi splitter datasettet i training og testing:

```r
set.seed(426)
attrition_split <- initial_split(attrition, prop = .7)
training <- training(attrition_split)
testing  <- testing(attrition_split)
```

4.2 Logistisk regresjon som baseline

Før vi lager klassifikasjonstrær er det nyttig å ha et sammenligningsgrunnlag. Vi bruker logistisk regresjon med alle variable:

```r
est.glm <- glm(Attrition ~ ., data = training, family = "binomial")
summary(est.glm)
```

```

Call:
glm(formula = Attrition ~ ., family = "binomial", data = training)

Coefficients:
                                  Estimate Std. Error z value Pr(>|z|)
(Intercept)                     -1.171e+01  7.504e+02  -0.016 0.987554
Age                             -3.647e-02  1.688e-02  -2.161 0.030667 *
BusinessTravelTravel_Frequently  1.648e+00  4.983e-01   3.308 0.000940 ***
BusinessTravelTravel_Rarely      9.891e-01  4.541e-01   2.178 0.029387 *
DailyRate                       -3.576e-04  2.753e-04  -1.299 0.193941
DepartmentResearch & Development 1.298e+01  7.504e+02   0.017 0.986196
DepartmentSales                  1.357e+01  7.504e+02   0.018 0.985571
DistanceFromHome                 4.746e-02  1.361e-02   3.488 0.000486 ***
Education                       -1.807e-02  1.084e-01  -0.167 0.867654
EducationFieldLife Sciences     -3.399e-01  1.129e+00  -0.301 0.763441
EducationFieldMarketing          3.270e-01  1.179e+00   0.277 0.781525
EducationFieldMedical           -5.647e-01  1.133e+00  -0.498 0.618260
EducationFieldOther             -5.878e-01  1.207e+00  -0.487 0.626217
EducationFieldTechnical Degree   5.805e-01  1.154e+00   0.503 0.615076
EnvironmentSatisfaction         -5.414e-01  1.053e-01  -5.140 2.74e-07 ***
GenderMale                       2.498e-01  2.252e-01   1.109 0.267218
HourlyRate                      -2.457e-03  5.598e-03  -0.439 0.660717
JobInvolvement                  -5.900e-01  1.538e-01  -3.835 0.000126 ***
JobLevel                         1.022e-01  3.942e-01   0.259 0.795375
JobRoleHuman Resources           1.399e+01  7.504e+02   0.019 0.985126
JobRoleLaboratory Technician     1.562e+00  5.660e-01   2.760 0.005777 **
JobRoleManager                  -1.454e+00  1.269e+00  -1.145 0.252018
JobRoleManufacturing Director   -3.797e-01  6.172e-01  -0.615 0.538366
JobRoleResearch Director        -2.476e+00  1.256e+00  -1.971 0.048692 *
JobRoleResearch Scientist        2.632e-01  5.858e-01   0.449 0.653273
JobRoleSales Executive          -1.863e-01  1.594e+00  -0.117 0.906944
JobRoleSales Representative      1.378e+00  1.648e+00   0.836 0.403020
JobSatisfaction                 -3.369e-01  1.022e-01  -3.298 0.000975 ***
MaritalStatusMarried             2.514e-01  3.242e-01   0.775 0.438093
MaritalStatusSingle              9.638e-01  4.225e-01   2.281 0.022524 *
MonthlyIncome                    8.287e-05  1.030e-04   0.804 0.421245
MonthlyRate                      6.922e-06  1.558e-05   0.444 0.656792
NumCompaniesWorked               1.638e-01  4.800e-02   3.413 0.000642 ***
```

```
OverTimeYes                      2.007e+00  2.437e-01   8.236  < 2e-16 ***
PercentSalaryHike               -6.251e-02  4.976e-02  -1.256 0.209020
PerformanceRating                6.650e-01  5.094e-01   1.305 0.191744
RelationshipSatisfaction        -3.787e-01  1.031e-01  -3.674 0.000239 ***
StockOptionLevel                -2.105e-01  1.875e-01  -1.123 0.261579
TotalWorkingYears               -5.930e-02  3.680e-02  -1.611 0.107130
TrainingTimesLastYear           -1.856e-01  9.002e-02  -2.061 0.039275 *
WorkLifeBalance                 -1.886e-01  1.525e-01  -1.236 0.216394
YearsAtCompany                   4.602e-02  5.114e-02   0.900 0.368242
YearsInCurrentRole              -1.046e-01  5.771e-02  -1.812 0.069940 .
YearsSinceLastPromotion          2.017e-01  5.441e-02   3.707 0.000210 ***
YearsWithCurrManager            -1.745e-01  6.343e-02  -2.752 0.005927 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 875.64  on 1028  degrees of freedom
Residual deviance: 566.77  on  984  degrees of freedom
AIC: 656.77

Number of Fisher Scoring iterations: 15
```

Vi predikerer på testing-data og gjør en klassifisering med cut-off på 0.5:

```r
testing_glm <- testing %>%
  mutate(prob_glm = predict(est.glm, newdata = testing, type = "response")) %>%
  mutate(klassifiser = factor(ifelse(prob_glm < .5, "No", "Yes")))

confusionMatrix(reference = testing_glm$Attrition,
                testing_glm$klassifiser,
                positive = "Yes")
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  345  52
       Yes  15  29

               Accuracy : 0.8481
                 95% CI : (0.8111, 0.8803)
```

```
    No Information Rate : 0.8163
    P-Value [Acc > NIR] : 0.04596

                  Kappa : 0.3844

 Mcnemar's Test P-Value : 1.092e-05

            Sensitivity : 0.35802
            Specificity : 0.95833
         Pos Pred Value : 0.65909
         Neg Pred Value : 0.86902
             Prevalence : 0.18367
         Detection Rate : 0.06576
   Detection Prevalence : 0.09977
      Balanced Accuracy : 0.65818

       'Positive' Class : Yes
```

Denne confusion matrix gir oss et referansepunkt for å vurdere klassifikasjonstrærne.

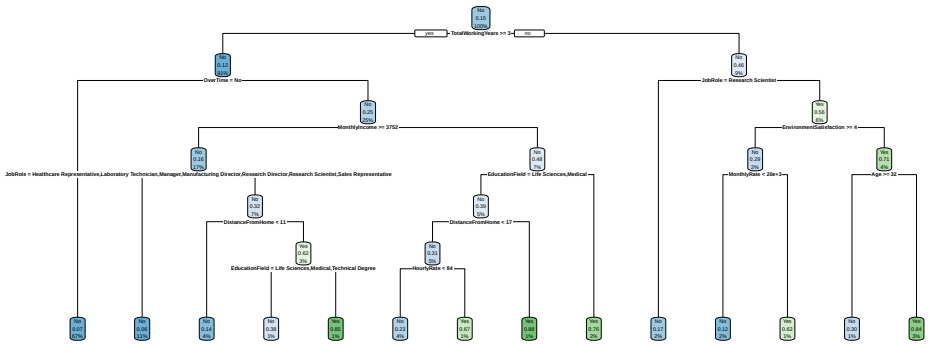
4.3 Klassifikasjonstre

Utfallsvariabel og prediktorer spesifiseres som en formel på samme måte som for regresjon. Siden vi her har en klassifikasjon må vi spesifisere `method = "class"`. Hvis ikke vil `rpart()` gjette hva slags modell (som kanskje er riktig), så du kan få andre resultater enn du forventet.

```
klass_tre <- rpart(Attrition ~ .,
                   data = training, method = "class")
```

Resultatet kan fremstilles grafisk med funksjonen `rpart.plot()` slik:

```
rpart.plot(klass_tre)
```



Treet leses fra toppen og nedover. I hver node vises den predikerte klassen, andelen som tilhører denne klassen, og andelen av totalen som havner i denne noden. Hver split angir et kriterium: observasjoner som oppfyller kriteriet går til venstre, resten til høyre. Bladnodene (nederst) gir den endelige klassifiseringen.

Vi kan også få printet ut resultatet som regler i en tabell med `rpart.rules()`:

```
rpart.rules(klass_tre, extra = 4)
```

```
 Attrition   No Yes
        No [.94 .06] when TotalWorkingYears >= 3 & OverTime is Yes & JobRole is Healthcare Re
        No [.93 .07] when TotalWorkingYears >= 3 & OverTime is  No
        No [.88 .12] when TotalWorkingYears <  3                  & JobRole is
        No [.86 .14] when TotalWorkingYears >= 3 & OverTime is Yes & JobRole is
        No [.83 .17] when TotalWorkingYears <  3                  & JobRole is
        No [.77 .23] when TotalWorkingYears >= 3 & OverTime is Yes
        No [.70 .30] when TotalWorkingYears <  3                  & JobRole is
        No [.62 .38] when TotalWorkingYears >= 3 & OverTime is Yes & JobRole is
       Yes [.38 .62] when TotalWorkingYears <  3                  & JobRole is
       Yes [.33 .67] when TotalWorkingYears >= 3 & OverTime is Yes
       Yes [.24 .76] when TotalWorkingYears >= 3 & OverTime is Yes
       Yes [.16 .84] when TotalWorkingYears <  3                  & JobRole is
       Yes [.15 .85] when TotalWorkingYears >= 3 & OverTime is Yes & JobRole is
       Yes [.12 .88] when TotalWorkingYears >= 3 & OverTime is Yes
```

Da kan vi sammenligne prediksjoner med observert utfall for testingdataene. I `predict()` må det angis `type = "class"` for å spesifisere at det skal være klassifikasjon.

```
testing_pred <- testing %>%
  mutate(Attrition_pred = predict(klass_tre, newdata = testing, type = "class"))


confusionMatrix(reference = testing_pred$Attrition, testing_pred$Attrition_pred, positive = "
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  342  67
       Yes  18  14

               Accuracy : 0.8073
                 95% CI : (0.7673, 0.843)
    No Information Rate : 0.8163
    P-Value [Acc > NIR] : 0.7131

                  Kappa : 0.1605

 Mcnemar's Test P-Value : 1.926e-07

            Sensitivity : 0.17284
            Specificity : 0.95000
         Pos Pred Value : 0.43750
         Neg Pred Value : 0.83619
             Prevalence : 0.18367
         Detection Rate : 0.03175
   Detection Prevalence : 0.07256
      Balanced Accuracy : 0.56142

       'Positive' Class : Yes
```

Sammenlign dette resultatet med logistisk regresjon-baseline ovenfor. Spesielt interessant er accuracy, sensitivity (andelen av de som faktisk slutter som vi fanger opp) og specificity (andelen av de som blir som vi riktig klassifiserer). Et enkelt tre med forvalgte parametre gjør det ofte ganske greit, men vi kan forsøke å forbedre det med tuning.

4.4 Tuning/pruning

Vi har vært inne på tidligere at vi kan styre hvordan algoritmen fungerer. Det er noen parametres om styrer prosessen, og disse kan vi justere. Her tar vi for oss de viktigste.

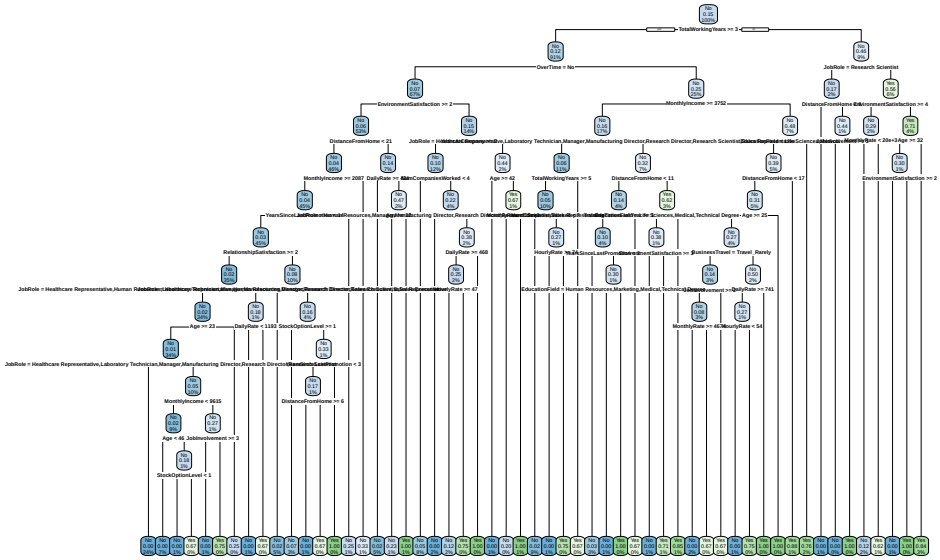
Kort fortalt styrer disse parametrene hvor komplekse trærne kan bli. Husk nå fra tidligere kapittel: mer kompleks modell gir bedre tilpassning til trainingdata - men kan gi dårligere tilpassning til testingdata. Målet er altså å finne en slags balansert kompleksitet.

Nå er arbeidsflyten slik at man skrur litt på disse parametrene, sjekker resultatet og justerer på nytt og sjekker… osv. Da er det viktig at bruker trainingdata! Ikke bruke testingdata før du er rimelig fornøyd med resultatet.

4.4.1 Bruk av `cp = ...`

`cp` er complexity parameter som setter et krav på hvor mye hver split skal bidra til modellens tilpassning til data. Forvalget er 0.01. Med en lavere verdi tillates mer komplekse trær:

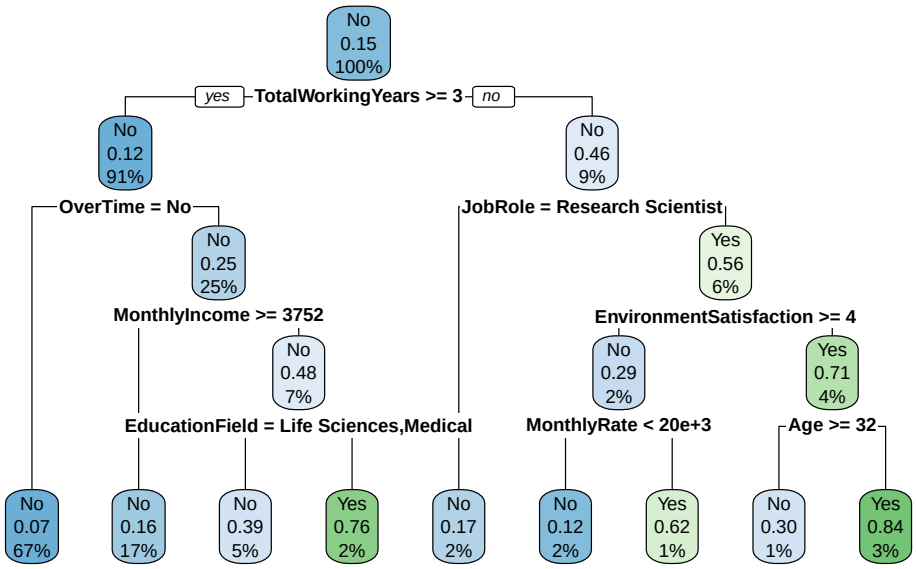
```
klass_tre2 <- rpart(Attrition ~ .,
                    data = training, method = "class",
                    cp = 0.0000001, maxdepth = 20, minbucket = 3)

rpart.plot(klass_tre2)
```



4.4.2 Bruk av `maxdepth = ...`

Parameteren `maxdepth` setter en grense for hvor mange splitter det kan gjøres i hver forgrening:

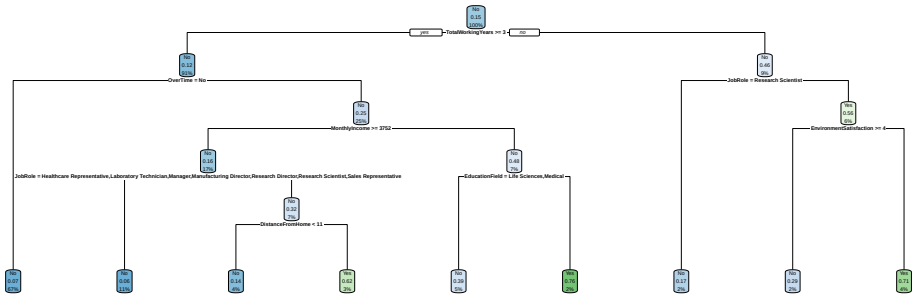
```
klass_tre3 <- rpart(Attrition ~ .,
                    data = training, method = "class",
                    maxdepth = 4)

rpart.plot(klass_tre3)
```



4.4.3 Bruk av `minbucket = ...`

Parameterne `minbucket = ...` styrer hvor mange observasjoner det minst må være i den siste noden. Forvalget er en 1/3 av `minsplit`, altså 7 hvis man ikke har endret på `minsplit`.

```
klass_tre4 <- rpart(Attrition ~ .,
                    data = training, method = "class",
                    minbucket = 15)

rpart.plot(klass_tre4)
```

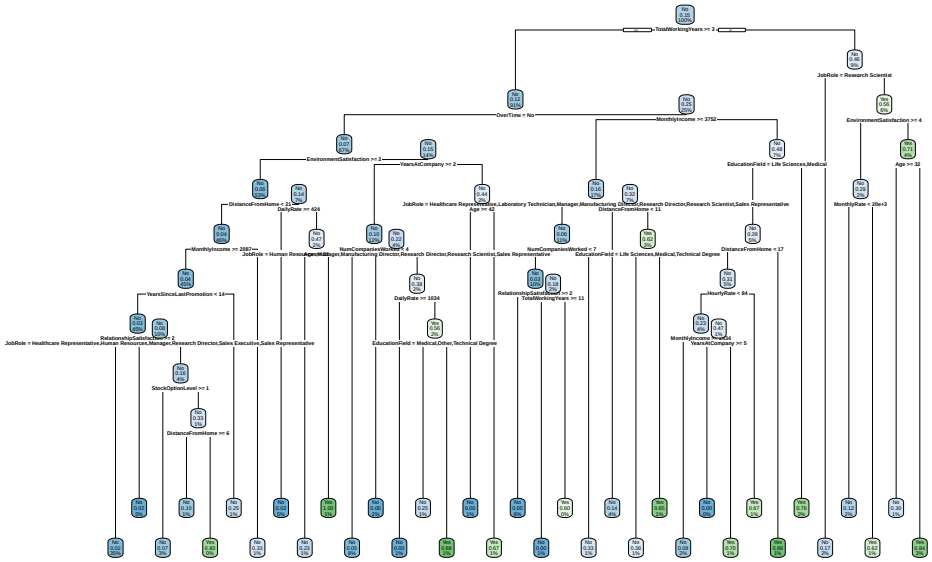


4.4.4 Sette delene sammen

Du kan kombinere disse parametrene. Alle har forvalgte verdier, så du bruker dem uansett – forskjellen er bare om du har tatt et eksplisitt valg eller overlater det hele til softwaren.

Her er et eksempel der vi bruker et ganske komplekst tre med justerte parametre:

```
klass_tre5 <- rpart(Attrition ~ .,
                    data = training, method = "class",
                    cp = 0, maxdepth = 25, minbucket = 5)

rpart.plot(klass_tre5)
```




```
rpart.rules(klass_tre5)
```

```
Attrition
     0.00 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction <  2
     0.00 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction <  2
     0.00 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction <  2
     0.00 when TotalWorkingYears >=      3 & OverTime is Yes
     0.00 when TotalWorkingYears >=     11 & OverTime is Yes
     0.00 when TotalWorkingYears >=      3 & OverTime is Yes
     0.02 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.02 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.02 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.05 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction <  2
     0.07 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.08 when TotalWorkingYears >=      3 & OverTime is Yes
     0.10 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.12 when TotalWorkingYears <  3                        & EnvironmentSatisfaction >= 4
     0.14 when TotalWorkingYears >=      3 & OverTime is Yes
     0.17 when TotalWorkingYears <  3
     0.23 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.25 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction >= 2
     0.25 when TotalWorkingYears >=      3 & OverTime is  No & EnvironmentSatisfaction <  2
```

```
      0.30 when TotalWorkingYears <  3                        & EnvironmentSatisfaction <  4
      0.33 when TotalWorkingYears >=       3 & OverTime is  No & EnvironmentSatisfaction >= 2
      0.33 when TotalWorkingYears >=       3 & OverTime is Yes
      0.38 when TotalWorkingYears >=       3 & OverTime is Yes
      0.60 when TotalWorkingYears is 3 to 11 & OverTime is Yes
      0.62 when TotalWorkingYears <  3                        & EnvironmentSatisfaction >= 4
      0.67 when TotalWorkingYears >=       3 & OverTime is  No & EnvironmentSatisfaction <  2
      0.67 when TotalWorkingYears >=       3 & OverTime is Yes
      0.70 when TotalWorkingYears >=       3 & OverTime is Yes
      0.76 when TotalWorkingYears >=       3 & OverTime is Yes
      0.80 when TotalWorkingYears >=       3 & OverTime is  No & EnvironmentSatisfaction >= 2
      0.84 when TotalWorkingYears <  3                        & EnvironmentSatisfaction <  4
      0.85 when TotalWorkingYears >=       3 & OverTime is Yes
      0.88 when TotalWorkingYears >=       3 & OverTime is  No & EnvironmentSatisfaction <  2
      0.88 when TotalWorkingYears >=       3 & OverTime is Yes
      1.00 when TotalWorkingYears >=       3 & OverTime is  No & EnvironmentSatisfaction >= 2
```

Vi kan sjekke resultatet med confusion matrix på testing-data:

```r
testing_pred <- testing %>%
  mutate(Attrition_pred = predict(klass_tre5, newdata = testing, type = "class"))

confusionMatrix(reference = testing_pred$Attrition,
                testing_pred$Attrition_pred,
                positive = "Yes")
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  328  64
       Yes  32  17

               Accuracy : 0.7823
                 95% CI : (0.7408, 0.82)
    No Information Rate : 0.8163
    P-Value [Acc > NIR] : 0.969618

                  Kappa : 0.1429

 Mcnemar's Test P-Value : 0.001557
```

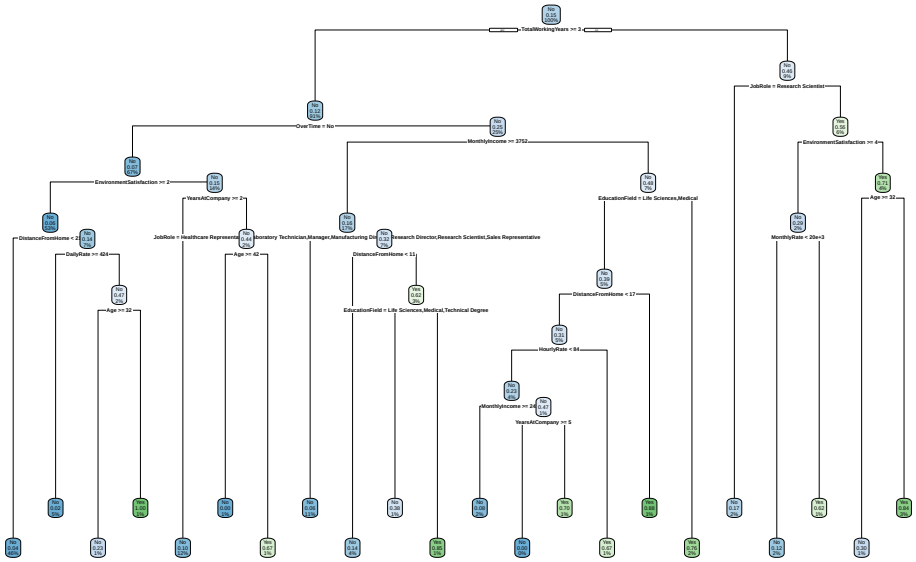
```
            Sensitivity : 0.20988
            Specificity : 0.91111
         Pos Pred Value : 0.34694
         Neg Pred Value : 0.83673
             Prevalence : 0.18367
         Detection Rate : 0.03855
   Detection Prevalence : 0.11111
      Balanced Accuracy : 0.56049

       'Positive' Class : Yes
```

Legg merke til at et veldig komplekst tre ikke nødvendigvis gir bedre resultater på testing-data. Treet kan ha blitt for tilpasset training-dataene (overfitting). En måte å motvirke dette på er å *beskjære* treet i etterkant.

4.4.5 Pruning

En relatert teknikk er å beskjære treet basert på `cp`. Altså, når du har bygget et tre som du tenker er for komplekst, så kan du beskjære grenene slik at de minst viktige grenene kuttes. Funksjonen `prune()` gjør jobben:

```r
pruned_tre <- prune(klass_tre5, cp = .01)
rpart.plot(pruned_tre)
```



4.5 Asymetriske kostnader med loss matrix (Ekstramateriale)

Loss matrix gjør det mulig å vekte ulike typer feil forskjellig. Utgangspunktet er følgende matrise:

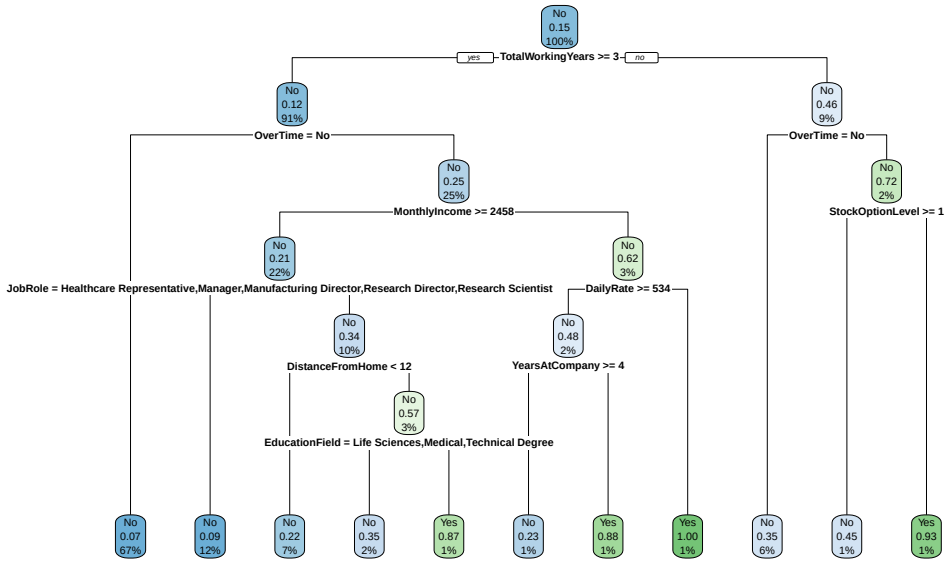
$$loss = \begin{bmatrix} TN & FN \\ FP & TP \end{bmatrix}$$

Vi setter alltid vektingen av sanne positive og sanne negative til 0. Feilene vektes. Her er et eksempel der falske negative (at vi ikke fanger opp noen som faktisk slutter) veier 4 ganger tyngre enn falske positive:

```
lossm <- matrix(c(0, 1, 4, 0), ncol=2)
lossm
```

```
     [,1] [,2]
[1,]    0    4
[2,]    1    0
```

```r
rpart_loss <- rpart(Attrition ~ . ,
                    data = training,
                    parms = list(loss = lossm),
                    method = "class")

rpart.plot(rpart_loss)
```



Sjekk mot testingdata:

```r
testing_pred <- testing %>%
  mutate(Attrition_pred = predict(rpart_loss, newdata = testing, type = "class"))

tab <- testing_pred %>%
  select(Attrition_pred, Attrition) %>%
  table()
confusionMatrix(tab, positive = "Yes")
```

```
Confusion Matrix and Statistics

              Attrition
Attrition_pred  No Yes
           No  356  68
```

```
          Yes   4  13

               Accuracy : 0.8367
                 95% CI : (0.7989, 0.87)
    No Information Rate : 0.8163
    P-Value [Acc > NIR] : 0.1476

                  Kappa : 0.2153

 Mcnemar's Test P-Value : 1.131e-13

            Sensitivity : 0.16049
            Specificity : 0.98889
         Pos Pred Value : 0.76471
         Neg Pred Value : 0.83962
             Prevalence : 0.18367
         Detection Rate : 0.02948
   Detection Prevalence : 0.03855
      Balanced Accuracy : 0.57469

       'Positive' Class : Yes
```

Sammenlign sensitivity og specificity med den opprinnelige modellen uten loss matrix. Ved å vekte falske negative tyngre tvinger vi modellen til å fange opp flere av de som faktisk slutter – men på bekostning av flere falske positive. Denne avveiningen mellom ulike typer feil er sentral i resten av kurset.

5 Bagging

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(rpart)       # for fitting decision trees
library(rpart.plot)  # for å plotte trær
library(ipred)       # for fitting bagged decision trees
library(caret)       # for general model fitting og confusionMatrix()
library(forcats)     # for rekoding av factor-variable
```

6 Introduksjon til bagging

Merk at (R. Berk 2016) gjør et poeng av at med *bagging* gjør vi et større steg vekk fra mer vanlige statistiske prosedyrer. Regresjon og klassifikasjonstrær (og det meste annet vi er mer vant med) gir *ett* sett av resultater. Med bagging får vi en hel haug av resultater. Prinsippet er rett og slett at man trekker et tilfeldig utvalg av dataene, bygger et klassifikasjonstre og gjør klassifikasjonen. Så gjentar man dette med et nytt tilfeldig utvalg fra det opprinnelige datasettet.[1] Så gjentar man dette mange ganger.[2]

Hver observasjon i det opprinnelige datasettet har da blitt klassifisert mange ganger, men ikke nødvendigvis likt i hvert tre. (Enkle klassifikasjonstrær er nemlig litt ustabile greier). Hvis man har gjort prosedyren 100 ganger, så vil hver observasjon bli klassifisert 100 ganger - en gang i hvert tre. For prediksjon på trainingdata brukes rett og slett en opptelling over alle trærne for hver observasjon. Når (R. Berk 2016) snakker om "votes", så er dette som menes: hvert tre har en stemme og så stemmes det over hva som blir utfallet for den enkelte.

Dette betyr at antall trær kan ha noe å si. Forvalget for funksjonen `bagging()` er 25. Det er en god start, men flere trær gir jo mer stabile resultater.

Husk at klassifikasjonstrær er litt ustabile greier, og små tilfeldige variasjoner kan påvirke en enkelt split. Ved å bruke bagging jevnes de tilfeldige feilene ut slik at man totalt sett skal få et mer stabilt resultat. Altså: at faren for overfitting reduseres. Hvor mange trær som trengs er litt verre å si noe om.

6.1 Empirisk eksempel: Syntetiske data

Vi bruker de syntetiske fødselskohort-dataene. Disse er generert for å etterligne en kohort født i 1996, der utfallsvariabelen `felonies` angir om personen har begått alvorlige lovbrudd (1) eller ikke (0). Vi fjerner variablene `drugs` og `violence` fordi disse er delkomponenter av utfallet. Vi demonstrerer også med Attrition-dataene.

[1] Dette er altså det vi ofte kaller tilfeldig utvalg *med* tilbakelegging.

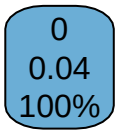
[2] Bagging ligner altså veldig på det vi i vanlig statistikk kaller for bootstrapping. Men i bootstrapping er ofte formålet å korrigere standardfeilene eller noe slikt og ikke så mye punktestimatet. Skjønt, det går an å bruke bagging på regresjonsmodeller også, og da tror jeg forskjellen mot bootstrapping er mest semantisk.

```r
load("data/synthetic_birthCohort96.RData")

synth <- synthetic_birthCohort %>%
  select(-drugs, -violence) %>%
  mutate(felonies = factor(felonies))

set.seed(426)
synth_split <- initial_split(synth)
training <- training(synth_split)
testing  <- testing(synth_split)
```

6.1.1 Klassifikasjonstre som utgangspunkt

Først lager vi et klassifikasjonstre for sammenligning:

```r
klass_tre <- rpart(felonies ~ ., data = training, method = "class")
rpart.plot(klass_tre)
```



6.1.2 Bagging

Funksjonen `bagging()` har tilsvarende oppbygning som tidligere modeller. Her angir vi 100 trær med `nbagg = 100` og bruker OOB med `coob = TRUE`:

```
set.seed(42)
bag <- bagging(felonies ~ ., data = training,
               nbagg = 100, coob = TRUE)

bag
```

```
Bagging classification trees with 100 bootstrap replications

Call: bagging.data.frame(formula = felonies ~ ., data = training, nbagg = 100,
    coob = TRUE)

Out-of-bag estimate of misclassification error:  0.0411
```

Vi kan legge til justeringer for de enkelte trærne med `control = rpart.control()`:

```
set.seed(42)
bag2 <- bagging(felonies ~ ., data = training,
                nbagg = 100, coob = TRUE,
                control = rpart.control(cp = 0.0000001, maxdepth = 20, minbucket = 3))

bag2
```

```
Bagging classification trees with 100 bootstrap replications

Call: bagging.data.frame(formula = felonies ~ ., data = training, nbagg = 100,
    coob = TRUE, control = rpart.control(cp = 1e-07, maxdepth = 20,
        minbucket = 3))

Out-of-bag estimate of misclassification error:  0.0403
```

Sammenlign med testing-data. Her lager vi prediksjoner fra både det enkle klassifikasjonstreet og bagging-modellen, slik at vi kan sammenligne dem direkte:

```r
testing_pred <- testing %>%
  mutate(pred_tre = predict(klass_tre, newdata = testing, type = "class"),
         pred_bag = predict(bag, newdata = testing))

confusionMatrix(reference = testing_pred$felonies, testing_pred$pred_tre, positive = "1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 7193  307
         1    0    0

               Accuracy : 0.9591
                 95% CI : (0.9543, 0.9634)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 0.5152

                  Kappa : 0

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.00000
            Specificity : 1.00000
         Pos Pred Value :     NaN
         Neg Pred Value : 0.95907
             Prevalence : 0.04093
         Detection Rate : 0.00000
   Detection Prevalence : 0.00000
      Balanced Accuracy : 0.50000

       'Positive' Class : 1
```

```r
confusionMatrix(reference = testing_pred$felonies, testing_pred$pred_bag, positive = "1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 7183  302
```

```
         1   10    5

               Accuracy : 0.9584
                 95% CI : (0.9536, 0.9628)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 0.6287

                  Kappa : 0.0273

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.0162866
            Specificity : 0.9986098
         Pos Pred Value : 0.3333333
         Neg Pred Value : 0.9596526
             Prevalence : 0.0409333
         Detection Rate : 0.0006667
   Detection Prevalence : 0.0020000
      Balanced Accuracy : 0.5074482

       'Positive' Class : 1
```

Sammenlign accuracy, sensitivity og specificity for de to modellene. Bagging gir typisk bedre og mer stabile resultater enn et enkelt tre fordi de tilfeldige feilene jevner seg ut over mange trær.

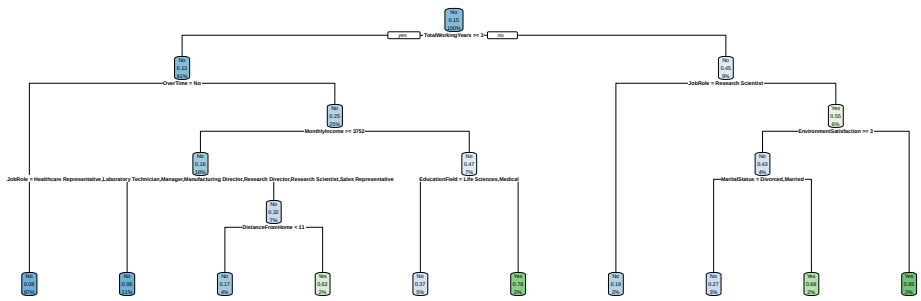
6.2 Eksempel med Attrition-data

Vi bruker også Attrition-dataene for å vise at bagging fungerer på tvers av datasett:

```r
attrition <- readRDS("data/Attrition.rds") %>%
  select(-EmployeeNumber)

set.seed(426)
attrition_split <- initial_split(attrition)
attr_train <- training(attrition_split)
attr_test  <- testing(attrition_split)
```

Klassifikasjonstre, bagging og confusion matrix:

```r
attr_tre <- rpart(Attrition ~ ., data = attr_train, cp = 0.00001, maxdepth = 5,
                  minbucket = 15, method = "class")
rpart.plot(attr_tre)
```




```r
set.seed(42)
attr_bag <- bagging(Attrition ~ ., data = attr_train,
                    nbagg = 500, coob = TRUE)
attr_bag
```

```
Bagging classification trees with 500 bootstrap replications

Call: bagging.data.frame(formula = Attrition ~ ., data = attr_train,
    nbagg = 500, coob = TRUE)

Out-of-bag estimate of misclassification error:  0.1325
```

```r
attr_test_pred <- attr_test %>%
  mutate(pred_tre = predict(attr_tre, newdata = attr_test, type = "class"),
         pred_bag = predict(attr_bag, newdata = attr_test, type = "class"))

confusionMatrix(reference = attr_test_pred$Attrition, attr_test_pred$pred_tre, positive = "Ye
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  288  57
       Yes  12  11

               Accuracy : 0.8125
                 95% CI : (0.7688, 0.8511)
    No Information Rate : 0.8152
    P-Value [Acc > NIR] : 0.5851

                  Kappa : 0.1636

 Mcnemar's Test P-Value : 1.177e-07

            Sensitivity : 0.16176
            Specificity : 0.96000
         Pos Pred Value : 0.47826
         Neg Pred Value : 0.83478
             Prevalence : 0.18478
         Detection Rate : 0.02989
   Detection Prevalence : 0.06250
      Balanced Accuracy : 0.56088

       'Positive' Class : Yes
```

```r
confusionMatrix(reference = attr_test_pred$Attrition, attr_test_pred$pred_bag, positive = "Ye
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  296  55
       Yes   4  13

               Accuracy : 0.8397
                 95% CI : (0.7981, 0.8757)
    No Information Rate : 0.8152
    P-Value [Acc > NIR] : 0.1258
```

```
                 Kappa : 0.2505

 Mcnemar's Test P-Value : 7.543e-11

            Sensitivity : 0.19118
            Specificity : 0.98667
         Pos Pred Value : 0.76471
         Neg Pred Value : 0.84330
             Prevalence : 0.18478
         Detection Rate : 0.03533
   Detection Prevalence : 0.04620
      Balanced Accuracy : 0.58892

       'Positive' Class : Yes
```

Igjen ser vi at bagging typisk gir et resultat som er noe mer stabilt enn et enkelt tre. Merk at forbedringen ikke alltid er like dramatisk – det kommer an på dataene og det underliggende mønsteret.

6.3 Out-of-bag data (OOB)

Bagging kan brukes på training data og testing data som vi har gjort før. Men en ulempe med dette er jo at man får et litt mindre datasett å tilpasse modellene med. Merk at bagging i utgangspunktet baserer seg på å trekke et tilfeldig *utvalg* fra de opprinnelige dataene. Så for hvert tre er det en del observasjoner som ikke blir brukt til å bygge treet. Altså har vi et testing-datasett umiddelbart tilgjengelig!

Altså: hvis man bruker 70% av dataene til å bygge hvert enkelt tre, så har man også 30% testingdata tilgjengelig for å gjøre klassifiseringen på det enkelte treet. Dette testing-datasettet kalles *out-of-bag* (OOB) data, men er altså i prinsippet det samme som et testingdatasett.

Forvalget i `bagging()` er imidlertid å *ikke* bruke OOB. For å gjøre dette legges det til argumentet `coob = TRUE` og man får utregnet klassifikasjonsfeilen i output.

6.4 Mer tuning

Man kan si at bagging består av to deler: 1) en grunnleggende klassifikasjonsteknikk (noen ganger omtalt som "base learner"), og 2) en bootstrapping med gjentatte estimeringer på utvalg av data.

Forvalget i `bagging()` er å bruke klassifikasjonstrær med `rpart()`. Det betyr at tuning som kan gjøres med bruk av `rpart()` også kan gjøres med `bagging()`. Dette gjøres ved å legge til argumentet `control = rpart.control()`:

```
set.seed(42)
bag_tuned <- bagging(felonies ~ ., data = training,
                     nbagg = 500, coob = TRUE,
                     control = rpart.control(maxdepth = 6, cp = 0.0001))
bag_tuned
```

```
Bagging classification trees with 500 bootstrap replications

Call: bagging.data.frame(formula = felonies ~ ., data = training, nbagg = 500,
    coob = TRUE, control = rpart.control(maxdepth = 6, cp = 1e-04))

Out-of-bag estimate of misclassification error:  0.0403
```

Det viktige poenget akkurat nå er egentlig ikke all tuning som er mulig å gjøre i bagging, men at det er bygget på en annen underliggende algoritme. Vi bruker i praksis ikke bagging noe særlig alene, men derimot er random forest en variant av bagging med litt ekstra saker. Slik sett er bagging å regne som en byggesten til random forest sammen med klassifikasjonstrær. Så derfor skal vi ikke bruke så mye tid på bagging, men gå raskt videre til random forest.

7 Random forest

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(randomForest)
library(caret)
```

Random forest bruker klassifikasjonstrær og bagging som byggestener. I prinsippet er det "bagged trees", men i stedet for å bagge samme type trær, så gjøres det en endring i hvert tre: I hver split i hvert enkelt tre trekkes det bare et fåtall variable for å bestemme hver split. Det brukes med andre ord *mindre* informasjon i hvert enkelt tre! Man skulle intuitivt tro at dette vil gi dårligere prediksjon, og det stemmer forsåvidt i hvert enkelt tre. Men på den annen side lages det mange trær og det er det aggregerte resultatet som blir prediksjonen. På samme måte som i bagging, så avgjøres klassifikasjonen ved majoritetsstemme over alle trærne.

Husk nå at random forest er en variant av bagging, så det som i forrige kapittel ble sagt om OOB gjelder også her. Med andre ord: det training og testing datasett benyttes internt i algoritmen. Slik sett trenger man ikke splitte datasettet i forkant. Resultatene og feilratene er beregnet med OOB-dataene. I utgangspunktet trenger man altså ikke å splitte datasettet.

Men: Det går an å bruke gjøre en split likevel. Da er det mer å regne som et *valideringsdatasett*. Hvis man tuner modellen og sjekker OOB-resultatene, så står man i fare for å overfitte mot OOB.[1] Med andre ord: Det kan være en god ide å splitte dataene først likevel.

7.1 Eksempel med COMPAS-data

COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) er et risikoverktøy brukt av politi og rettsvesen i flere amerikanske stater. Datasettet inneholder informasjon om personer som har vært gjennom rettssystemet, der utfallsvariabelen `Two_yr_Recidivism` angir om personen begikk ny kriminalitet innen to år. Vi bruker dette datasettet gjennom flere kapitler i heftet.

[1] Jeg har forstått det slik at de lærde strides noe om dette.

```r
compas <- readRDS("data/compas.rds")
glimpse(compas)
```

```
Rows: 6,172
Columns: 7
$ Two_yr_Recidivism    <fct> 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1~
$ Number_of_Priors     <int> 0, 0, 4, 0, 14, 3, 0, 0, 3, 0, 0, 1, 7, 0, 3, 6, ~
$ Age_Above_FourtyFive <fct> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0~
$ Age_Below_TwentyFive <fct> 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
$ Misdemeanor          <fct> 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0~
$ Ethnicity            <fct> Other, African_American, African_American, Other,~
$ Sex                  <fct> Male, Male, Male, Male, Male, Male, Female, Male,~
```

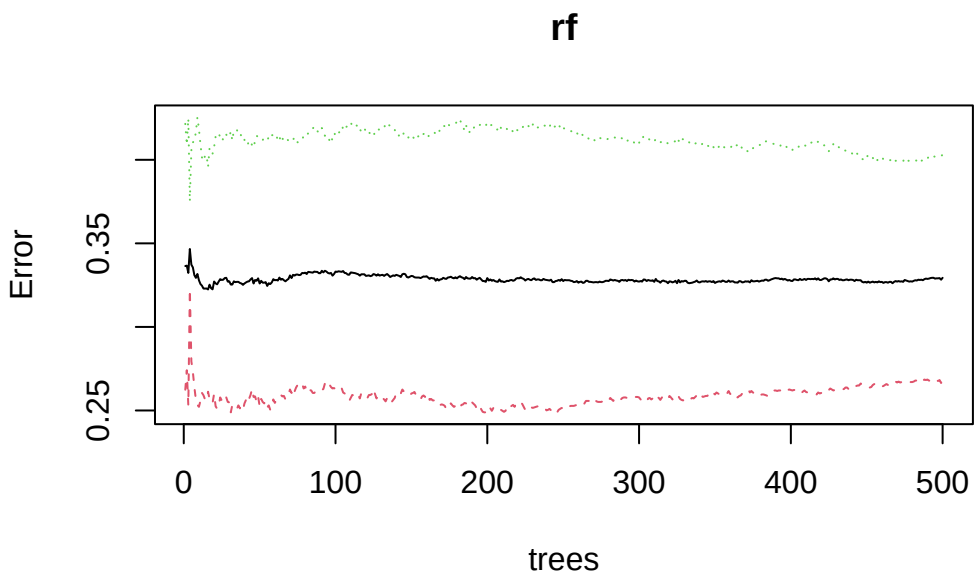
Estimerer random forest med alle variable.

```r
set.seed(4356)
rf <- randomForest(Two_yr_Recidivism ~ . ,
                   data = compas)
rf
```

```
Call:
 randomForest(formula = Two_yr_Recidivism ~ ., data = compas)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 32.94%
Confusion matrix:
     0    1 class.error
0 2462  901   0.2679156
1 1132 1677   0.4029904
```

Følgende plot gir en oversikt over feilrater for random forest etter hvor mange trær. Den svarte linjen er den totale feilraten, den grønne er falske positive, og den røde er falske negative. I utgangspunktet bruker random forest 500 trær. Dette plottet viser når resultatene stabiliserer seg.

```r
plot(rf)
```



Prediksjon fungerer på tilsvarende måte som vi har gjort tidligere.

```
compas_p <- compas %>%
  mutate(pred_rf = predict(rf))
```

Confusion matrix:

```
tab <- table(compas_p$pred_rf, compas_p$Two_yr_Recidivism)
tab
```

```
       0    1
  0 2462 1132
  1  901 1677
```

```
confusionMatrix(tab, positive="1")
```

```
Confusion Matrix and Statistics


        0    1
```

```
  0 2462 1132
  1  901 1677

               Accuracy : 0.6706
                 95% CI : (0.6587, 0.6823)
    No Information Rate : 0.5449
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.3313

 Mcnemar's Test P-Value : 3.378e-07

            Sensitivity : 0.5970
            Specificity : 0.7321
         Pos Pred Value : 0.6505
         Neg Pred Value : 0.6850
             Prevalence : 0.4551
         Detection Rate : 0.2717
   Detection Prevalence : 0.4177
      Balanced Accuracy : 0.6645

       'Positive' Class : 1
```

Vi kan justere resultatet med å endre antall variable som tas med i hver split (i hvert tre). Parameteren `mtry` styrer dette:

```r
set.seed(4356)
rf2 <- randomForest(Two_yr_Recidivism ~ . ,
                    mtry=4,
                    data = compas)
rf2
```

```
Call:
 randomForest(formula = Two_yr_Recidivism ~ ., data = compas,      mtry = 4)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 4

        OOB estimate of  error rate: 34.36%
Confusion matrix:
```

```
     0    1 class.error
0 2608  755   0.2245019
1 1366 1443   0.4862941
```

7.1.1 Variable importance

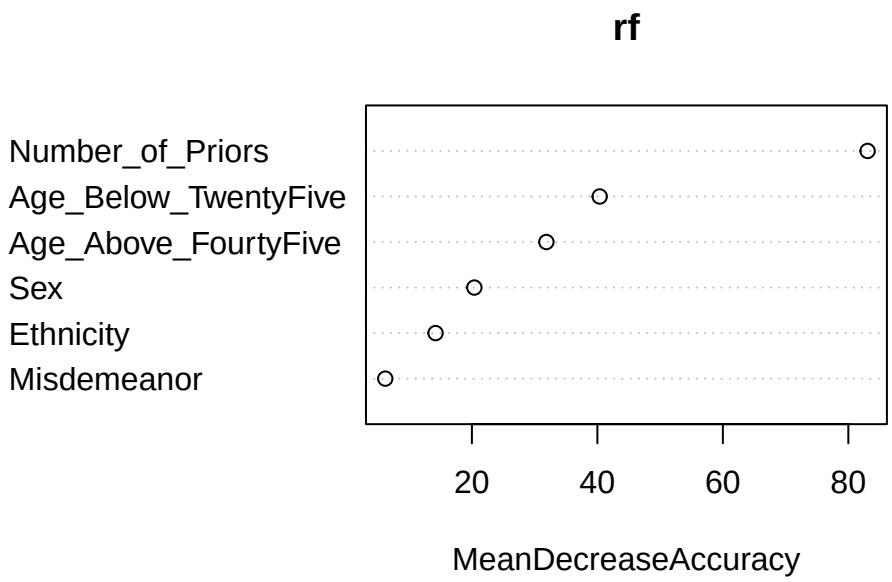
For å få ut variable importance må dette settes i estimeringen med `importance = TRUE`. Det tar nå litt lengre tid å estimere, så med store datasett bør du vente med dette til du ellers er fornøyd med modellen.

```
set.seed(4356)
rf <- randomForest(Two_yr_Recidivism ~ . ,
                   importance = TRUE,
                    data = compas)
rf
```

```
Call:
 randomForest(formula = Two_yr_Recidivism ~ ., data = compas,      importance = TRUE)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 32.42%
Confusion matrix:
     0    1 class.error
0 2538  825   0.2453167
1 1176 1633   0.4186543
```

Vi kan da plotte variable importance plot. Set `type = 1` for at det skal vise gjennomsnittlig reduksjon i accuracy. Plottet viser hvor mye dårligere modellens prediksjon blir hvis en variabel permuteres (stokkes om). Variable som er høyt oppe i plottet er mest viktige for prediksjonen.

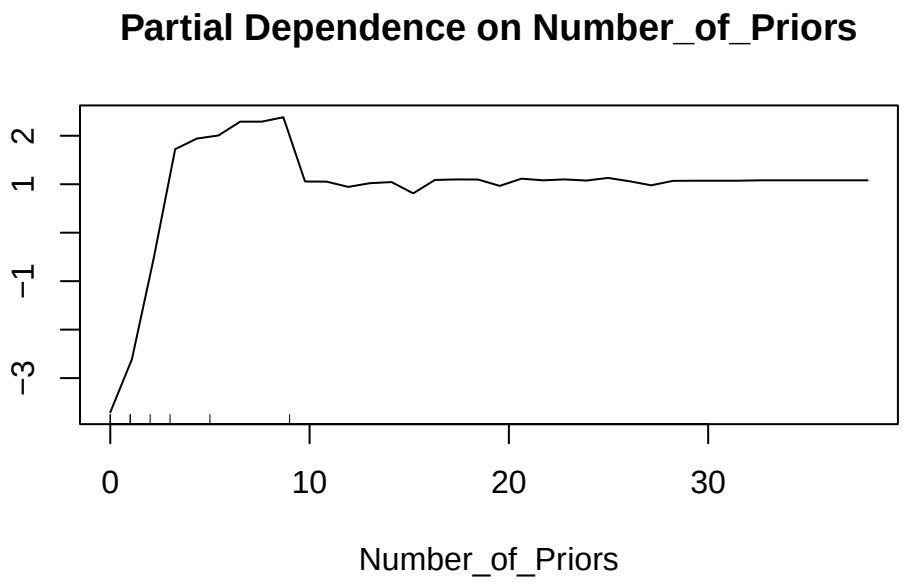
```
varImpPlot(rf, type = 1)
```



7.1.2 Partial dependence

Variable importance forteller oss *hvilke* variable som er viktige, men ikke *hvordan* de påvirker prediksjonen. Partial dependence plot viser sammenhengen mellom en enkelt variabel og den predikerte sannsynligheten, justert for alle andre variable. Her må du velge hvilken variabel du ønsker å se på, og `which.class` angir hvilken klasse du vil se sannsynligheten for.

```
partialPlot(rf, pred.data = compas,
            x.var = Number_of_Priors,
            which.class = "1")
```



Plottet viser at sannsynligheten for tilbakefall øker med antall tidligere lovbrudd, noe som er intuitivt rimelig.

7.2 Tuning av random forest med sampsize

Det er mest vanlig å justere sampling-skjemaet i baggingen: altså hvor mange observasjoner som trekkes til å bygge hvert tre. Ved å bruke argumentet `sampsize = ...` kan vi angi antallet som trekkes fra hver kategori i utfallsvariabelen.

Hensikten med å gjøre dette er at hvis det er et mindretall som har det ene utfallet, så blir hvert tre bygget med mer informasjon om majoritetsgruppen. Hvis vi vekter opp minoritetsgruppen, så får disse større inflytelse på hvert tre.

7.2.1 Eksempel med COMPAS-data

Her setter vi `sampsize = c(1900, 1900)`, altså like mange fra hver kategori av utfallsvariabelen. Det første tallet er antallet som trekkes fra kategori 0 (ingen tilbakefall) og det andre fra kategori 1 (tilbakefall):

```r
set.seed(4356)
rf3 <- randomForest(Two_yr_Recidivism ~ . ,
                    sampsize = c(1900, 1900),
                    data = compas)
rf3
```

```
Call:
 randomForest(formula = Two_yr_Recidivism ~ ., data = compas,      sampsize = c(1900, 1900))
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 32.73%
Confusion matrix:
     0    1 class.error
0 2321 1042   0.3098424
1  978 1831   0.3481666
```

Her er et eksempel der gruppene veies ulikt – vi trekker færre fra kategori 0 enn fra kategori 1, noe som gir mer vekt til tilbakefalls-gruppen:

```r
set.seed(4356)
rf4 <- randomForest(Two_yr_Recidivism ~ . ,
                    sampsize = c(1000, 1900),
                   data = compas)
rf4
```

```
Call:
 randomForest(formula = Two_yr_Recidivism ~ ., data = compas,      sampsize = c(1000, 1900))
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 41.15%
Confusion matrix:
     0    1 class.error
0 1170 2193   0.6520963
1  347 2462   0.1235315
```

Det viktige nå er at feilratene for falske positive og falske negative blir vesentlig forskjellig! Det betyr at ved hvordan vi estimerer modellen kan vi legge sterke føringer på resultatet. Vi bør derfor ta stilling til *på forhånd* hvilke feilrater vi er villig til å akseptere.

7.2.2 Eksempel med syntetiske data

Vi kan gjøre tilsvarende med de syntetiske fødselskohort-dataene. Her håndterer vi først eventuelle manglende verdier med `replace_na()`. Merk at `randomForest()` ikke håndterer manglende verdier (NA), så disse må erstattes før estimeringen. En enkel løsning er å sette NA til 0, men i praksis bør man vurdere om dette er faglig rimelig.

```
load("data/synthetic_birthCohort96.RData")

synth <- synthetic_birthCohort %>%
  select(-drugs, -violence) %>%
  mutate(felonies = factor(felonies)) %>%
  mutate(across(everything(), ~replace_na(.x, 0)))

set.seed(426)
synth_split <- initial_split(synth)
synth_train <- training(synth_split)
synth_test  <- testing(synth_split)

table(synth_train$felonies)
```

```
    0     1
21579   921
```

Vi sammenligner flere ulike konfigurasjoner av `sampsize`. Først kjører vi med forvalget, uten å angi `sampsize`. Da trekker `randomForest()` like mange observasjoner som det er i datasettet, men med tilbakelegging:

```
# Default (ingen sampsize angitt)
rf_s1 <- randomForest(felonies ~ ., data = synth_train)

synth_test1 <- synth_test %>%
  mutate(pred = predict(rf_s1, newdata = ., type = "response"))

confusionMatrix(reference = synth_test1$felonies, synth_test1$pred, positive = "1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 7193  306
         1    0    1

               Accuracy : 0.9592
                 95% CI : (0.9545, 0.9636)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 0.4919

                  Kappa : 0.0062

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.0032573
            Specificity : 1.0000000
         Pos Pred Value : 1.0000000
         Neg Pred Value : 0.9591946
             Prevalence : 0.0409333
         Detection Rate : 0.0001333
   Detection Prevalence : 0.0001333
      Balanced Accuracy : 0.5016287

       'Positive' Class : 1
```

Deretter prøver vi en balansert sampsize der vi trekker like mange fra hver kategori:

```r
# Balansert sampsize
rf_s2 <- randomForest(felonies ~ .,
                      sampsize = c(600, 600),
                      data = synth_train)

synth_test2 <- synth_test %>%
  mutate(pred = predict(rf_s2, newdata = ., type = "response"))

confusionMatrix(reference = synth_test2$felonies, synth_test2$pred, positive = "1")
```

```
Confusion Matrix and Statistics
```

```
          Reference
Prediction    0    1
         0 5516  145
         1 1677  162

               Accuracy : 0.7571
                 95% CI : (0.7472, 0.7667)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0869

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.52769
            Specificity : 0.76686
         Pos Pred Value : 0.08809
         Neg Pred Value : 0.97439
             Prevalence : 0.04093
         Detection Rate : 0.02160
   Detection Prevalence : 0.24520
      Balanced Accuracy : 0.64727

       'Positive' Class : 1
```

Vi kan også prøve en ubalansert sampsize der vi trekker litt flere fra minoritetsgruppen:

```r
# Ubalansert sampsize
rf_s3 <- randomForest(felonies ~ .,
                      sampsize = c(500, 600),
                      data = synth_train)

synth_test3 <- synth_test %>%
  mutate(pred = predict(rf_s3, newdata = ., type = "response"))

confusionMatrix(reference = synth_test3$felonies, synth_test3$pred, positive = "1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
```

```
         0 4922  106
         1 2271  201

               Accuracy : 0.6831
                 95% CI : (0.6724, 0.6936)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0775

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.65472
            Specificity : 0.68428
         Pos Pred Value : 0.08131
         Neg Pred Value : 0.97892
             Prevalence : 0.04093
         Detection Rate : 0.02680
   Detection Prevalence : 0.32960
      Balanced Accuracy : 0.66950

       'Positive' Class : 1
```

Til sist prøver vi en konfigurasjon der vi trekker langt flere fra kategori 0 enn fra kategori 1:

```r
# Større sampsize
rf_s4 <- randomForest(felonies ~ .,
                      sampsize = c(800, 400),
                      data = synth_train)

synth_test4 <- synth_test %>%
  mutate(pred = predict(rf_s4, newdata = ., type = "response"))

confusionMatrix(reference = synth_test4$felonies, synth_test4$pred, positive = "1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 6861  245
         1  332   62
```

```
               Accuracy : 0.9231
                 95% CI : (0.9168, 0.929)
    No Information Rate : 0.9591
    P-Value [Acc > NIR] : 1.0000000

                  Kappa : 0.1372

 Mcnemar's Test P-Value : 0.0003433

            Sensitivity : 0.201954
            Specificity : 0.953844
         Pos Pred Value : 0.157360
         Neg Pred Value : 0.965522
             Prevalence : 0.040933
         Detection Rate : 0.008267
   Detection Prevalence : 0.052533
      Balanced Accuracy : 0.577899

       'Positive' Class : 1
```

Merk at de ulike konfigurasjonene gir tydelig forskjellige feilrater. Det er dette Berk (2016) kaller *asymetriske kostnader* og må vurderes i henhold til konsekvenser av hva prediksjonen skal brukes til.

8 Boosting

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(gbm)         # Funksjoner for boosting
library(caret)       # For funksjonen confusionMatrix()
library(fairness)    # For fairnessmetrics
library(gtsummary)   # For å lage litt penere tabeller
```

Det er flere typer boosting-algoritmer. Men vi skal fokusere på stochastic gradient boosting. Adaptive boosting er presentert i læreboka, men først og fremst som et illustrativt eksempel eller en pedagogisk introduksjon. Andre boosting-algoritmer er varianter av gradient boosting.

Det er også et poeng at softwaren for adaptive boosting er mer begrenset enn for gradient boosting. Vi bruker 'gbm' pakken som gir grunnleggende funksjonalitet.

Det sentrale punktet for boosting er at modellen bygges steg for steg med en gradvis forbedring fra forrige runde slik at modellen blir gradvis bedre og bedre. Dette til kontrast til *bagging* der styrken ligger i avstemning på tvers av mange klassifikasjonstrær. I boosting er det altså samme modell som forbedres i hvert steg. Dette gjøres på en måte der feilklassifiseringen vektes opp i hver iterasjon. I adaptive boosting lages den en vekt litt mekanisk, mens i gradient boosting baseres det på en gradient fra velkjente statistiske fordelinger, f.eks. binomisk fordeling for kategorisk utfall.

Vi bruker compas-dataene igjen. Men nå må vi omkode utfallsvariabelen til en numerisk variabel fordi funksjonen `gbm()` krever numerisk utfallsvariabel. Vi bruker `na.omit()` for å fjerne eventuelle rader med manglende verdier.

```
compas <- readRDS("data/compas.rds") %>%
  mutate(Two_yr_Recidivism = ifelse(Two_yr_Recidivism == "1", 1, 0)) %>%
  na.omit()
glimpse(compas)
```

```
Rows: 6,172
Columns: 7
$ Two_yr_Recidivism    <dbl> 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1~
$ Number_of_Priors     <int> 0, 0, 4, 0, 14, 3, 0, 0, 3, 0, 0, 1, 7, 0, 3, 6, ~
$ Age_Above_FourtyFive <fct> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0~
$ Age_Below_TwentyFive <fct> 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
$ Misdemeanor          <fct> 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0~
$ Ethnicity            <fct> Other, African_American, African_American, Other,~
$ Sex                  <fct> Male, Male, Male, Male, Male, Male, Female, Male,~
```

Vi lager en split som tidligere

```r
set.seed(42)
training_init <- initial_split(compas)

training <- training(training_init)
testing  <- testing(training_init)
```

8.1 Stochastic gradient boosting

Gradient boosting for klassifisering bruker ikke vekter på samme måte som adaboost, men bruker residualene. For et binomisk utfall (to kategorier kodet 0 eller 1) er residualen, r_i, er definert som

$$r_i = y_i - \frac{1}{e^{-f(x_i)}}$$

Litt forenklet kan vi si at dette er det observert utfallet minus det predikerte utfallet fra logistisk regresjon. "Gradienten" er da $f(x)$.

Gradient boosting fungerer med flere andre fordelinger, men hvis vi begrenser oss til klassifikasjon kan vi angi `distribution = "bernoulli"`. Merk at for `gbm` må utfallsvariabelen være numerisk, ikke factor.

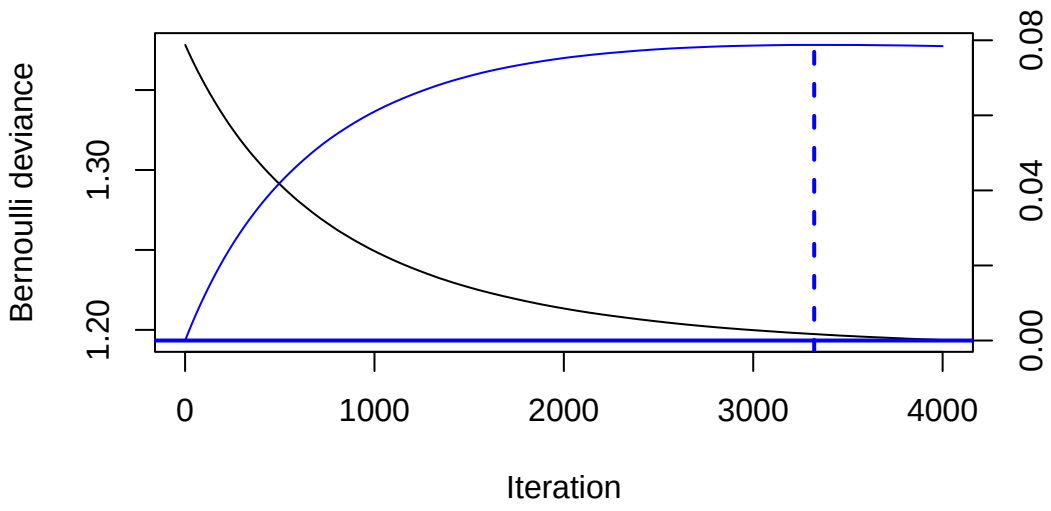
```r
set.seed(542)
gradboost1 <- gbm(formula = Two_yr_Recidivism ~ .,
                  data = training,
                  distribution = "bernoulli",
                  n.trees = 4000,
                  interaction.depth = 3,
                  n.minobsinnode = 1,
```

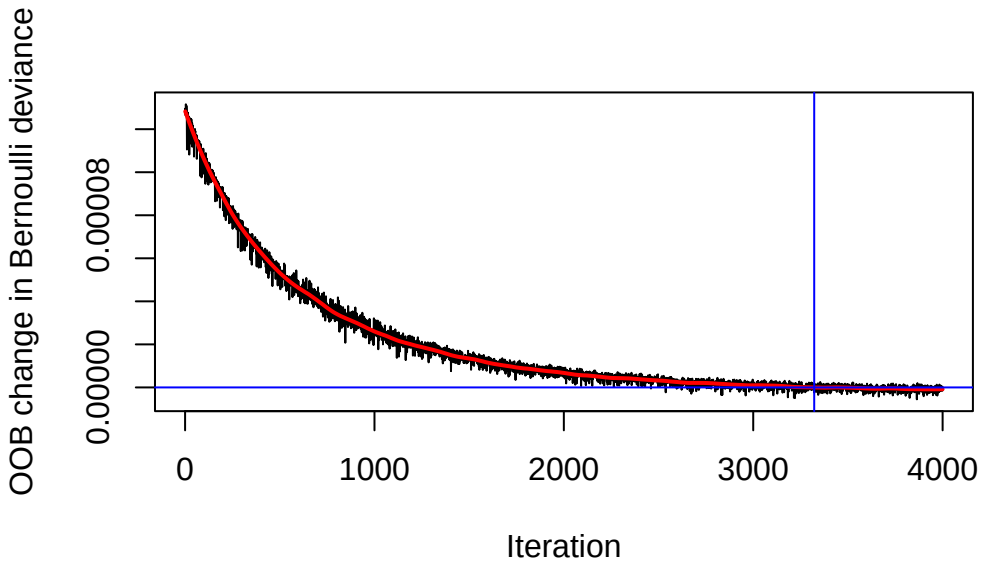
```
                    shrinkage = 0.001,
                    bag.fraction = 0.5)
```

Merk angir argumentet `bag.fraction = 0.5` i modellen (som forøvrig også er forvalget, så man behøver ikke skrive det, egentlig). I hver split brukes altså halve utvalget til å bestemme neste split slik at andre halvdel er out-of-bag data. Hvis dette argumentet settes til 1 brukes hele datasettet i hver split.

Standard plot av gis ved `gbm.perf()` som nedenfor, og viser forbedring for hver iterasjon. Ytterligere forbedring stopper opp ved nærmere 2500 iterasjoner, markert ved den vertikale blå linjen.

```
gbm.perf(gradboost1, oobag.curve = TRUE, method = "OOB",
            plot.it=T, overlay = T)
```






```
[1] 3322
attr(,"smoother")
Call:
loess(formula = object$oobag.improve ~ x, enp.target = min(max(4,
    length(x)/10), 50))

Number of Observations: 4000
Equivalent Number of Parameters: 39.99
Residual Standard Error: 2.048e-06
```

Denne estimeringen kan justeres ved å endre tuningparametrene.

- `n.trees` er antall iterasjoner, dvs. antall klassifikasjonstrær. Forvalget er 100, som er åpenbart altfor lavt, så sett noen tusen.
- `interaction.depth` er antall split i hvert tre. Forvalget er 1, men Berk sier at 1-3 er ok.
- `n.minobsinnode` er minste antall observasjoner i siste node. Forvalg er 1, men Berk anslår at 5-15 fungerer bra. (Hvis modellen har bare en split er det ikke sikkert dette er så viktig)
- `shrinkage` er hvor store skritt langs gradienten som testes i hver iterasjon, og dette skal være lavt. Forvalget er 0.001. Kostnaden er at det tar lengre tid enn ved et høyere tall (opp til 10 kan testes).

- `bag.fraction` er hvor stor andel av data som brukes i hver iterasjon. Dette hjelper for å redusere overfitting. Forvalget er 0.5, som innebærer at andre del av data kan fungere som OOB. Merk at Berk understreker at OOB bare kan brukes til å vurdere tilpassingen slik som i figuren over.

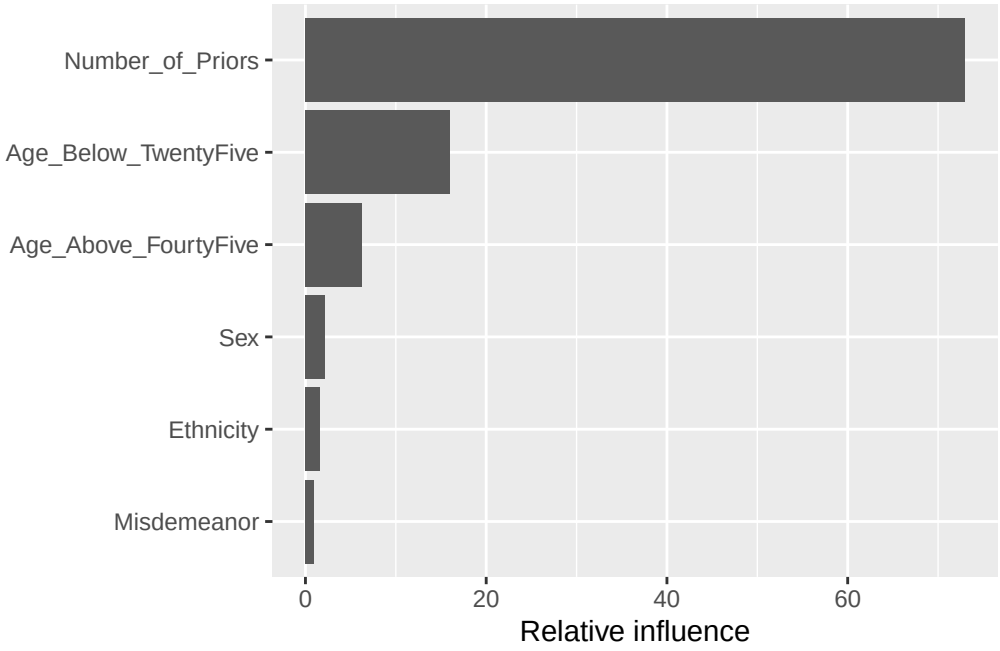
8.2 Tolkbarhet

Som i random forest kan vi undersøke hvilke variable som er mest betydningsfulle i prediksjonen. I utgangspunktet gir `summary()` mot et gbm-objekt et plot, men dette er stygt og kan være vanskelig å lese skikkelig. Derfor inneholder koden nedenfor argumentet `plotit=FALSE`, så lages det et bedre plot med `ggplot()` etterpå.[1]

Resultatene er tolkbare tilsvarende som relative importance som vi så i random forest.

```
sumboost <- summary(gradboost1, method = permutation.test.gbm, normalize = T,
                    plotit = F)

ggplot(sumboost, aes(x = reorder(var, rel.inf), y = rel.inf)) +
  geom_col()+
  ylab("Relative influence")+
  xlab("")+
  coord_flip()
```

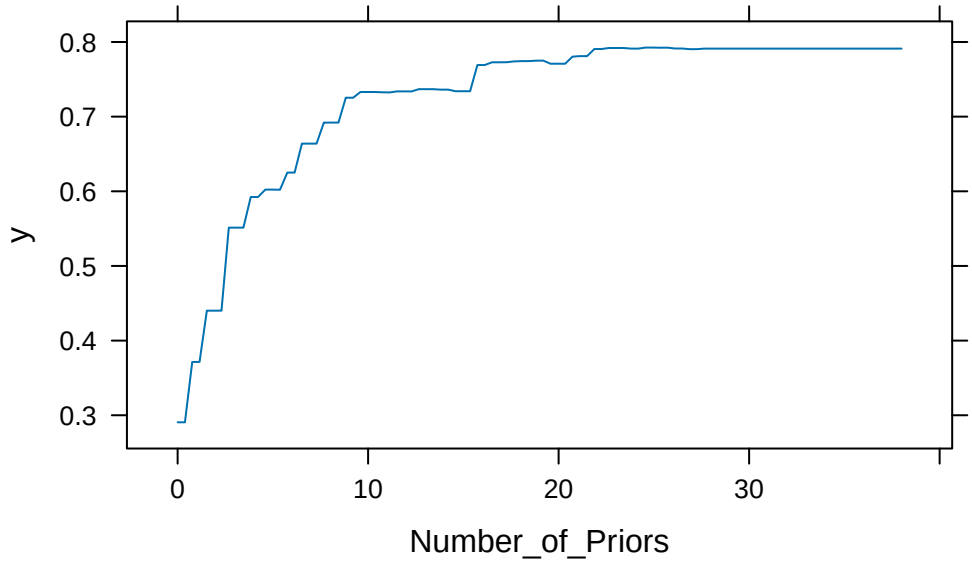
[1]Estetikken er imidlertid underordet her. Sett gjerne `plotit = TRUE` for et mer quisk-and-dirty plot.



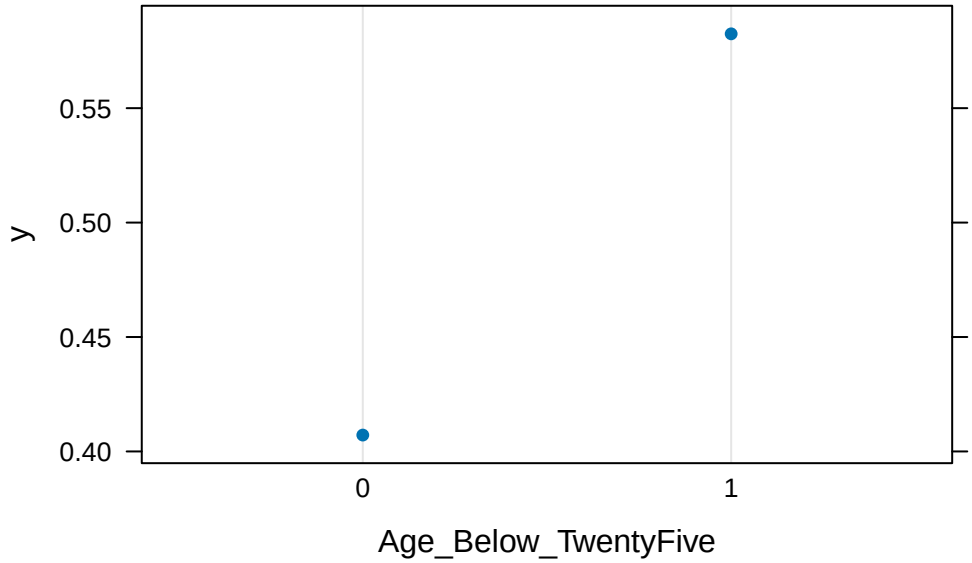
Merk at resultatet fra `summary()` her returnerer en data.frame som kan plottes direkte med `ggplot`. Eneste mystiske som skjer er at `x` er angitt som sortert etter verdier på `y`-variabelen. Så er plottet snudd om med `coord_flip` til slutt.

Man kan også plotte hvordan resultatet endres med ulike verdier på prediktorene. Dette tilsvarer *partial dependence*. Her er det mest hensiktsmessig å bruke den innebygde funksjonen `plot()` som kaller en underliggende plot-funksjon for gbm-objekter.

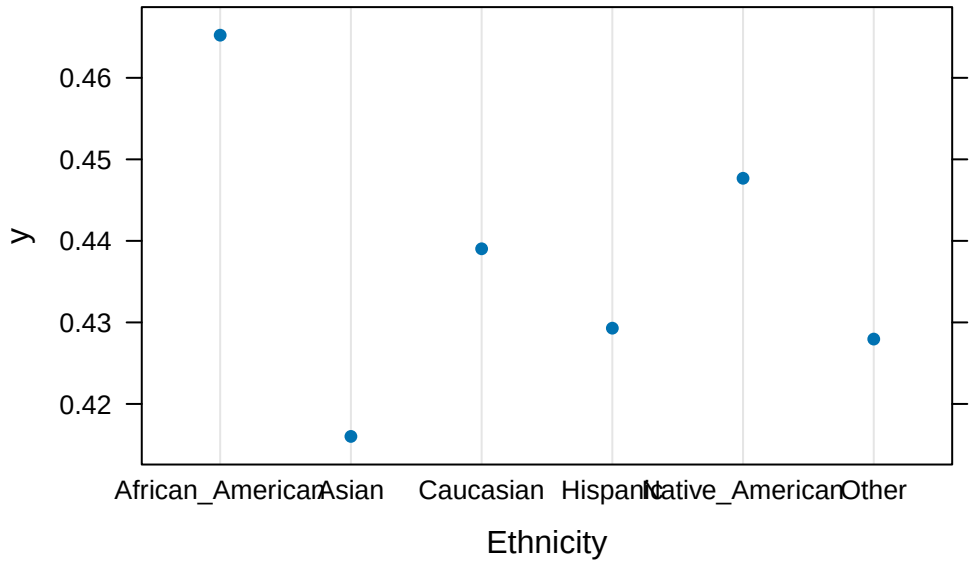
```
plot(gradboost1, "Number_of_Priors", type = "response")
```



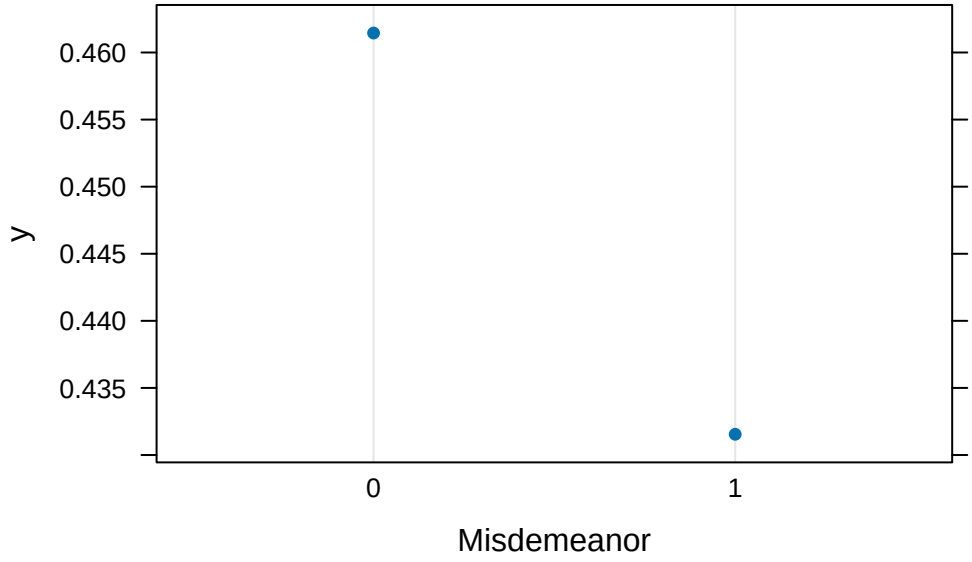

```
plot(gradboost1, "Age_Below_TwentyFive", type = "response")
```



```r
plot(gradboost1, "Ethnicity", type = "response")
```




```r
plot(gradboost1, "Misdemeanor", type = "response")
```



8.3 Prediksjon og confusion matrix

Vi gjør prediksjon på tilsvarende måte som før, men det er et par viktige detaljer. Forvalget for `predict()` for gbm-objekter er $f(x)$ som her er på log odds skalaen, men hvis vi setter `type = "response"` så får vi ut en sannsynlighet (dvs. et tall mellom 0 og 1). Da kan vi klassifisere etter hvilken gruppe som er mest sannsynlig, dvs. hvis høyere enn 0.5

```
compas_p <- training %>%
  mutate(pred = predict(gradboost1, type = "response"),
         p_klass = ifelse(pred > 0.5, 1, 0))
```

Lager enkel krysstabell med predikert mot observert (dvs confusion matrix)

```
tab <- table(compas_p$p_klass, training$Two_yr_Recidivism)
tab
```

```
       0    1
  0 1936  850
  1  583 1260
```

Lager bedre confusion matrix med alle øvrige utregninger. NB! Husk å presisere hva som er positiv verdi for at tallene skal blir riktig vei.

```
confusionMatrix(tab, positive="1")
```

```
Confusion Matrix and Statistics


       0    1
  0 1936  850
  1  583 1260

               Accuracy : 0.6904
                 95% CI : (0.6769, 0.7037)
    No Information Rate : 0.5442
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.3695
```

```
 Mcnemar's Test P-Value : 2.113e-12

            Sensitivity : 0.5972
            Specificity : 0.7686
         Pos Pred Value : 0.6837
         Neg Pred Value : 0.6949
             Prevalence : 0.4558
         Detection Rate : 0.2722
   Detection Prevalence : 0.3981
      Balanced Accuracy : 0.6829

       'Positive' Class : 1
```

8.4 Asymetriske kostnader

Vi har det vanlige problemet med å vurdere om falske positive og falske negative er like problematisk. Igjen er det slik at den vurderingen krever en vurdering av utfallet man ønsker gjøre noe med og konsekvensene av tiltaket som skal iverksettes. For credit-dataene kan det jo være at noen ikke vurderes som kredittverdige.[2]

For å håndtere asymetriske kostnader kan vi vekte utfallene forskjellig og angi denne vekten i prosedyren. Argumentet `weights = ...` tar en vektor med verdier (ét tall for hver observasjon i dataene) og skal ikke være en del av datasettet.

Her er et eksempel der *negative* tillegges 2 ganger så mye vekt som *positive* i modellen. Når utfallene vektes ulikt i modellen vil det dermed slå ut på feilratene slik at det blir ulikt forhold for falske postive og falske negative (se mer i Berk kap. 6.4 og eksempel s. 277).

Vær obs på at det er ikke så lett å vite helt hvordan slik vekting slår ut på resultatene. Det er ikke et 1-til-1 forhold mellom vekting og feilrater. Som i annen justering kan dette gå ut over accuracy og andre mål, så det er vanligvis en trade-off her. Du må derfor sjekke resultatene og så evt. gå tilbake og justere vektene hvis det ikke ble slik du ønsket.

```r
wts <- training %>%
  mutate(wts = ifelse(Two_yr_Recidivism == 1, 1, 2)) %>%   # se begrunnelse s. 274 i Berk
  pull(wts)

set.seed(542)
```

[2] Det er forresten også en implisitt vurdering her om hvem sitt problem dette er: banken behøver ikke synes det er problematisk å si nei til et boliglån, mens det fra samfunnets synsvinkel kan være uheldig at noen grupper sperres ute av boligmarkedet. Hvem som er "stakeholder" her er altså også et poeng, som vi lar ligge i denne sammenheng.

```r
gradboost2 <- gbm(formula = Two_yr_Recidivism ~ .,
                  data = training,
                  weights = wts,
                  distribution = "bernoulli",
                  n.trees = 4000,
                  interaction.depth = 3,
                  n.minobsinnode = 1,
                  shrinkage = 0.001,
                  bag.fraction = 0.5)
```

Så kan vi gjøre prediksjon og confusion matrix på nytt.

```r
compas_p <- training %>%
  mutate(pred = predict(gradboost2, type = "response"),
         p_klass = ifelse(pred > 0.5, 1, 0))
```

Lager enkel krysstabell med predikert mot observert (dvs confusion matrix)

```r
tab <- table(compas_p$p_klass, training$Two_yr_Recidivism)
tab
```

```
       0    1
  0 2329 1447
  1  190  663
```

Så kan vi legge tabellen inn i funksjonen `confusionMatrix()` for å også få diverse utregninger. (Husk å presisere hva som er positiv verdi for at tallene skal blir riktig vei.) Dette er altså akkurat samme prosedyre som vi har brukt ved tidligere prediksjoner.

```r
confusionMatrix(tab, positive="1")
```

```
Confusion Matrix and Statistics


       0    1
  0 2329 1447
  1  190  663

               Accuracy : 0.6464
```

```
                95% CI : (0.6324, 0.6601)
    No Information Rate : 0.5442
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.2509

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.3142
            Specificity : 0.9246
         Pos Pred Value : 0.7773
         Neg Pred Value : 0.6168
             Prevalence : 0.4558
         Detection Rate : 0.1432
   Detection Prevalence : 0.1843
      Balanced Accuracy : 0.6194

       'Positive' Class : 1
```

8.5 Hva med bias og fairness?

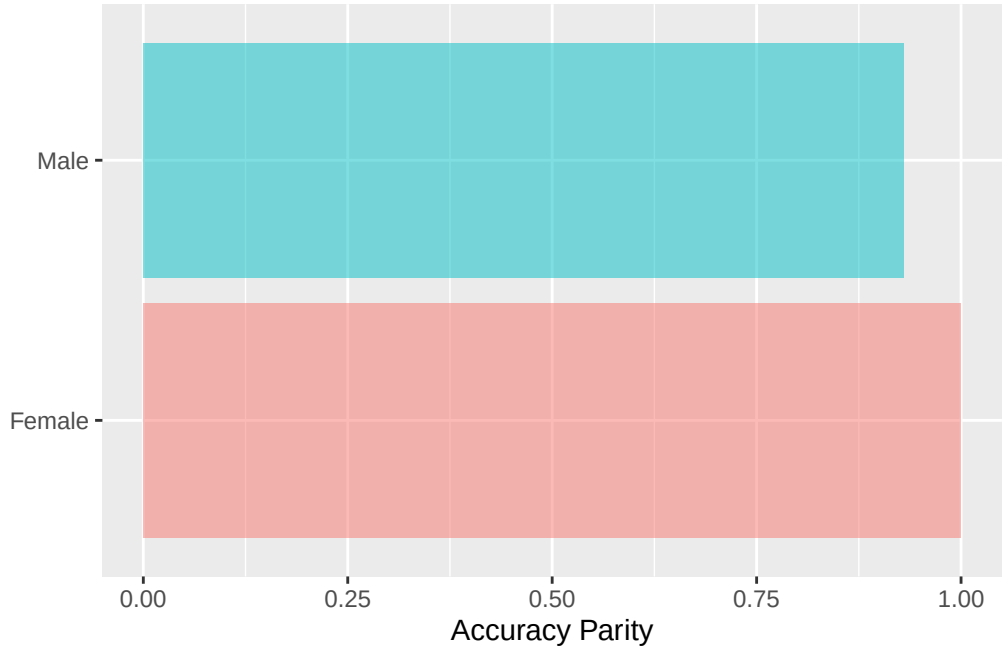
Det er heller ingenting nytt når det gjelder om prediksjonen kan slå ut skjevt for ulike grupper. Selvfølgelig kan det skje, og det bør jo undersøkes. Vi kan bruke akkurat de samme funksjonene som før.

Her er et eksempel med forskjeller i "accuracy" for menn og kvinner. Det er altså høyere andel riktig klassifiserte for kvinner enn for menn.

```r
acc <- fairness::acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Sex',
                  preds        = 'p_klass',
                  base         = 'Female')

acc
```

```
$Metric
                     Female         Male
Accuracy          0.6853933    0.6370687
Accuracy Parity   1.0000000    0.9294937
```

```
Group size      890.0000000 3739.0000000

$Metric_plot
```



8.6 Justere rettferdighet med vekting

Vi har allerede sett at vi kan justere algoritmen for asymetriske kostnader ved å vekte utfallet forskjellig. Vi kan også justere slik at vi også vekter opp undergrupper. Igjen er det ikke helt åpenbart hvordan det vil slå ut å vekte en gruppe opp eller ned, så det må vi sjekke.

Her er en kode for å lage vekter og legger den vekten inn i `gbm`. Tanken i koden her er å først angi vektingen for kostnader og undergrupper hver for seg, og så legge disse til per person i datasettet.

La oss først se på fordelingen av utfallet etter kjønn ved en enkel krysstabell. Her har jeg brukt funksjonen `tbl_cross()` for å få en penere tabell med summeringer, men en enklere tabell gjør også nytten.

```
training %>%
   tbl_cross( row = Sex, col =  Two_yr_Recidivism)
```

	Two_yr_Recidivism		
	0	1	Total
Sex			
Female	575	315	890
Male	1,944	1,795	3,739
Total	2,519	2,110	4,629

Vi ser her at i datamaterialet er det lavere residiv blant kvinner enn menn, men det er også færre kvinner enn menn totalt. Dette kan jo være en indikator på hva som bør vektes opp. Her er et forslag der kvinner vektes opp generelt, men kvinner med residiv vektes opp enda mer. For menn vektes også residiv opp.

Vekten skal lages så det blir et tall per observasjon i datasettet, så utgangspunktet er trainingdataene. Så la oss lage en vekt som tar utgangpunktet i menn uten residiv og vekter det som 1. Hvis vi vil vekte opp residiv, så settes menn med residiv til 2. Tilsvarende for kvinner kan vi sette vektene høyere, f.eks. henholdsvis 2 og 4.

I koden nedenfor lages også en krysstabell bare for å sjekke at det ble slik det var tenkt. Tabellen har bare funksjon for å sjekke at det ble riktig.

```
wts <- training %>%
  mutate(wts1 = case_when(
    Sex == "Male" & Two_yr_Recidivism == 0 ~ 1,
    Sex == "Male" & Two_yr_Recidivism == 1 ~ 2,
    Sex == "Female" & Two_yr_Recidivism == 0 ~ 2,
    Sex == "Female" & Two_yr_Recidivism == 1 ~ 4)) %>%
    pull(wts1)

library(gtsummary)
df <- training %>%
  mutate(wts = factor(wts))

df %>%
  select(Sex, Two_yr_Recidivism, wts) %>%
 tbl_strata(
    strata = Sex,
    ~.x %>%
      tbl_summary(
        by = Two_yr_Recidivism,
        #type = everything()~"categorical",
        statistic = all_categorical() ~ "{n}"
```

Characteristic	Female		Male	
	0 N = 575[1]	**1** N = 315[1]	**0** N = 1,944[1]	**1** N = 1,795[1]
wts				
1	0	0	1,944	0
2	575	0	0	1,795
4	0	315	0	0

[1] n

```
      )
  )
```

Kodingen av vekten kan treng litt forklaring. Det er forsåvidt standard R-kode, men likevel:

- `case_when()` brukes for gjentatt if-else i flere ledd. Altså først vurderes første kriterium og får tilskrevet den verdien som følger etter `~`. *For øvrige observasjoner* sjekkes neste kriterium på samme måte.
- `pull()` brukes når man bare skal beholde én variabel som en vektor (ikke ny data.frame)
- `tbl_strata` fra pakken gtsummary brukes til å lage en tre-veis tabell. Det spiller liten rolle hvordan tabellen lages, men denne gir en pen og lett lesbar tabell.

Da kan vi gjøre en ny boostingmodell og sjekke resultatene. Koden er identisk som ovenfor, bare at vektene er endret

```
set.seed(542)
gradboost3 <- gbm(formula = Two_yr_Recidivism ~ .,
                  data = training,
                  weights = wts,
                  distribution = "bernoulli",
                  n.trees = 4000,
                  interaction.depth = 3,
                  n.minobsinnode = 1,
                  shrinkage = 0.001,
                  bag.fraction = 0.5)
```

Vi gjør prediksjon og klassifisering på samme måte som før:

```
compas_p <- training %>%
  mutate(pred = predict(gradboost3, type = "response"),
         p_klass = ifelse(pred > 0.5, 1, 0))
```

Lager enkel krysstabell med predikert mot observert (dvs confusion matrix):

```
tab <- table(compas_p$p_klass, training$Two_yr_Recidivism)
tab
```

```
       0    1
  0 1231  369
  1 1288 1741
```

Lager bedre confusion matrix med alle øvrige utregninger. NB! Husk å presisere hva som er positiv verdi for at tallene skal blir riktig vei.

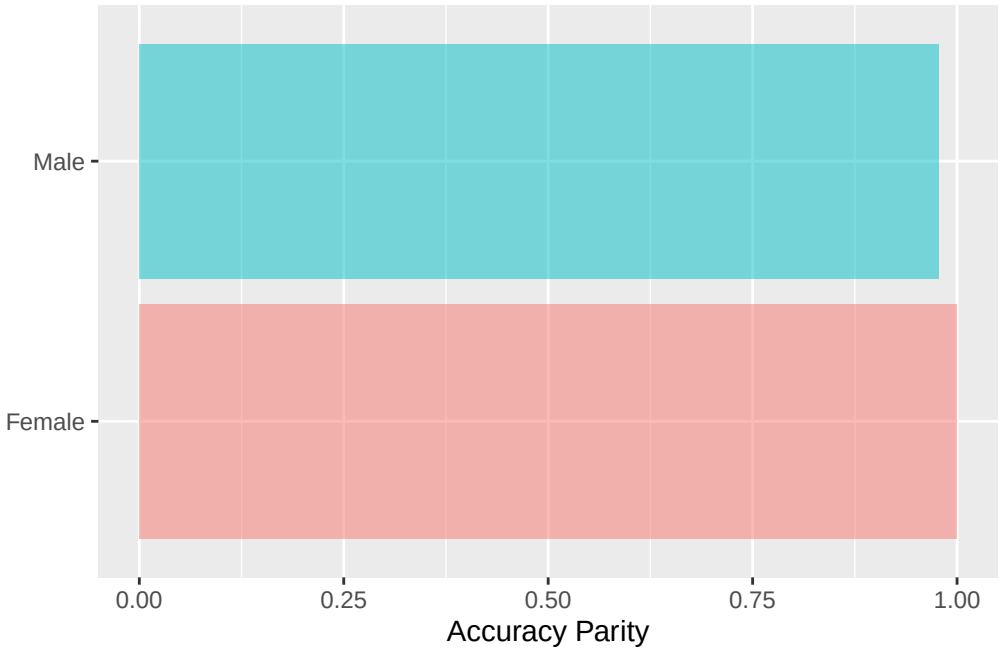
```
confusionMatrix(tab, positive="1")
```

```
Confusion Matrix and Statistics


       0    1
  0 1231  369
  1 1288 1741

               Accuracy : 0.642
                 95% CI : (0.628, 0.6559)
    No Information Rate : 0.5442
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.3031

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.8251
            Specificity : 0.4887
         Pos Pred Value : 0.5748
         Neg Pred Value : 0.7694
             Prevalence : 0.4558
         Detection Rate : 0.3761
   Detection Prevalence : 0.6544
      Balanced Accuracy : 0.6569

       'Positive' Class : 1
```

Sammenlign nå feilratene med den forrige modellen (uten gruppevekting). Ble forholdet mellom falske positive og falske negative annerledes?

Så sjekker vi om den nye vektingen har gjort modellen mer rettferdig med hensyn på kjønn:

```
acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Sex',
                  preds        = 'p_klass',
                  base         = 'Female')
acc[[2]]
```



Sammenlign dette med fairness-resultatet fra den uvektede modellen. Ble det mer rettferdig? Vekting er altså et verktøy for å justere bias i boosting på tilsvarende måte som `sampsize` er det for random forest. Men som alltid: det er en trade-off mellom rettferdighet og overall accuracy.

9 En lett introduksjon til fairness

I dette kapittelt skal vi bruke følgende pakker:

```
library(tidyverse)   # Datahåndtering, grafikk og glimpse()
library(rsample)     # for å dele data i training og testing
library(caret)       # Funksjonen confusionMatrix()
library(fairness)    # Beregne mål på fairness
```

Nå som vi har sett på klassifikasjonstrær, bagging og random forest er det på tide å gjøre noen rettferdighetsbetraktninger. Vi starter med et eksempel basert på logistisk regresjon for å introdusere begreper og mål, og i neste kapittel skal vi se hvordan vi kan *tune* random forest for å motvirke bias.

9.1 Hva slags rettferdighet

I denne settingen kan rettferdighet kommer i betraktning på flere måter, herunder følgende:

- I hvilken grad maskiner vs mennesker tar avgjørelser, og herunder mulighet til å bli hørt og legge frem sin sak
- I hvilken grad *dataene* algoritmen er trent opp på inneholder skjevheter i utgangspunktet som så reproduseres i videre implementering
- I hvilken grad sluttresultatet har rimelig presisjon og akseptable feilrater, herunder vurdering av *asymetriske feilrater*
- I hvilken grad forrige punkt er avpasset mot hvilke *tiltak* man så setter i verk
- I hvilken grad feilrater og presisjon varierer systematisk med undergrupper i populasjonen

Det er nok av ting å tak i her, men vi skal her fokusere på det som kan tallfestes gitt den modellen man har. Men for all del: Hvis datakvaliteten er det begrenset hvor bra det kan bli uansett. Selv om kjente skjevheter i dataene kan i prinsippet motarbeides, så er det vel i praksis slik at en skjevhet kommer sjelden alene?

Vurderinger av overordnet feilrater er et gjennomgående tema, så vi starter med det. Deretter skal vi se på mål på skjevheter over undergrupper. Prinsippet er relativt enkelt, uten at vurderingene blir enkle av den grunn.

9.2 Mer confusion matrix

Et utgangspunkt er det vi kaller confusion matrix. Det er rett og slett en krysstabell der vi sammenholder predikert og observert utfall. Å lage en slik tabell avhenger ikke av metoden vi bruker, bare at vi kan sammenlignet prediksjon med faktisk utfall.

Vi bruker eksempelet fra forrige kapittel med logistisk regresjon.

```r
est_multlogit <- glm(Attrition ~ ., data = training, family = "binomial")
summary(est_multlogit)
```

```
Call:
glm(formula = Attrition ~ ., family = "binomial", data = training)

Coefficients:
                                  Estimate Std. Error z value Pr(>|z|)
(Intercept)                     -1.090e+01  7.450e+02  -0.015 0.988322
Age                             -3.671e-02  1.637e-02  -2.243 0.024906 *
BusinessTravelTravel_Frequently  1.627e+00  4.663e-01   3.490 0.000483 ***
BusinessTravelTravel_Rarely      8.847e-01  4.265e-01   2.074 0.038070 *
DailyRate                       -3.249e-04  2.657e-04  -1.223 0.221450
DepartmentResearch & Development 1.303e+01  7.450e+02   0.017 0.986048
DepartmentSales                  1.348e+01  7.450e+02   0.018 0.985565
DistanceFromHome                 4.325e-02  1.308e-02   3.307 0.000944 ***
Education                        2.259e-03  1.047e-01   0.022 0.982789
EducationFieldLife Sciences     -8.614e-01  1.036e+00  -0.832 0.405637
EducationFieldMarketing         -2.986e-01  1.085e+00  -0.275 0.783245
EducationFieldMedical           -9.341e-01  1.037e+00  -0.901 0.367673
EducationFieldOther             -8.090e-01  1.106e+00  -0.731 0.464508
EducationFieldTechnical Degree   6.014e-02  1.057e+00   0.057 0.954635
EmployeeNumber                  -3.297e-04  1.818e-04  -1.814 0.069727 .
EnvironmentSatisfaction         -4.976e-01  9.990e-02  -4.981 6.31e-07 ***
GenderMale                       2.224e-01  2.157e-01   1.031 0.302405
HourlyRate                      -3.877e-03  5.392e-03  -0.719 0.472136
JobInvolvement                  -5.454e-01  1.458e-01  -3.741 0.000183 ***
JobLevel                        -1.437e-02  3.834e-01  -0.037 0.970101
JobRoleHuman Resources           1.413e+01  7.450e+02   0.019 0.984868
JobRoleLaboratory Technician     1.579e+00  5.597e-01   2.822 0.004773 **
JobRoleManager                  -1.613e+00  1.280e+00  -1.260 0.207596
JobRoleManufacturing Director   -2.403e-01  6.135e-01  -0.392 0.695291
JobRoleResearch Director        -2.531e+00  1.243e+00  -2.035 0.041806 *
```

```
JobRoleResearch Scientist         3.845e-01  5.748e-01   0.669 0.503513
JobRoleSales Executive            3.167e-01  1.614e+00   0.196 0.844462
JobRoleSales Representative       1.478e+00  1.662e+00   0.890 0.373555
JobSatisfaction                  -3.238e-01  9.696e-02  -3.340 0.000839 ***
MaritalStatusMarried              3.633e-01  3.130e-01   1.161 0.245800
MaritalStatusSingle               9.102e-01  4.057e-01   2.243 0.024875 *
MonthlyIncome                     9.730e-05  9.890e-05   0.984 0.325190
MonthlyRate                       1.173e-05  1.499e-05   0.783 0.433906
NumCompaniesWorked                1.705e-01  4.652e-02   3.665 0.000248 ***
OverTimeYes                       1.892e+00  2.308e-01   8.195 2.50e-16 ***
PercentSalaryHike                -6.513e-02  4.714e-02  -1.382 0.167122
PerformanceRating                 6.772e-01  4.816e-01   1.406 0.159676
RelationshipSatisfaction         -3.888e-01  9.884e-02  -3.934 8.37e-05 ***
StockOptionLevel                 -2.737e-01  1.822e-01  -1.502 0.133202
TotalWorkingYears                -6.528e-02  3.587e-02  -1.820 0.068731 .
TrainingTimesLastYear            -2.136e-01  8.571e-02  -2.492 0.012712 *
WorkLifeBalance                  -2.117e-01  1.469e-01  -1.441 0.149470
YearsAtCompany                    6.602e-02  4.876e-02   1.354 0.175764
YearsInCurrentRole               -1.021e-01  5.548e-02  -1.840 0.065748 .
YearsSinceLastPromotion           2.056e-01  5.166e-02   3.979 6.91e-05 ***
YearsWithCurrManager             -1.933e-01  6.078e-02  -3.180 0.001473 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 944.39  on 1101  degrees of freedom
Residual deviance: 616.66  on 1056  degrees of freedom
AIC: 708.66

Number of Fisher Scoring iterations: 15
```

Så kan vi gjøre en prediksjon på testing-datasettet og lagre resultatet i et nytt objekt. Vi gjør en klassifisering ved å bestemme en cut-off på f.eks. en sannsynligheten over 50%.

```r
attrition_test <- testing %>%
  mutate(prob = predict(est_multlogit, newdata = testing, type = "response")) %>%
    mutate(attrition_class = as.factor(ifelse(prob < .5, "No", "Yes")))
```

Så kan vi lage confusion matrix som følger. Vi bruker funksjonen `confusionMatrix()` for at diverse mål skal blir regnet ut automatisk for oss, men du kan godt regne ut dette for hånd også (hvis du gidder).

```r
cm <- confusionMatrix(attrition_test$Attrition, attrition_test$attrition_class, positive = "Y

cm
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  288  12
       Yes  42  26

               Accuracy : 0.8533
                 95% CI : (0.8129, 0.8878)
    No Information Rate : 0.8967
    P-Value [Acc > NIR] : 0.9965

                  Kappa : 0.4128

 Mcnemar's Test P-Value : 7.933e-05

            Sensitivity : 0.68421
            Specificity : 0.87273
         Pos Pred Value : 0.38235
         Neg Pred Value : 0.96000
             Prevalence : 0.10326
         Detection Rate : 0.07065
   Detection Prevalence : 0.18478
      Balanced Accuracy : 0.77847

       'Positive' Class : Yes
```

Hvorvidt dette er "rettferdig" eller ikke kommer jo an på hva man skal gjøre med denne prediksjonen. Så det er jo en første vurdering. Synes man feilratene er ok? Er evt. konsekvensene av å ikke gjøre noe ok?

I dette eksempelet er positive predictive value 0.38 som altså er andelen av de som er predikert som positive som er rett. Det betyr at hvis vi setter i verk tiltak mot alle som er predikert positive, så vil 0.62 være bortkastet. På den annen side, vil 0.68 av de som faktisk er positive fanges opp (sensitivity).[1]

[1]For akkurat dette kurset er ikke disse tekniske begrepene som sensitivity, specificity osv sentrale. (Em-

Characteristic	Overall N = 368[1]	No N = 300[1]	Yes N = 68[1]
JobRole			
Healthcare Representative	32 (100%)	31 (97%)	1 (3.1%)
Human Resources	14 (100%)	11 (79%)	3 (21%)
Laboratory Technician	64 (100%)	47 (73%)	17 (27%)
Manager	27 (100%)	24 (89%)	3 (11%)
Manufacturing Director	32 (100%)	29 (91%)	3 (9.4%)
Research Director	18 (100%)	17 (94%)	1 (5.6%)
Research Scientist	77 (100%)	59 (77%)	18 (23%)
Sales Executive	84 (100%)	70 (83%)	14 (17%)
Sales Representative	20 (100%)	12 (60%)	8 (40%)

[1] n (%)

9.3 Mål på fairness

9.3.1 Predictive rate parity

Ovenfor så vi blant annet at "positive predicted value" , altså andelen sanne positive av alle predikerte positive, er 0.382.

Men det er ulike typer jobber i denne bedriften. Her er fordelingen for test-datasettet:

For illustrasjonens skyld kan vi da dele inn datamaterialet i to deler: lab-teknikkere og resten. For hver gruppe kan vi så lage en confusion matrix og undersøke verdiene.

```
labTech <- attrition_test %>%
  filter(JobRole %in% c("Laboratory Technician"))

others <- attrition_test %>%
  filter( !(JobRole %in% c("Laboratory Technician") ))

cm1 <- confusionMatrix(labTech$Attrition, labTech$attrition_class, positive = "Yes")

cm2 <- confusionMatrix(others$Attrition, others$attrition_class, positive = "Yes")

cm1
```

```
Confusion Matrix and Statistics
```

neansvarlig går surr i disse selv). Men det forventes at du skal kunne formulere det på tilsvarende måte som her. Men du må huske *accuracy*, som er den enkleste.

```
          Reference
Prediction No Yes
       No  39   8
       Yes 10   7

               Accuracy : 0.7188
                 95% CI : (0.5924, 0.824)
    No Information Rate : 0.7656
    P-Value [Acc > NIR] : 0.8490

                  Kappa : 0.251

 Mcnemar's Test P-Value : 0.8137

            Sensitivity : 0.4667
            Specificity : 0.7959
         Pos Pred Value : 0.4118
         Neg Pred Value : 0.8298
             Prevalence : 0.2344
         Detection Rate : 0.1094
   Detection Prevalence : 0.2656
      Balanced Accuracy : 0.6313

       'Positive' Class : Yes
```

```r
cm2
```

```
Confusion Matrix and Statistics

          Reference
Prediction  No Yes
       No  249   4
       Yes  32  19

               Accuracy : 0.8816
                 95% CI : (0.8398, 0.9157)
    No Information Rate : 0.9243
    P-Value [Acc > NIR] : 0.997

                  Kappa : 0.4569
```

```
 Mcnemar's Test P-Value : 6.795e-06

            Sensitivity : 0.82609
            Specificity : 0.88612
         Pos Pred Value : 0.37255
         Neg Pred Value : 0.98419
             Prevalence : 0.07566
         Detection Rate : 0.06250
   Detection Prevalence : 0.16776
      Balanced Accuracy : 0.85610

       'Positive' Class : Yes
```

Positive predicted value for lab-teknikker er 0.412 og for resten 0.373.

Forholdstallet mellom disse er 0.9047619, alså nesten likt. Slik sett kan vi si at modellen er rettferdig på dette målet ved at disse to gruppene er like.

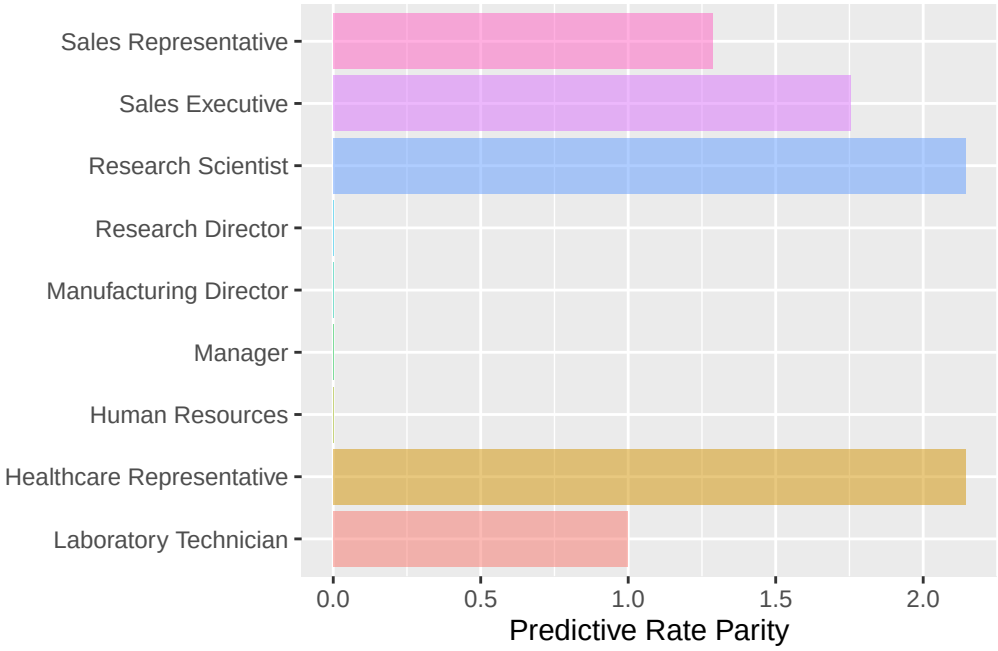
Men hvis du sjekker output fra confusionMatrix ovenfor, så er jo ikke alle tallene like. Så det kommer an på hvilke mål du sammenligner.

Pakken *fairness* gjør en tilsvarende beregning for deg og kan gi resultatet grafisk. Her er sammenligning av positive predicted value gjort for alle grupper av jobber:

```r
pred_rate_parity(data = attrition_test,
                 outcome = "Attrition",
                 group = "JobRole",
                 preds = "attrition_class",
                 base = "Laboratory Technician"
                 )
```

```
$Metric
                      Laboratory Technician Healthcare Representative
Precision                           0.4666667                  1.000000
Predictive Rate Parity              1.0000000                  2.142857
Group size                         64.0000000                 32.000000
                      Human Resources Manager Manufacturing Director
Precision                              NA      NA                     NA
Predictive Rate Parity                 NA      NA                     NA
Group size                             14      27                     32
                      Research Director Research Scientist Sales Executive
```

```
Precision                              NA          1.000000       0.8181818
Predictive Rate Parity                 NA          2.142857       1.7532468
Group size                             18         77.000000      84.0000000
                       Sales Representative
Precision                          0.600000
Predictive Rate Parity             1.285714
Group size                        20.000000

$Metric_plot
```



Nå er det vesentig større forskjeller. Utvilsomt er grunnen at gruppen av 'andre' var sammensatt av veldig ulike grupper som var veldig forskjellig innbyrdes, men som jevnet hverandre ut i snitt. Noen av disse gruppene var dessuten små.

Man kan mene ulikt om slikt, men det er i hvert fall ikke likt på tvers av grupper.

Men hvilke konsekvenser har dette? Treffsikkerheten er da lavere for laboratorie-teknikkerne enn for de andre gruppene. Ikke sikkert det er så nøye.

Men la oss si at man bestemmer seg for å gi en solid lønnsforhøyelse (eller andre goder) til de man tror er i fare for å bytte jobb. Da vil bedriften bruke unødig mye midler på lab-teknikkerne (dvs. relativt mange falske positive) relativt til andre grupper .

Men man kunne også tenke seg et negativt tiltak: de som er predikert å ville slutte vil *ikke* få lønnsforhøyelse eller andre goder. Hvis vurderingen er at de vil slutte uansett, så er det bortkastede ressurser. Flere blant lab-teknikerne ville da urettmessig få avslag.

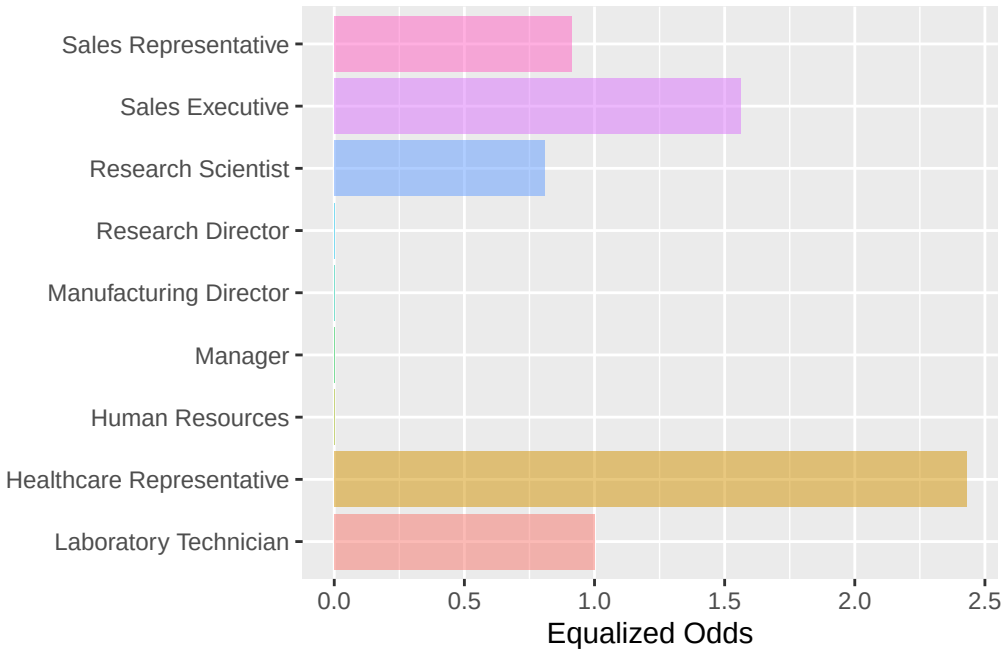
Ok. Det ville uansett være en lite klok arbeidsgiver som baserer seg kun på dette. Det finnes bedre måter å beholde arbeidstakerne sine på. Men man kan tenke seg andre situasjoner der konsekvensene er tydeligere: Hva om det var en automatisert sjekk av kredittverdighet for å få boliglån? Eller kanskje opptak til et lederprogram? Eller risikovurdering for prøveløslatelse fra fengsel?

9.3.2 Equalized odds

En annen variant er å se på andelen sanne positive av andelen som faktisk er positive. Altså: andelen som er predikert å slutte av de som faktisk slutter. Hvis dette tallet er likt mellom grupper betyr det at modellen er like god til å fange opp de som faktisk slutter uavhengig av jobbtype. Hvis det er ulikt betyr det at modellen er flinkere til å oppdage sluttere i noen jobber enn andre.

(I følgende kode er det lagt på `[[2]]` på slutten for å bare vise figuren, altså droppe tabellen. Akkurat den delen av koden er ikke viktig).

```
equal_odds(data = attrition_test,
                 outcome = "Attrition",
                 group = "JobRole",
                 preds = "attrition_class",
                 base = "Laboratory Technician"
                 )[[2]]
```



9.3.3 False positive rate parity

Er falske positive rate i ulike grupper lik? Falsk positiv rate er andelen av de som *ikke* slutter som likevel blir feilaktig klassifisert som at de vil slutte. Hvis denne raten er ulik mellom grupper, vil noen grupper bli utsatt for flere unødvendige tiltak enn andre.

```
fpr_parity(data = attrition_test,
                 outcome = "Attrition",
                 group = "JobRole",
                 preds = "attrition_class",
                 base = "Laboratory Technician"
                 )[[2]]
```

9.4 Litt oppsummerende

Den R-pakken vi bruker her har en hel rekke ulike mål innebygget. Se en oversikt på pakkens *vignette*.

Hvilket mål som er viktigst er det derimot ikke noe klart svar på. Det kommer an på hva man har tenkt til å bruke prediksjonen til, hvilke konsekvenser tiltaket har, og hvilke konsekvenser det har å ikke gjøre noe. Disse konsekvensene kan vurderes forskjellig for personer etter om det er riktig eller feil prediksjon.

Det du må ta stilling til er egentlig hvor viktig du synes likebehandling er! Ofte er jo det åpenbart veldig viktig.

Så bør det nevnes at noen ganger ville man gjort noe uansett. Altså gjort de samme tiltakene basert på skjønn eller andre typer vurderinger. Det vil også ha feilrater og forskjeller mellom undergrupper i slike tilfeller, selv om man ikke har et oppsett som gir testing-data å estimere dette på. Det betyr at du ikke bare må ta stilling til om algoritmen er "fair" eller ikke, men også om den er mer eller mindre "fair" enn den alternative fremgangsmåten.

10 Fairness og tuning

10.1 Introduksjon til fairness

I forrige kapittel så vi at `sampsize` er et viktig verktøy for å justere feilratene i random forest. Nå skal vi bruke dette til å adressere *bias* mellom undergrupper. En ting er å avdekke bias og urettferdighet, en annen ting er å gjøre noe med det!

```
library(tidyverse)
library(randomForest)
library(caret)
library(fairness)
```

Compas er et risikoverktøy brukt av amerikansk politi i flere stater som benyttes på individnivå. Bruken av dette verktøyet har vært kontroversielt i flere år og kraftig kritisert av flere. En viktig grunn er at prediksjonene slår forskjellig ut for ulike grupper og er slik sett "biased" mot bl.a. svarte borgere. Resultatet er at de blir mer utsatt for politiets oppmerksomhet enn andre.[1] Et datasett er gjort tilgjengelig av Propublica her som vi skal bruke.

```
compas <- readRDS("data/compas.rds")

glimpse(compas)
```

```
Rows: 6,172
Columns: 7
$ Two_yr_Recidivism    <fct> 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1~
$ Number_of_Priors     <int> 0, 0, 4, 0, 14, 3, 0, 0, 3, 0, 0, 1, 7, 0, 3, 6, ~
$ Age_Above_FourtyFive <fct> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0~
$ Age_Below_TwentyFive <fct> 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
$ Misdemeanor          <fct> 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0~
$ Ethnicity            <fct> Other, African_American, African_American, Other,~
$ Sex                  <fct> Male, Male, Male, Male, Male, Male, Female, Male,~
```

[1] Det er verd å minne på at politi i USA i stor grad er mer hardhendte enn norsk politi. Konsekvensene er altså litt mer alvorlig enn at unødig mange føler seg unødig mistenkte.

Vi tilpasser først en random forest modell.

```
set.seed(4356)
glimpse(compas)
```

```
Rows: 6,172
Columns: 7
$ Two_yr_Recidivism    <fct> 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1~
$ Number_of_Priors     <int> 0, 0, 4, 0, 14, 3, 0, 0, 3, 0, 0, 1, 7, 0, 3, 6, ~
$ Age_Above_FourtyFive <fct> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0~
$ Age_Below_TwentyFive <fct> 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
$ Misdemeanor          <fct> 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0~
$ Ethnicity            <fct> Other, African_American, African_American, Other,~
$ Sex                  <fct> Male, Male, Male, Male, Male, Male, Female, Male,~
```

```
rf <- randomForest(Two_yr_Recidivism ~ .,
                   #importance = TRUE,
                    data = compas)
```

Lager en prediksjon i en kopi av datasett, mest for å ikke blande sammen prediksjonene med det opprinnelige datasettet.

```
compas_p <- compas %>%
  mutate(pred_rf = predict(rf))
```

Da kan vi lage en confusion matrix for å se hvordan modellen fungerer.

```
confusionMatrix(compas_p$pred_rf,
                compas_p$Two_yr_Recidivism, positive="1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 2462 1132
         1  901 1677

               Accuracy : 0.6706
                 95% CI : (0.6587, 0.6823)
    No Information Rate : 0.5449
```

```
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.3313

 Mcnemar's Test P-Value : 3.378e-07

            Sensitivity : 0.5970
            Specificity : 0.7321
         Pos Pred Value : 0.6505
         Neg Pred Value : 0.6850
             Prevalence : 0.4551
         Detection Rate : 0.2717
   Detection Prevalence : 0.4177
      Balanced Accuracy : 0.6645

       'Positive' Class : 1
```

For å få litt bedre grep om hvordan fairness-pakken fungerer kan gjøre det mer manuelt først. Vi kan f.eks. se på hvordan modellen slår ut for menn og kvinner.

Først kan vi da splitte datasettet i to etter kjønn. Her for menn.

```r
compas_1 <- compas_p %>%
  filter(Sex == "Male")
```

Confusion matrix for menn blir da tilsvarende som tidligere.

```r
confusionMatrix(compas_1$pred_rf,
                compas_1$Two_yr_Recidivism, positive="1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 1807  897
         1  794 1499

               Accuracy : 0.6616
                 95% CI : (0.6483, 0.6747)
    No Information Rate : 0.5205
    P-Value [Acc > NIR] : < 2e-16
```

```
                 Kappa : 0.3209

 Mcnemar's Test P-Value : 0.01312

            Sensitivity : 0.6256
            Specificity : 0.6947
         Pos Pred Value : 0.6537
         Neg Pred Value : 0.6683
             Prevalence : 0.4795
         Detection Rate : 0.3000
   Detection Prevalence : 0.4589
      Balanced Accuracy : 0.6602

       'Positive' Class : 1
```

Merk at vi her fikk en accuracy på 0.66 for menn. Splitter datasettet i to etter kjønn. Her for kvinner.

```r
compas_2 <- compas_p %>%
  filter(Sex == "Female")
```

Confusion matrix for kvinner

```r
confusionMatrix(compas_2$pred_rf,
                compas_2$Two_yr_Recidivism, positive="1")
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 655 235
         1 107 178

               Accuracy : 0.7089
                 95% CI : (0.682, 0.7348)
    No Information Rate : 0.6485
    P-Value [Acc > NIR] : 6.251e-06

                  Kappa : 0.3128
```

```
 Mcnemar's Test P-Value : 6.539e-12

            Sensitivity : 0.4310
            Specificity : 0.8596
         Pos Pred Value : 0.6246
         Neg Pred Value : 0.7360
             Prevalence : 0.3515
         Detection Rate : 0.1515
   Detection Prevalence : 0.2426
      Balanced Accuracy : 0.6453

       'Positive' Class : 1
```

Merk at vi her fikk en accuracy på 0.71 for kvinner. Modellen er altså mer presis for kvinner enn for menn. Dette er et eksempel på *bias* i modellen. Forholdet mellom feilrater er da $.66/.71 = .93$. Dette tallet sier at modellens accuracy for menn er 93% av accuracy for kvinner, og dette er et mål på *bias* i modellen. Om man synes det er mye eller lite er derimot en vurderingssak!

Man kan også regne ut den andre veien, altså kvinner delt på menn. Da får vi 1.08, som sier at modellen er 8% mer presis for kvinner enn for menn. Det blir jo det samme, men motsatt vei. Vi må altså velge *referansegruppe* for sammenligningen.

Funksjoner `acc_parity` fra fairness-pakken gjør akkurat dette: regner ut accuracy for ulike grupper og sammenligner dem. Her er koden:

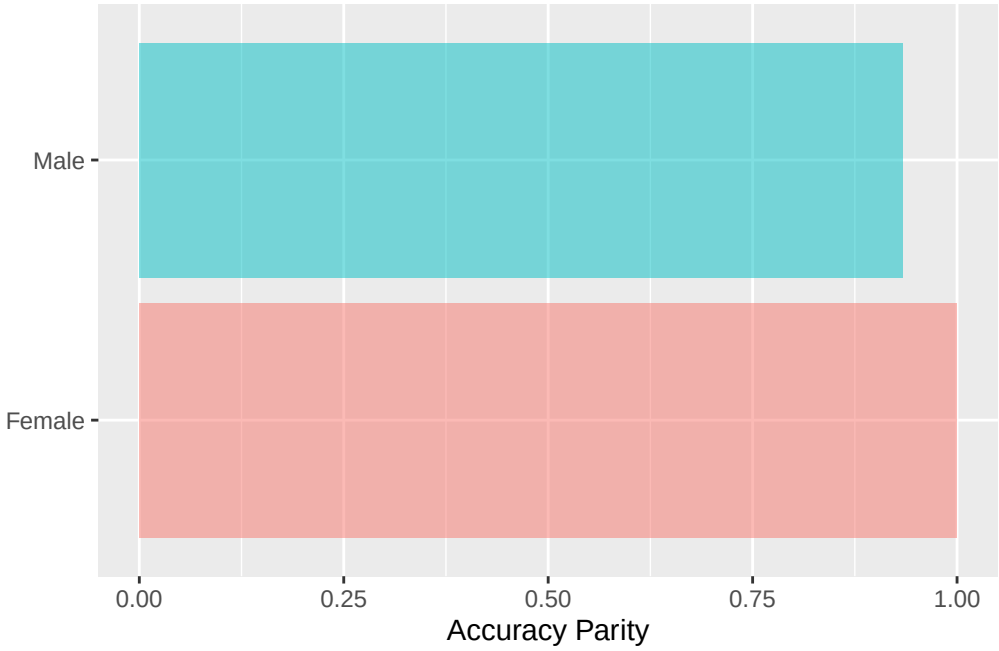
```r
acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Sex',
                  preds        = 'pred_rf',
                  base         = 'Female')
acc$Metric
```

```
                      Female        Male
Accuracy           0.7089362    0.661597
Accuracy Parity    1.0000000    0.933225
Group size      1175.0000000 4997.000000
```

Men spesifiserer altså hvilke variable som er utfallsvariabel, hvilken som er gruppevariabel, hvilken som er prediksjonsvariabel og hvilken gruppe som skal være referanse.

Fairness-pakken har også noen innebygde funksjoner for grafiske fremstillinger.

```
acc$Metric_plot
```



10.2 Fairness: tuning med stratifisering

Vi har lært litt om grunnleggende tuning. For å justere feilratene er den mest effektive måten i random forest å endre hvordan samplingen skjer ved `sampsize`.[2] Dette hjelper imidlertid ikke nødvendigvis mot *bias*, altså ulike feilrater for undergrupper. Det vil jo være nyttig om man kunne gjøre noe med slike skjevheter! Heldigvis kan vi det, men det er ikke så lett som man skulle håpe.

Det viktige punktet å huske på er at random forest trekker tilfeldige utvalg for hvert tre og parameteren `sampsize` justerer hvor mange observasjoner som trekkes. Vi kan nå justere algoritmen til å gjøre en *stratifisert* trekning. Med andre ord: det gjøres en tilfeldig trekning innenfor subgrupper. Ovenfor har vi altså sett at modellen ikke er helt rettferdig med hensyn på kjønn. La oss fikse det!

Først må vi lage en vektor for å stratifisere etter. Dette bør være variabelen for kjønn *kombinert med* utfallsvariabelen. Da får vi fire kategorier: menn med og uten tilbakefall, og kvinner med og uten tilbakefall. Så kan vi bruke sampsize til å angi hvor mange som skal trekkes fra hver av disse fire gruppene.

[2] Dette er ikke eneste måten å tune på, men det vi dekker her. Det finnes andre funksjoner for random forest som inneholder mer funksjonalitet for tuning.

Her er en kode for å lage en slik stratifiseringsvektor. Pussig nok skal denne vektoren ikke legges inn som en del av datasett, men i et eget objekt. I koden nedenfor bruker vi `mutate()` for å lage en ny variabel og `paste0()` for å "lime sammen" de to variabelene. Til sist bruker `pull()` for å trekke ut kun den variabelen i en egen vektor.[3] Koden nedenfor gir også en tabell med antall observasjoner i hver kategori av denne vektoren.

```
strat <- compas %>%
  mutate(strat = paste0(Sex, Two_yr_Recidivism)) %>%
  pull(strat)

table(strat)
```

```
strat
Female0 Female1   Male0   Male1
    762     413    2601    2396
```

Merk at denne vektoren har verdiene som er en kombinasjon av de opprinnelige variablene. Når vi nå angir tall i `sampsize()` så er det i denne samme rekkefølgen. Her blir det altså:

1) Kvinner uten tilbakefall
2) Kvinner med tilbakefall
3) Menn uten tilbakefall
4) Menn med tilbakefall

Merk da at tallene som angis for `sampsize` ikke kan være høyere enn antallet i hver gruppe, og helst ikke høyere enn ca 70% av dette. Men det er primært *forholdet mellom tallene* som er viktig.

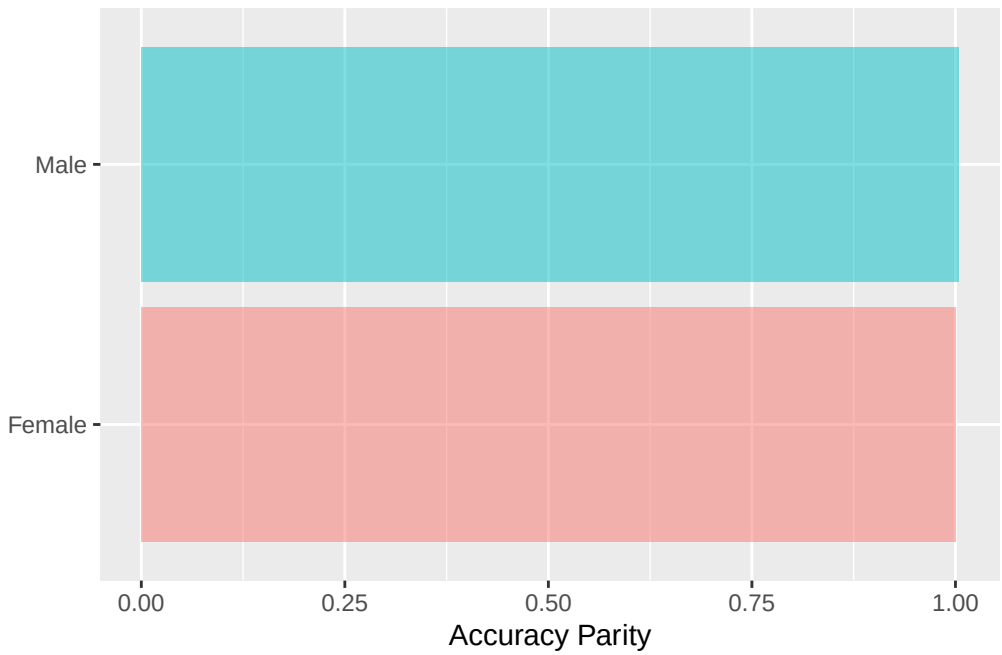
Her er kode for random forest med stratifisert utvalg og angitte verdier av sampsize. Tallene oppgis i den rekkefølgen som er angitt i variabelen `strata`, altså som i tabellen ovenfor. Her er altså kvinner vektet lavere enn menn, som burde øke accuracy for menn. Samtidig er kvinner med tilbakefall vektet litt høyere enn kvinner uten tilbakefall.

```
set.seed(45)
rf <- randomForest(Two_yr_Recidivism ~ .,
                   data = compas,
                   strata = strat,
                   sampsize = c(215, 290, 1500, 1500),
                   ntree=800)
```

[3]For de av dere som kan en del R fra før: `select()` ville gjort noe tilsvarende, men da ville objektet være en data.frame med én variabel fremfor en vektor. Forskjellen er minimal, men `randomForest()` vil ha det på denne måten.

Nå kan vi lage en prediksjon og se om modellen er mer rettferdig på samme måte som vi har gjort tidligere.

```
compas_p <- compas %>%
  mutate(pred_rf = predict(rf))

acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Sex',
                  preds        = 'pred_rf',
                  base         = 'Female')
acc[[2]]
```



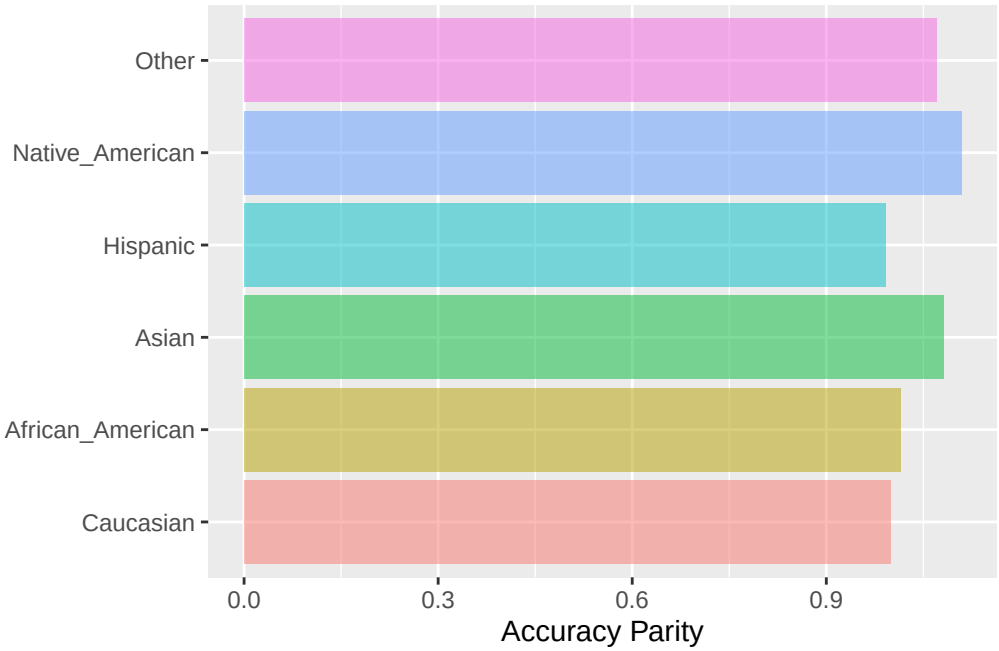
Magisk! Plutselig har vi endret en modell med innebygde kjønnsforskjeller til en kjønnsnøytral modell. I hvert fall på akkurat dette målet, da. Man bør jo også sjekke andre relevante mål på fairness.

Nå lurer du helt sikkert på hvordan du skal velge verdier for `sampsize` i et slikt tilfelle. Det litt skuffende svaret er at du må prøve deg litt frem. Men det hjelper å vite hvilket tall som betyr hva. I dette eksempelet sikres det at det trekkes omtrent like mange kvinner med og uten tilbakefall, og tilsvarende for menn. Så er det justert litt for å oversample kvinner med tilbakefall. Men for å være ærlig: jeg testet en god del varianter her før jeg fikk det resultatet jeg ville ha.

10.2.1 Hva andre subgrupper?

Så er spørsmålet om den balansering vi nå har gjort slår ut på andre grupper også. La oss se på etnisitet. Vi bruker den samme modellen som over, men bytter ut variabelen for kjønn med variabelen for etnisitet.

```
acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Ethnicity',
                  preds        = 'pred_rf',
                  base         = 'Caucasian')
acc[[2]]
```



Det er lov å bli litt skuffa nå. Dette ble jo ikke like pent balansert som for kjønn. Men det er jo ikke så rart, egentlig. Det er jo ikke gjort noe for å balansere på etnisitet.

En første mulighet er å gjøre en vurdering på om kjønnsbalanse eller etnisitet er viktigst. Kanskje man heller skulle fokusere på etnisitet? Det vil antakeligvis ikke være noen dum ide.

En annen mulighet er å stratifisere på begge variable samtidig. La oss undersøke muligheten ved å lage en tilsvarende stratifiserings-vektor og se på frekvensfordelingen.

```r
strat <- compas %>%
  mutate(strat = paste0(Sex, Ethnicity, Two_yr_Recidivism)) %>%
  pull(strat)

table(strat)
```

```
strat
FemaleAfrican_American0 FemaleAfrican_American1            FemaleAsian0
                    346                     203                       1
           FemaleAsian1        FemaleCaucasian0        FemaleCaucasian1
                      1                     312                     170
        FemaleHispanic0         FemaleHispanic1  FemaleNative_American1
                     56                      26                       2
           FemaleOther0            FemaleOther1   MaleAfrican_American0
                     47                      11                    1168
  MaleAfrican_American1              MaleAsian0              MaleAsian1
                   1458                      22                       7
         MaleCaucasian0          MaleCaucasian1           MaleHispanic0
                    969                     652                     264
          MaleHispanic1    MaleNative_American0    MaleNative_American1
                    163                       6                       3
             MaleOther0              MaleOther1
                    172                     113
```

Problemet nå er jo at noen grupper er veldig, veldig små. Å stratifiser på disse vil ikke fungere rett og slett. Det er som i annen statistikk: Estimere på veldig små grupper innebærer at det blir bare støy og tilfeldigheter i estimatene.

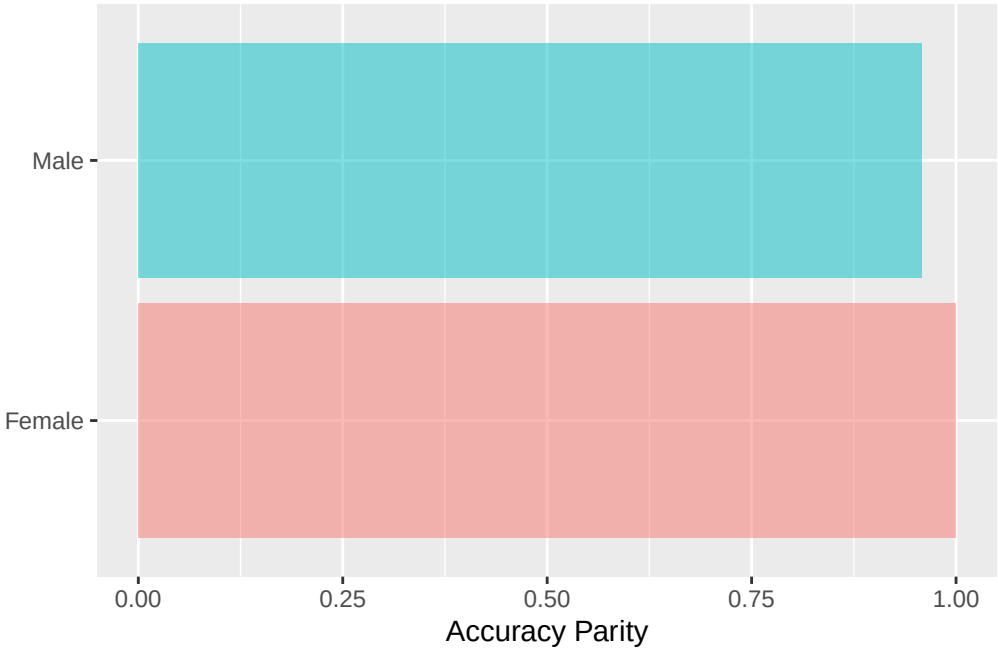
```r
strat <- compas %>%
  mutate(caucasian = ifelse(Ethnicity == "Caucasian", "Caucasian", "Other")) %>%
  mutate(strat = paste0(Sex, caucasian, Two_yr_Recidivism)) %>%
  pull(strat)

table(strat)
```

```
strat
FemaleCaucasian0 FemaleCaucasian1     FemaleOther0     FemaleOther1
             312              170              450              243
  MaleCaucasian0   MaleCaucasian1       MaleOther0       MaleOther1
             969              652             1632             1744
```

La oss prøve med dette.

```
set.seed(45)
rf <- randomForest(Two_yr_Recidivism ~ .,
                   data = compas,
                   strata = strat,
                   sampsize = c(120, 120,
                                200, 200,
                                400, 400,
                                900, 900),
                   ntree=800)

#rf
```

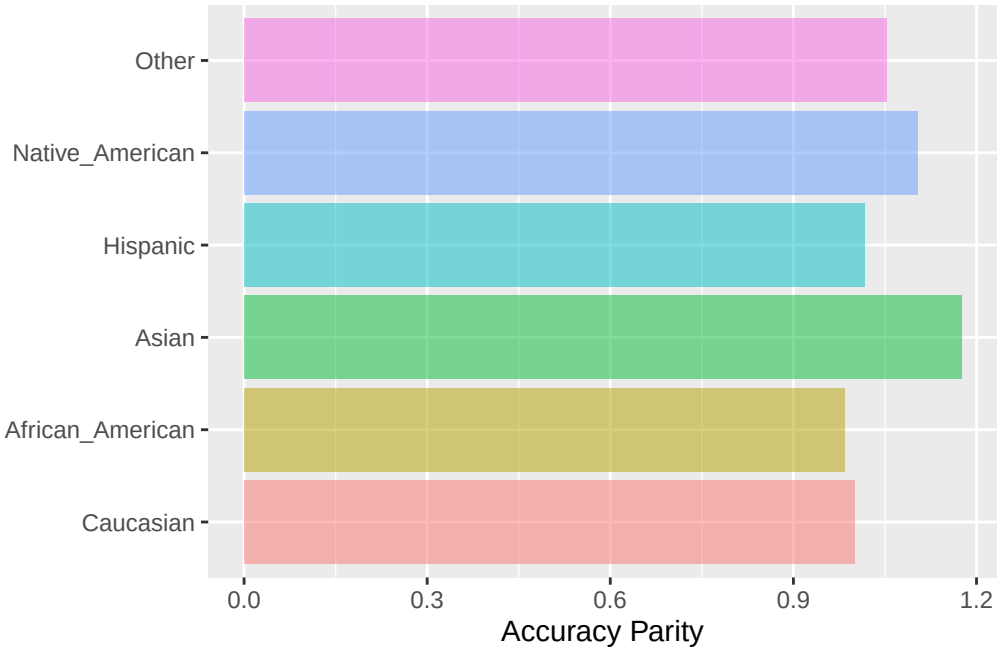
```
compas_p <- compas %>%
  mutate(pred_rf = predict(rf))

acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Sex',
                  preds        = 'pred_rf',
                  base         = 'Female')
acc[[2]]
```



Nå sjekker vi kjønnsbalansen igjen. Sammenlign med resultatet fra tidligere – ble det mer balansert?

La oss også sjekke om den samme modellen er rettferdig med hensyn på etnisitet:

```
acc <- acc_parity(data = compas_p,
                  outcome      = 'Two_yr_Recidivism',
                  group        = 'Ethnicity',
                  preds        = 'pred_rf',
                  base         = 'Caucasian')
acc[[2]]
```



Her ble det i hvert fall annerledes, men ikke helt likt nå heller. Nå kan man drive på en stund å justere og se hva man får til. Hvis man har et veldig stort datasett (som vi ikke har her) så kan man i teorien justere for flere ulike egenskaper samtidig og i større detalj. Man er rett og slett litt begrenset av dataene.

Problemet er selvsagt at det ikke går an å justere i det uendelige for alle variable. I tillegg kan det uansett være andre egenskaper du ikke har data for som det senere vil vise seg er urettferdig. Det kan f.eks. slå ut svært skjevt for type nabolag, sosioøkonomisk status, religiøs tilhørighet, lengde på håret - eller hva som helst annet. Man trenger data for å sjekke, men også data for å bygge algoritmen.

Jeg tror konklusjonen er at man justere for noe, men ikke alt. Men med nok data kan man justere for *mer* - men fremdeles ikke alt. I tillegg er som regel ikke justeringer gratis. Kanskje blir presisjonen i modellen totalt sett dårligere?

Så da er vi tilbake til det litt ubehagelige gjennomgangstemaet om prioriteringer og valg: Vil du f.eks. ha en rettferdig modell eller en presis modell? Hvilke grupper bør den være rettferdig for - og hvilke grupper er det ikke så farlig om den er urettferdig for? Dette kan lett være umulige valg. Men hva er så alternativet? Det er jo heller ikke sikkert er bedre på noen av disse parametrene.

11 Oppgaver

Her er alle oppgavene samlet. De er organisert etter kapittel.

11.1 Logistisk regresjon

Disse oppgavene vil være ganske tilsvarende som for oppgavene med lineær regresjon. Men du skal nå bruke logistisk regresjon med tilhørende teknikker og vurderinger.

Exercise 11.1. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer.

Exercise 11.2. Velg et datasettet og formuler hva en prediksjonsmodell kan kunne brukes til. Se for deg at tiltak du foreslår vil altså ha faktiske konsekvenser, så gjør en vurdering av hvorvidt feilprediksjoner vil være problematiske og i så fall på hvilken måte. Vurder mulighetene for feil opp mot gevinst ved riktig prediksjon.

Merk: det er ikke viktig at anvendelsen skal være realistisk, men du må alltid ta konsekvensen i vurderingene.

Exercise 11.3. Last inn valgte datasett og splitt i et training og et testing datasett. Sett splitten ved .70. Bruk training-data til å gjøre deg kjent med dataene og estimere modellene. Ikke bruk testing-dataene inntil du får beskjed om det.

Exercise 11.4. Gjør deg kjent med innholdet i disse training-dataene. Du kan gjøre f.eks. følgende:

a) Bruk `glimpse()` til å få oversikt over innholdet i datasettet
b) Hvis det er noen variable du ikke kommer til å bruke, slett gjerne disse med en gang
c) Lag noen tabeller og plot som viser hvordan utfallsvariabelen er fordelt etter andre variable

Exercise 11.5. Estimer flere logistiske regresjonsmodeller med et fåtall prediktorer. Gjør et utvalg av de variablene du mener er mest relevant for å forklare utfallet. Estimer flere regresjonsmodeller for å predikere utfallet, og sammenlign hvor gode prediksjoner disse gir. Mest relevante statistikk er AUC og ROC-curve.

a) Velg ut tre forklaringsvariable og estimer en regresjonsmodell
b) Estimer en ny modell med alle variable i datasettet
c) Estimer en ny modell og inkluder noen få polynomer og/eller interaksjonsledd

Lag gjerne noen plot av ROC-curve for i hvert fall noen av modellene slik at du får en følelse med hva AUC egentlig betyr. Plot også predikert verdi mot observert verdi og gjør en vurdering av RMSE.

Exercise 11.6. I forrige oppgave brukte du training-datasettet til både å estimere modellene og vurdere resultatet. Nå skal du bruke testing-datasettet til å vurdere de samme resultatene. Dette gjør du ved å predikere på testing-datasettet og regne ut AUC for disse dataene. For hver modell i forrige oppgave, gjør som følger:

a) Prediker utfallet på testing-datasettet
b) Regn ut AUC
c) Hvor stor er *endringen* i AUC er fra resultatene når du brukte training-datasettet?

Vurdering: En mer komplisert modell beskriver dataene bedre. Men er det like stor *endring* i AUC og RMSE for enkle og mer kompliserte modeller? Beskriv hva du ser og gi en forklaring.

Exercise 11.7. Når over har predikert en sannsynlighet og regnet ut AUC har du ennå ikke tatt noen avgjørelse. Bestem deg for et grenseverdi for når du vil klassifisere som det ene eller andre. (Alså: ved hvilken sannsynlighet). Gjør så en klassifikasjon og lag en confusion matrix. Gi en vurdering av resultatet.

11.2 Klassifikasjonstrær

Exercise 11.8. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer.

Exercise 11.9. Prøv deg frem med å justere tuning-parametrene `cp`, `minbucket` og `maxdepth`. Se hvordan treet endrer seg og sammenlign confusion matrix for ulike varianter. Bruk training-data til å justere og testing-data til sluttevaluering.

Exercise 11.10. Sammenlign resultatet fra klassifikasjonstreet med logistisk regresjon (baseline). Hvilken modell gir best prediksjon? Vurder accuracy, sensitivity og specificity.

Exercise 11.11. Juster loss matrix for å endre forholdet mellom falske positive og falske negative. Tenk deg en praktisk anvendelse og vurder hva slags feilrater som er akseptable.

Exercise 11.12. Bruk `rpart.rules()` til å skrive ut reglene fra et klassifikasjonstre. Gi en tolkning av reglene – hvem blir klassifisert som hva, og gir det mening?

11.3 Bagging

Exercise 11.13. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer.

Exercise 11.14. Prøv deg litt frem med å angi hvor mange trær som skal bygges. Sammenlign modell med og uten OOB. Bruk `set.seed()` slik at evt. forskjeller i resultatene ikke skyldes tilfeldigheter.

Exercise 11.15. Velg et nytt datasett og gjør en tilsvarende analyse med bagging. Vurder confusion matrix og sammenlign med et enkelt klassifikasjonstre.

Exercise 11.16. Gjør en prediksjon med bruk av bagging og vurder confusion matrix. Prøv deg litt frem med å justere på tuningparametrene, f.eks. `cp` og `maxdepth` via `control = rpart.control()`.

Exercise 11.17. Forklar for en medstudent (eller andre) hva OOB er for noe. Hva er forskjellen på å splitte i training/testing datasett og å bruke OOB?

11.4 Random forest

Exercise 11.18. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer. Men gjør én endring: split datasettet først i testing og training.

Exercise 11.19. Prøv deg frem med å justere `sampsize`, `mtry` og `ntree`. Se hvordan feilratene endrer seg. Gjør deg først opp en mening om hvordan du vil at confusion matrix skal se ut (f.eks. cost-ratio) og prøv å nærme deg dette.

Exercise 11.20. Tolk random forest: a) Hvilke variable har størst prediktiv verdi? Lag et variable importance plot og gi en tolkning. a) Velg noen av variablene (gjerne f.eks. de med størst prediktiv verdi) og lag partial dependence plot.

Exercise 11.21. Bruk de syntetiske dataene og håndter manglende verdier med `replace_na()`. Sammenlign resultater med ulike konfigurasjoner av `sampsize`. Hvilken konfigurasjon gir best balanse mellom sensitivity og specificity?

11.5 Intro til fairness

Exercise 11.22. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer.

Exercise 11.23. Regn ut minst tre ulike mål på fairness og velg selv over hvilke grupper. Gi en forklaring på hva hver av dem betyr.

Exercise 11.24. Det er mange muligheter her: ulike mål og flere grupper. Man kan også kombinere grupper på flere måter. Gjør følgende vurderinger:

- Er det rimelig å gjøre en prediksjon som gir like resultater på tvers av alle mål og grupper? Kan du i det hele tatt få en "fair" modell?
- Hvilke mål på "fairness" vil du si er viktigst i dette eksempelet? Hvorfor?

Exercise 11.25. Kanskje hjelper det å estimere en annen modell? Estimer en ny logistisk regresjon, men velg bare et fåtall variable som du velger selv. Hold det enkelt i første omgang. Se på resultatene og vurder:

- Ble accuracy bedre eller verre?
- Ble resultatet mer "fair"?

Exercise 11.26. Velg et nytt datasett, gjør en prediksjon med logistisk regresjon og regn ut mål på "fairness" igjen. Gjør tilsvarende som over.

11.5.1 Refleksjonsoppgaver

Exercise 11.27. Studien til Caspi et al. (2017) bruker regresjon for å predikere hvem som tilhører den gruppen med konsentrasjon av sosiale problemer. (Det er en litt annen type modell enn logisk regresjon, men det spiller egentlig liten rolle). Diskuter følgende:

- Hva slags informasjon synes du *mangler* for å vurdere treffsikkerheten på prediksjonsmodellen?
- Hva slags tiltak er det tenkt å settes i verk når man har identifisert hvem som får problemer i voksen alder?
- Basert på hvordan prediksjonen skal brukes hva tenker du er akseptable feilrater?
- Er det behov for å vurdere "fairness" grundigere her?

Exercise 11.28. Studien til R. A. Berk, Sorenson, and Barnes (2016) bruker andre typer modeller enn vi har sett på til nå. Se bort fra akkurat det tekniske i modeller og justeringer, men se diskusjonen av resultatet presentert som confusion matrix. Se for deg at dette blir de faktiske resultatene for fengsling eller ikke-fengsling og vurder følgende:

- Synes du at forholdstallet 1 til 10 for falske positive/negative er ok i denne settingen? Hvis du skal foreslå noe annet, hva vil du si da?
- Er det grunn til å tro at man bør undersøke "fairness" grundigere her? I så fall: hvilke undergrupper mener du er mest viktig å undersøke dette for?

11.6 Fairness og tuning

Exercise 11.29. Gå gjennom eksempelet over og sjekk at det fungerer og at du skjønner hvert steg.

Exercise 11.30. Velg deg et annet datasett og gjør tilsvarende analyse. Det er viktig at du nå gjør en vurdering av hvilke undergrupper du synes er viktigst å motvirke urettferdighet for. Her bør du ta hensyn også til hva slags tiltak og konsekvenser det er snakk om! Skriv ned dine vurderinger om dette som begrunnelse for hvordan du vil motvirke urettferdighet i modellen.

a) Vurder hvilke konsekvenser prediksjonen kan få, med henblikk på et tenkt praktisk tiltak. Hvilke feilrater vil du akseptere? Ta et valg.
b) Estimer modellen som tidligere, lag en confusion matrix og vurder resultatet. Juster modellen til du er fornøyd med resultatet.
c) Vurder hvilken undergruppe du synes det er viktigst at ikke blir biased.
d) Hvilket mål på fairness synes du er viktigst i akkurat dette tilfellet? Velg ett mål, begrunn det valget og sjekk.
e) Estimer modellen på nytt med stratifisert utvalg og justert `sampsize`. Prøv deg frem til du får et akseptabelt mål på fairness.
f) Sjekk confusion matrix og sammenlign med den første modellen. Modellen ble kanskje mer rettferdig, men gikk f.eks. accuracy ned eller endrede feilrater? Eller litt flåsete sagt: Var det verd å få en mer rettferdig modell?
g) Sjekk nå det samme målet på fairness med to andre undergrupper. Ble det like bra?

Exercise 11.31. Velg et annet datasett og gjør tilsvarende øvelse.

11.7 Boosting

Exercise 11.32. Se over de vurderingene du gjorde i tidligere analyser av compas-dataene, herunder hva slags feilrater du er villig til å akseptere og hva slags konsekvenser de ulike typene feil kan få. Vil du gjøre andre vurderinger nå? Ta stilling til om du fremdeles synes tidligere vurderinger var ok. Bestem deg for hvor mange falske positive du er villig til å godta per falske negative (dvs asymetriske kostnader). Bruk dette videre i neste oppgave.

Exercise 11.33. Gå gjennom eksempelet over og repliker disse analysene slik at du ser at du skjønner hvordan det fungerer. Men du må gjøre tre endringer:

1) Vurder asymetriske konstnader, spesielt cost-ratio (dvs. forholdet mellom falske positive og falske negative). Se om du klarer å justere modellen så du får dette resultatet.
2) Sjekk mål på fairness også for etnisitet. Test ut flere forskjellige mål fra fairness-pakken. Bruk fortsatt *training*datasettet.
3) Etter at du har kjørt gjennom alt med *training*datasettet må du sjekke resultatet mot *testing*datasettet. Ble det vesentlig endring i resultatene?

Exercise 11.34. Bruk et annet datasett og gjør tilsvarende analyser. Husk å få med følgende deler:

a) Vurdering av konsekvenser av feil: hva vil du godta? Begrunn svaret med et tenkt tiltak som skal settes i verk.
b) Tilpass modellen, og vurder om du har nok iterasjoner (antall runder i boostingen). Øk ved behov.
c) Prediker, lag confusion matrix og sammenlign med hva du i utgangspunktet bestemte deg for å godta. Hvis det trengs endringer, juster modellen på nytt til du er mer fornøyd.
d) Kommenter hvilke variable som er mest betydningsfulle for prediksjonen og på hvilken måte.
e) Hvilke mål på fairness synes du er mest relevant her og for hvilke undergrupper? Sjekk dette empirisk.
f) Prøv å justere modellen til et mer tilfredsstillende rettferdig resultat for valgte undergrupper.
g) Sjekk nå resultatene (inkl. fairness) for *testing*dataene også.
h) Vil du sette i verk tiltak basert på en slik prediksjonsmodell? Begrunn svaret.

References

Berk, Richard. 2016. *Statistical Learning from a Regression Perspective*. USA: Springer.

Berk, Richard A., Susan B. Sorenson, and Geoffrey Barnes. 2016. "Forecasting Domestic Violence: A Machine Learning Approach to Help Inform Arraignment Decisions." *Journal of Empirical Legal Studies* 13: 94–115. https://doi.org/10.1111/jels.12098.

Caspi, Avshalom, Renate M. Houts, Daniel W. Belsky, Honalee Harrington, Sean Hogan, Sandhya Ramrakha, Richie Poulton, and Terrie E. Moffitt. 2017. "Childhood Forecasting of a Small Segment of the Population with Large Economic Burden." *Nature Human Behavior* 1 (5): 223–40. https://doi.org/https://doi-org.ezproxy.uio.no/10.1038/s41562-016-0005.

Hsieh, John. 2008. "Receiver Operating Characteristic (ROC) Curve." In *Encyclopedia of Epidemiology*, 895–98. Thousand Oaks, California: Sage. https://doi.org/10.4135/9781412953948.